NoFeeSwap™ [Draft]

Yellow Paper*

February 22, 2025

Draft - NoFeeSwap

Abstract

NoFeeSwap is an Automated Market Making (AMM) protocol with several benefits compared to other Decentralized Exchanges (DEXs). In particular, it offers **zero spread between the buy and sell marginal prices**, while, at the same time, **guarantees liquidity growth for Liquidity Providers (LPs) after each swap**. This enables **higher trading activity, even after small fluctuations of the market price**, and, consequently, **higher profitability for LPs**. Various theoretical and empirical arguments are provided to demonstrate superior profitability for LPs compared to other platforms. The protocol has **inherent compounding for liquidity growth**, while still offering LPs the ability to concentrate liquidity within their desired ranges. **A wide variety of innovative customizations ensure utmost flexibility of liquidity distribution for LPs**. This document explains concepts that are central to the protocol framework and provides proofs for the efficacy and security of the protocol. Lastly, the description of smart contract methods, the initial incentive program, and tokenomics are covered in this document.

*Website: www.nofeeswap.org
Contact: contact@nofeeswap.org
US Patent Pending
Copyright 2025, NoFeeSwap LLC - All rights reserved.

1 Introduction

NoFeeSwap is a customizable Automated Market Making (AMM) protocol where traders do not pay fees and there is an inherent zero gap between the buy and sell marginal prices.

NoFeeSwap platform consists of liquidity pools. Each pool is dedicated to swapping between a pair of tokens, $\boldsymbol{x}$ and $\boldsymbol{y}$. Liquidity Providers (LPs) contribute to token reserves in order to grow their liquidity ownership. Furthermore, traders use the available reserves to swap one token for the other.

NoFeeSwap offers LPs the ability to concentrate their liquidity within the desired price ranges. To facilitate that, each liquidity pool is partitioned into a sequence of price intervals. LPs make a profit from the swaps resulting from price fluctuations within their selected range.

Consider a liquidity pool dedicated to the swap of tokens $\boldsymbol{x}$ and $\boldsymbol{y}$. Let $\mathbb{Z}_+ \triangleq \{1, 2, \ldots\}$ denote a sequence of swaps where each $t \in \mathbb{Z}_+$ represents the pool's state before the tth swap.[1] Denote the set of non-negative real numbers by $\mathbb{R}_+$. For each $t \in \mathbb{Z}_+$, let the pair $(x_t, y_t) \in \mathbb{R}_+^2$ represent the reserve amounts for tokens $\boldsymbol{x}$ and $\boldsymbol{y}$ before the tth trade.[2] A trader, through the tth trade, moves the reserve values to the point $(x_{t+1}, y_{t+1}) \in \mathbb{R}_+^2$ by exchanging $\delta x = x_{t+1} - x_t$ amount of token $\boldsymbol{x}$ and $\delta y = y_{t+1} - y_t$ amount of token $\boldsymbol{y}$, where the exchange amounts, δx and δy, are subject to a mathematical formula which is referred to as the *curve*.[3] The remainder of Section 1 discusses the following questions.

How to build a protocol that follows the market price in an automated fashion without external input from the LPs? Section 1.2 provides such a framework after the more introductory formulation of Section 1.1. How to make the platform profitable for LPs using the buy/sell spread? Section 1.3 explains the addition of a fee and its drawbacks. Why is it helpful to avoid the buy/sell spread? Section 1.3 briefly discusses this as well. How to avoid buy/sell spread? NoFeeSwap protocol is an answer.

Section 2 introduces a variety of tools and techniques. In Section 3, the NoFeeSwap protocol is discussed in detail, including references to the implementations. Section 4 explains the protocol's governance and tokenomics. Section 5 discusses an operator for interacting with the protocol. An incentive reward program is built as a hook and is discussed in Section 6.

[1] $t = 1$ represents the initial state of the pool.

[2] The terms *trade* and *swap* are used interchangeably in this document.

[3] The difference amount is positive for the token that is taken by the protocol and is negative for the other token that is given by the protocol.

Draft - NoFeeSwap

See NoFeeSwap's litepaper [1] for discussions on the advantages and the desired features of the protocol for various market participants and community members.

1.1 Constant Price Liquidity Pool

Consider a simple protocol which is governed by a constant price $\check{p}$. In other words, traders are allowed to exchange any arbitrary amount of token $\boldsymbol{y}$ with token $\boldsymbol{x}$ at price $\check{p}$, subject to availability. More precisely, a trader can move the liquidity reserve values from $(x_t, y_t) \in \mathbb{R}_+^2$ to $(x_{t+1}, y_{t+1}) \in \mathbb{R}_+^2$, if and only if:

$$\check{p}\, x_{t+1} + y_{t+1} = \check{p}\, x_t + y_t. \tag{1}$$

In other words, $\check{p} = \frac{y_t - y_{t+1}}{x_{t+1} - x_t}$. Given the market price of $\check{p}_t$, a rational trader/arbitrageur is incentivized to solve the following problem in order to maximize their profits:

$$\underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} \qquad \check{p}_t\,(x_{t+1} - x_t) + (y_{t+1} - y_t) \tag{2a}$$

$$\text{subject to} \qquad \check{p}\, x_{t+1} + y_{t+1} = \check{p}\, x_t + y_t. \tag{2b}$$

In simple terms, the objective is to determine the optimal values for x_{t+1} and y_{t+1} to minimize the loss (2a) (maximize the profit) subject to (2b). Observe that the objective function represents the total loss of the trader in terms of token $\boldsymbol{y}$, which is supposed to be a negative number because zero loss is attainable through $(x_{t+1}, y_{t+1}) = (x_t, y_t)$[4]. It is straightforward to verify that the answer to problem (2) is:

$$x_{t+1} = \begin{cases} 0 & \check{p} < \check{p}_t \\ x_t + \check{p}^{-1} y_t & \check{p} > \check{p}_t \end{cases}, \qquad y_{t+1} = \begin{cases} \check{p}\, x_t + y_t & \check{p} < \check{p}_t \\ 0 & \check{p} > \check{p}_t \end{cases}. \tag{3}$$

In other words, the token sold below the market price is depleted from the reserve.[5] This is illustrated in Figure 1.

This rudimentary trading protocol does not follow the market price and is not profitable for LPs. Section 1.2 deals with the first issue and Section 1.3 with the latter.

[4] which corresponds to zero swap

[5] If $\check{p}_t = \check{p}$, the trader's profit is zero.

Draft - NoFeeSwap

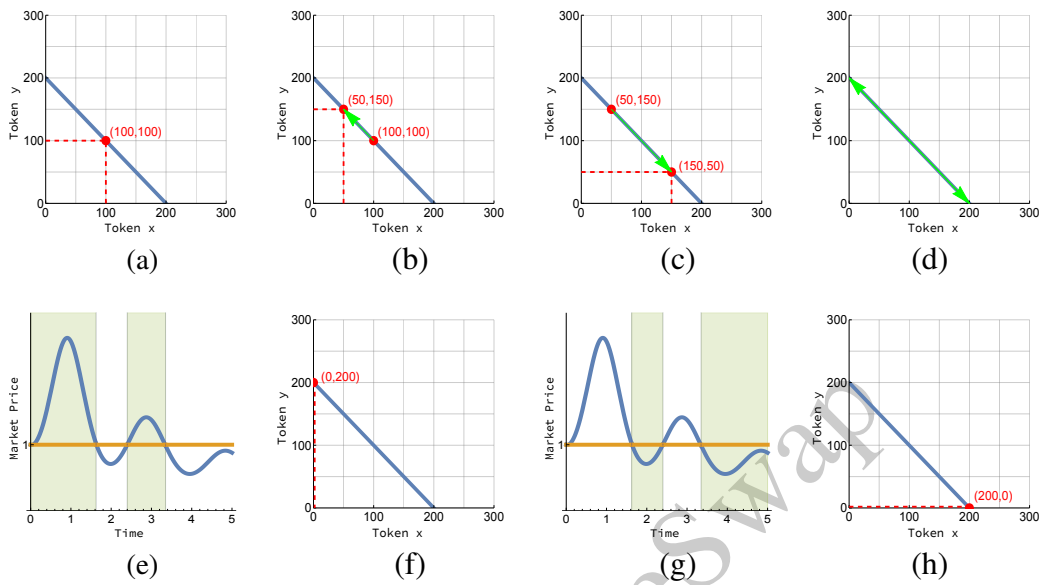


Figure 1: A constant price liquidity pool: (a) Consider 100 units of reserve for the tokens $\boldsymbol{x}$ and $\boldsymbol{y}$ with a constant price of 1; (b) A trader can move reserve values from the point $(100, 100)$ to the point $(50, 150)$ by trading 50 units of token $\boldsymbol{y}$ in exchange for 50 units of token $\boldsymbol{x}$; (c) Another trader can then move the reserve values from point $(50, 150)$ to point $(150, 50)$ by trading 100 units of token $\boldsymbol{x}$ in exchange for 100 units of token $\boldsymbol{y}$; (d) Traders are allowed to move the pool's reserve values to any point that belongs to the highlighted segment; (e) During the time when the market price exceeds 1 (the highlighted green area), rational traders are incentivized to buy the pool's entire reserve of token $\boldsymbol{x}$; (f) In this case, traders move the pool's reserve values to the most left corner of the segment, i.e., the point $(0, 200)$; (g) During the time when the market price is below 1 (the highlighted green area), rational traders are incentivized to buy the whole reserve of the $\boldsymbol{y}$ token in the pool; (h) In this case, traders move the reserve values to the right corner of the segment, i.e., the point $(200, 0)$.

Draft - NoFeeSwap

1.2 Following the Market Price

In this section, a variety of AMM frameworks are discussed that follow the market price without requiring any input from the LPs. To this end, consider a *strictly concave* and *differentiable* function $c : \mathbb{R}^2_+ \to \mathbb{R}$, where a trader is permitted to

move the reserve values from (x_t, y_t) to (x_{t+1}, y_{t+1}), if and only if:

$$c(x_{t+1}, y_{t+1}) = c(x_t, y_t). \tag{4}$$

Observe that the previously discussed constant price liquidity pool corresponds to $c(x, y) = \check{p}\, x + y$. However, the additional requirement of strict concavity incentivizes rational traders to synchronize the pool's marginal price with the market price. To verify this, let us calculate the marginal price of this protocol for an infinitesimal trade. If $x_{t+1} - x_t \to 0$ and $y_{t+1} - y_t \to 0$, then according to Taylor series expansion:

$$\begin{aligned} c(x_t, y_t) &= c(x_{t+1}, y_{t+1}) \\ &\approx c(x_t, y_t) + c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \tag{5}$$

which results in the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = \frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \tag{6}$$

Now, given the market price of $\check{p}_t$, consider a rational trader that intends to maximize their profit. This trader solves the following:

$$\underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} \quad \check{p}_t\,(x_{t+1} - x_t) + (y_{t+1} - y_t) \tag{7a}$$

$$\text{subject to} \quad c(x_{t+1}, y_{t+1}) = c(x_t, y_t). \tag{7b}$$

Due to continuity of c, one may relax constraint (7b) to an inequality, without affecting the optimal solution:[6]

$$\underset{x_{t+1}, y_{t+1} \geq 0}{\text{minimize}} \quad \check{p}_t\,(x_{t+1} - x_t) + (y_{t+1} - y_t) \tag{8a}$$

$$\text{subject to} \quad c(x_{t+1}, y_{t+1}) \geq c(x_t, y_t). \tag{8b}$$

The optimal solution satisfies the following stationarity conditions:

$$\check{p}_t + \lambda^* c^{(1,0)}(x^*_{t+1}, y^*_{t+1}) = 0, \tag{9}$$

$$1 + \lambda^* c^{(0,1)}(x^*_{t+1}, y^*_{t+1}) = 0, \tag{10}$$

[6] If (x^*_{t+1}, y^*_{t+1}) is an optimal solution and $c(x^*_{t+1}, y^*_{t+1}) > c(x_t, y_t)$, then for a sufficiently small $\varepsilon > 0$, $(x^*_{t+1} - \varepsilon, y^*_{t+1} - \varepsilon)$ would be feasible with a lower cost value, which contradicts the optimality of (x^*_{t+1}, y^*_{t+1}). Hence, $c(x^*_{t+1}, y^*_{t+1}) = c(x_t, y_t)$ holds for every optimal pair (x^*_{t+1}, y^*_{t+1}), and the transformation of equality into inequality is lossless.

Draft - NoFeeSwap

where λ^* is an optimal Lagrange multiplier and (x^*_{t+1}, y^*_{t+1}) is an optimal primal solution. Hence, canceling λ^* concludes:

$$\frac{c^{(1,0)}(x^*_{t+1}, y^*_{t+1})}{c^{(0,1)}(x^*_{t+1}, y^*_{t+1})} = \check{p}_t, \tag{11}$$

which means that, as a result of the profit-motivated trader's action, the pool's updated marginal price, $\frac{c^{(1,0)}(x_{t+1},y_{t+1})}{c^{(0,1)}(x_{t+1},y_{t+1})}$, is now synchronized with the market price, $\check{p}_t$.

A classical example of such a structure [2] is the constant product market setting framework for which $c(x, y) = xy$.[7] This framework is illustrated in Figure 2.

1.3 Trading Fees and Profitability for LPs

In this section, we focus on the profitability of the protocol for LPs. A popular approach[8] is to impose a fee ratio $0 \leq f < 1$ by allowing transactions that satisfy:

$$\begin{cases} c\big(x_t + (1-f)(x_{t+1} - x_t), y_{t+1}\big) = c(x_t, y_t) & \text{if} \quad x_{t+1} > x_t,\ y_{t+1} < y_t, \\ c\big(x_{t+1}, y_t + (1-f)(y_{t+1} - y_t)\big) = c(x_t, y_t) & \text{if} \quad x_{t+1} < x_t,\ y_{t+1} > y_t. \end{cases} \tag{12}$$

In other words, when the token $\boldsymbol{x}$ is traded in exchange for the token $\boldsymbol{y}$, the pool confiscates an f portion of the incoming amount $x_{t+1} - x_t$ as a fee. Similarly, when the token $\boldsymbol{y}$ is traded in exchange for the token $\boldsymbol{x}$, the pool confiscates an f portion of the incoming amount $y_{t+1} - y_t$ as a fee.

In this case, effectively, there are two distinct marginal prices, one for the case of the pool buying token $\boldsymbol{x}$ and one for the case of the pool buying token $\boldsymbol{y}$. In order to derive the first one, we resort to Taylor expansion again:

$$\begin{aligned} 0 &= c\big(x_t + (1-f)(x_{t+1} - x_t), y_{t+1}\big) - c(x_t, y_t) \\ &\approx (1-f)c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \tag{13}$$

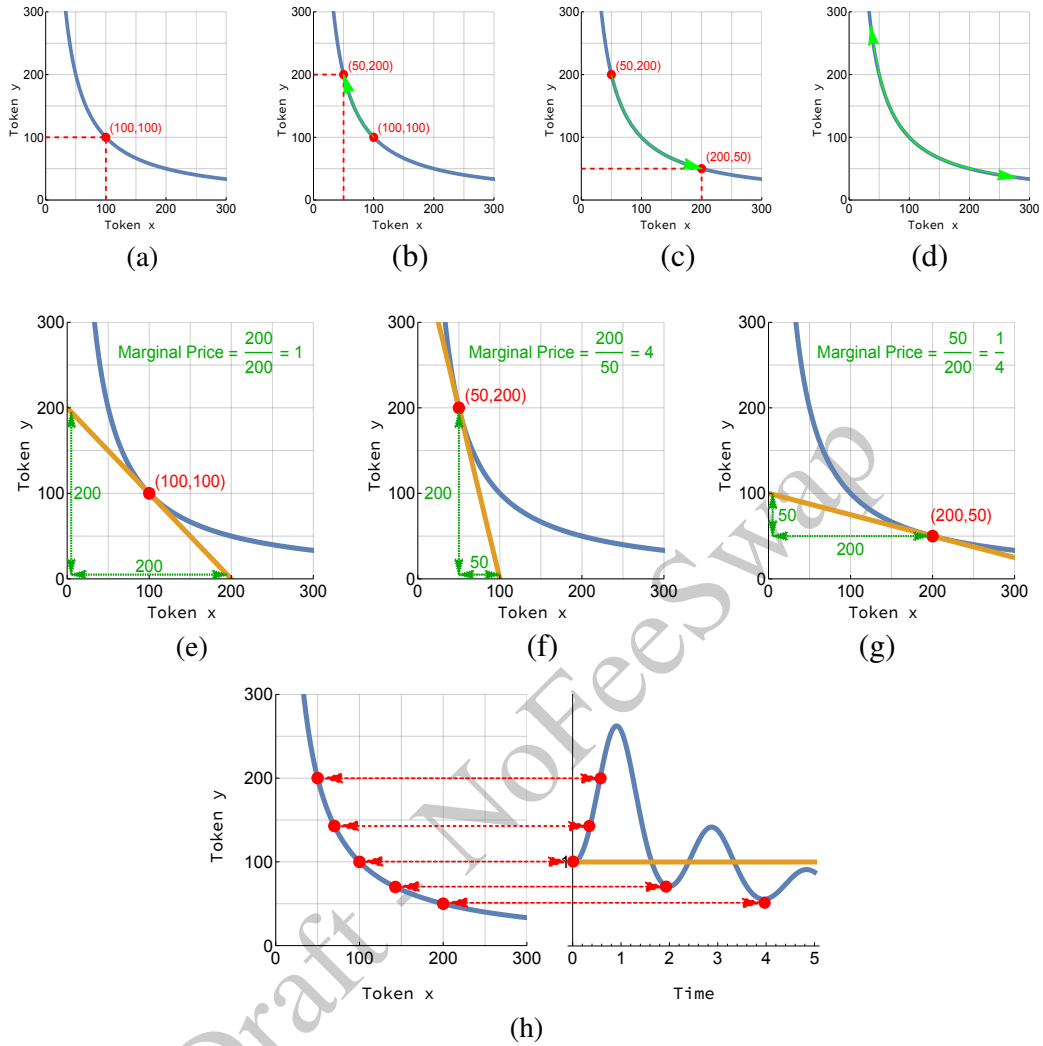
which leads to the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = (1-f)\frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \tag{14}$$

[7] In the context of cryptocurrency, the first mention of decentralized market making is by Ethereum's Vitalik Buterin. Martin Koppelmann of Gnosis later expanded on this idea and proposed a constant product AMM pricing scheme.

[8] and indeed the only approach before the NoFeeSwap protocol

Draft - NoFeeSwap



Draft - NoFeeSwap

Figure 2: A constant product liquidity pool: (a) Consider 100 units of reserve for both tokens $\boldsymbol{x}$ and $\boldsymbol{y}$; (b) A trader can move the reserve values from point $(100, 100)$ to point $(50, 200)$ by trading 100 units of token $\boldsymbol{y}$ in exchange for 50 units of token $\boldsymbol{x}$; (c) Another trader can then move the reserve values from point $(50, 200)$ to point $(200, 50)$ by trading 150 units of token $\boldsymbol{x}$ in exchange for 150 units of token $\boldsymbol{y}$; (d) Traders are allowed to move the pool's reserve values to any point that belongs to curve $xy = 10000$; (e), (f) and (g) illustrate the resulting marginal price changes as a result of the reserve movements; (h) The primary benefit of this protocol is that rational traders are always incentivized to adjust the pool's marginal price in accordance with the market price. As the token prices fluctuate (right-side), the pool's marginal price adjusts accordingly (left-side).

Similarly, when the pool sells token $\boldsymbol{x}$, the marginal price is derived as:

$$\begin{aligned} 0 &= c\big(x_{t+1}, y_t + (1-f)(y_{t+1} - y_t)\big) - c(x_t, y_t) \\ &\approx c^{(1,0)}(x_t, y_t)(x_{t+1} - x_t) + (1-f)c^{(0,1)}(x_t, y_t)(y_{t+1} - y_t), \end{aligned} \tag{15}$$

which leads to the marginal price of:

$$\frac{y_t - y_{t+1}}{x_{t+1} - x_t} = \frac{1}{1-f}\frac{c^{(1,0)}(x_t, y_t)}{c^{(0,1)}(x_t, y_t)}. \tag{16}$$

As demonstrated in Figure 3, the marginal prices of buy and sell are different here. Their gap is referred to as the *spread*. This results in a *liquidity growth* after each swap, which leads to profitability for LPs.

Remark 1. *Note that the buy and sell marginal prices are different, i.e., the curves are not differentiable. Also, the curves from before and after the update are not tangent to each other.*

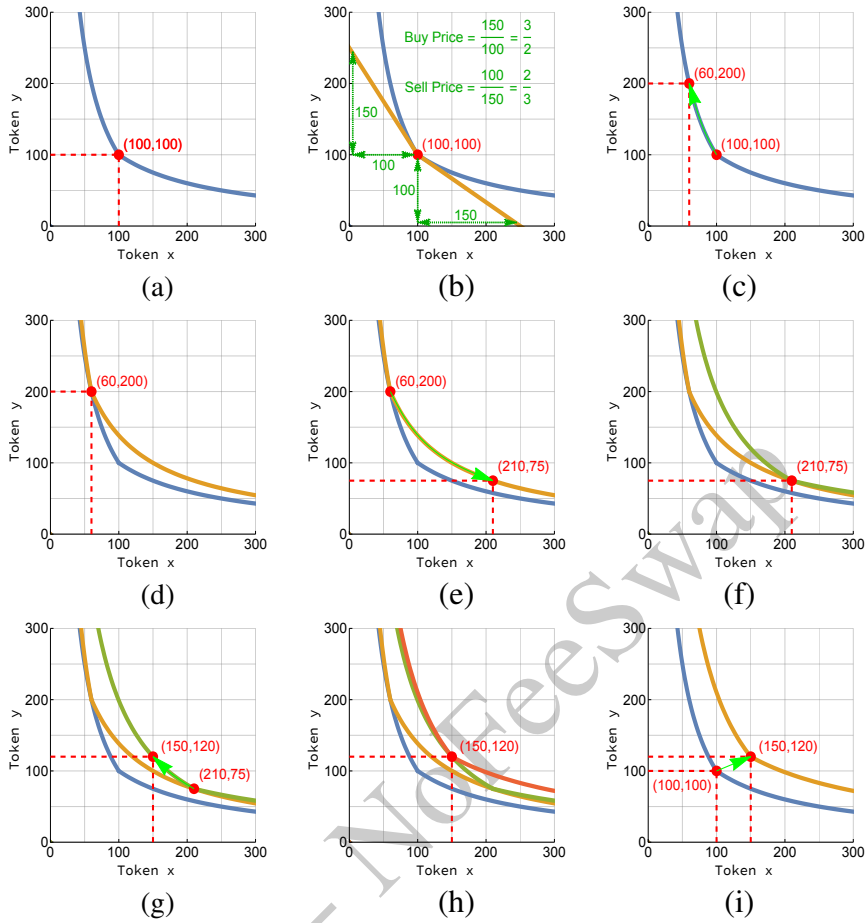
Despite the profitability of LPs, a drawback of incorporating fees is that no arbitrage trade takes place for small fluctuations in market price, which negatively impacts profits for LPs. More precisely,

- Fluctuations in the market price incentivize arbitrageurs to perform trades and synchronize pool prices with the market price.
- Trades result in liquidity growth and generate profit for LPs.
- Small price fluctuations are much more likely and more frequent than major price changes.
- If the market price falls within a pool's marginal spread interval, there would be no incentive for the arbitrageurs to trade in that pool.

Due to the above arguments, the existence of fees decreases arbitrage activities and the corresponding profits for LPs. Because of this, fee ratios are expected to be small in AMM pools. The second undesired consequence of fees is the corresponding price staleness. Last but not least, many traders, and particularly retail traders, do not consider trading in protocols/pools with significant fees altogether.

Given the above arguments, a question arises as to whether one can design an LP-profitable AMM platform with zero marginal spread, resulting in superior trade activity and profitability for LPs. Addressing this question is one of the core objectives of the NoFeeSwap protocol.

Draft - NoFeeSwap



Draft - NoFeeSwap

Figure 3: A constant product liquidity pool with an exaggerated $\frac{1}{3}$ fee ratio. The fee ratio typically does not exceed 1%. However, for visualization purposes, a high fee ratio is considered here: (a) Consider 100 units of reserve for each of the tokens $\boldsymbol{x}$ and $\boldsymbol{y}$; (b) Due to the presence of fees, the resulting curve is not smooth, which results in a gap between the buy and sell marginal prices; (c) A trader can move the reserve values from point $(100, 100)$ to point $(60, 200)$ by trading 100 units of token $\boldsymbol{y}$ in exchange for 40 units of token $\boldsymbol{x}$; (d) The curve then evolves into a new one; (e) Another trader can then move the reserve values from point $(60, 200)$ to point $(210, 75)$ by trading 150 units of token $\boldsymbol{x}$ in exchange for 125 units of token $\boldsymbol{y}$; (f) The curve then evolves into a new one; (g) Another trader may then move the reserve values from the point $(210, 75)$ to the point $(150, 120)$ by trading 45 units of token $\boldsymbol{y}$ in exchange for 60 units of token $\boldsymbol{x}$; (h) The curve then evolves into a new one; (i) This is profitable for LPs as the position of the pool changed from the initial point $(100, 100)$ to the point $(150, 120)$ with an overall profit of the 50 token $\boldsymbol{x}$ and the 20 token $\boldsymbol{y}$.

1.4 Concentrated Liquidity and Price Intervals

As explained before, a trade is tantamount to moving the price. An arbitrageur is incentivized to synchronize the price of a liquidity pool with the market price to gain profit. The higher the amount of liquidity in a pool, a fixed change in the market price results in larger trades in terms of tokens $\boldsymbol{x}$ and $\boldsymbol{y}$. Additionally, the amount of profit from a liquidity growth for each LP is proportional to their liquidity.

One of the main drawbacks of protocols without concentrated liquidity is that the deposited liquidity is distributed over all price values within the range $(0, \infty)$. This is an inherent inefficiency that is the focus of platforms such as Uniswap v3 [3] and Curve.fi [4]. For instance, consider two stable tokens, whose values, by definition, do not fluctuate significantly. The price of one token with respect to the other one is expected to remain within a small vicinity of a fixed value. A question arises as to whether an LP can concentrate their liquidity in that particular vicinity as opposed to the entire range $(0, \infty)$, to gain more profit. To this end, Uniswap v3 modified the constant product scheme by employing virtual reserves, with a drawback of losing the growth compounding feature.

NoFeeSwap offers concentrated liquidity by partitioning the price horizon $(0, \infty)$ into custom intervals and allowing LPs to concentrate their liquidity to any set of intervals of their choosing. This is further illustrated in the next section, where various examples of liquidity distribution are explained.[9]

Draft - NoFeeSwap

2 Zero Spread Market Making

In this section, a class of LP-profitable AMM protocols with zero marginal spread is constructed. For any arbitrary $t \in \mathbb{Z}_+$, consider a *strictly concave* and *differentiable* function $c_t : \mathbb{R}_+^2 \to \mathbb{R}$. A trader is permitted to move the reserve values from (x_t, y_t) to (x_{t+1}, y_{t+1}) if and only if:

$$c_t(x_{t+1}, y_{t+1}) = c_t(x_t, y_t). \tag{17}$$

[9] In the previous examples, there was no concentrated liquidity.

After this trade, a new function $c_{t+1} : \mathbb{R}_+^2 \to \mathbb{R}$ is generated that satisfies the following:

$$c_{t+1}(x_{t+1}, y_{t+1}) = c_t(x_{t+1}, y_{t+1}), \tag{18}$$

$$\frac{c_{t+1}^{(1,0)}(x_{t+1}, y_{t+1})}{c_{t+1}^{(0,1)}(x_{t+1}, y_{t+1})} = \frac{c_t^{(1,0)}(x_{t+1}, y_{t+1})}{c_t^{(0,1)}(x_{t+1}, y_{t+1})}. \tag{19}$$

This procedure is illustrated in Figure 4. According to the differentiability of each function c_t, the marginal prices of buy and sell are identical. Furthermore, requirement (18) ensures the *conservation* of tokens, i.e., the two curves intersect at (x_{t+1}, y_{t+1}). The next equation, (19), ensures that the two curves are *tangent* to each other at (x_{t+1}, y_{t+1}). Therefore, the marginal price does not change in the transition from one curve to the next. As in Remark 1, this contrasts the approach in Section 1.3.

In this document, we intend to answer several questions: How is it possible and sustainable (in terms of complexity) to evolve these curves? What explicit strategy should be pursued for curve updates? What curves and curve updates are safe from the LPs' perspective?

In the rest of this section, additional tools are built before answering these questions in Section 3.

Draft - NoFeeSwap

2.1 Coin Reserves as Functions of Price

It is instructive to view each trade as a price movement. If $\check{p} \in (0, \infty)$ represents the marginal price of the pool, define $\check{q} \triangleq \log_e(\check{p}) \in (-\infty, \infty)$. We work with the natural logarithm of the price since it simplifies the mathematical derivations due to the symmetry of the domain $(-\infty, \infty)$ and the logarithm is a common measure for price movements.

In the following, a parameterization of each curve with respect to $\check{q}$ is discussed. Let $e^{\check{q}_{\text{lower}}}$ and $e^{\check{q}_{\text{upper}}}$ represent the minimum and maximum price values for the current active liquidity interval, where the current price of the pool belongs. In other words, $\check{p}_{\text{current}}$ satisfies $\check{q}_{\text{current}} = \log_e(\check{p}_{\text{current}}) \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$.[10] For each

[10] When the pool price is on an interval boundary, one of the intervals on its left or right is active, and for moving to the other interval, a transition is needed. The transition is explained in Section 3.6. In Section 2, we focus only on the active liquidity interval.

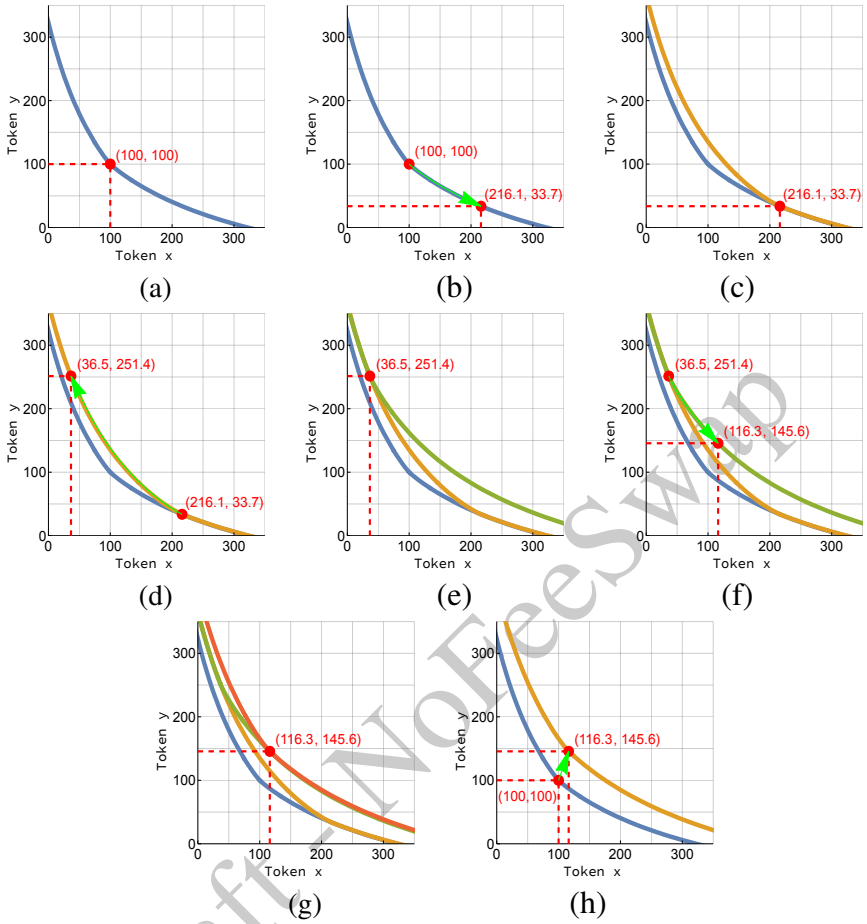


Figure 4: A zero spread market making pool with concentrated liquidity and an LP position within the price limits $p_{\min} = \frac{1}{4}$ and $p_{\max} = 4$: (a) Consider 100 units of reserve for each of the tokens $\boldsymbol{x}$ and $\boldsymbol{y}$; (b) A trader can move the reserve values from point $(100, 100)$ to point $(216.1,\ 33.7)$; (c) The curve then evolves into a new one, which is smooth and tangent to the initial curve; (d) Another trader can then move the reserve values from point $(216.1,\ 33.7)$ to point $(36.5,\ 251.4)$; (e) The curve then evolves into a new one; (f) Another trader can then move the reserve values from point $(36.5,\ 251.4)$ to point $(116.3,\ 145.6)$; (g) The curve then evolves into a new one; (h) This is profitable for LPs as the position of the pool changed from the initial point $(100, 100)$ to the point $(116.3,\ 145.6)$ with an overall profit of the 16.3 token $\boldsymbol{x}$ and the 45.6 token $\boldsymbol{y}$.

Draft - NoFeeSwap

$t \in \mathbb{Z}_+$, define $\mathsf{x}_t, \mathsf{y}_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \to \mathbb{R}_+$ such that:

$$c_t(\mathsf{x}_t(\check{q}), \mathsf{y}_t(\check{q})) = c_t(x_t, y_t), \qquad \forall \check{q} \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}], \tag{20a}$$

$$\frac{\mathsf{y}_t'(\check{q})}{\mathsf{x}_t'(\check{q})} = -e^{\check{q}}, \qquad \forall \check{q} \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}], \tag{20b}$$

where the values x_t, $y_t \in \mathbb{R}_+$ denote the amount of reserves in liquidity interval $[\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$ just before the tth swap. $\mathsf{x}_t(\check{q})$ and $\mathsf{y}_t(\check{q})$ are parameterized reserve functions of token amounts and represent the pair amounts that satisfy (20a) for all log-prices in this liquidity interval. These two functions specify all potential reserve values in this liquidity interval after the tth swap.

Equation (20a) is an immediate consequence of equation (17). Additionally, equation (20b) is the basic definition of the marginal price because:

$$\begin{aligned}\frac{\mathsf{y}'(\check{q})}{\mathsf{x}'(\check{q})} &= \frac{\frac{d\mathsf{y}}{d\check{q}}}{\frac{d\mathsf{x}}{d\check{q}}} = \frac{d\mathsf{y}}{d\mathsf{x}} = \frac{\text{small change in the reserve amount of token } \boldsymbol{y}}{\text{small change in the reserve amount of token } \boldsymbol{x}} \\ &= -\text{marginal price} = -\check{p} = -e^{\check{q}}.\end{aligned} \tag{21}$$

The above parameterization enables us to derive the token exchange amounts with respect to a change in the marginal price.

Now, a sequence of log-price values, $\{\check{q}_t \in \mathbb{R}\}_{t \in \mathbb{Z}_+}$, can be considered such that for each $t \in \mathbb{Z}_+$, the corresponding trade (t^{th} trade) involves a movement from the price $e^{\check{q}_t}$ to $e^{\check{q}_{t+1}}$. Additionally, for each $t \in \mathbb{Z}_+$:[11]

$$x_t = \mathsf{x}_t(\check{q}_t), \qquad y_t = \mathsf{y}_t(\check{q}_t), \tag{22a}$$

$$x_{t+1} = \mathsf{x}_t(\check{q}_{t+1}), \qquad y_{t+1} = \mathsf{y}_t(\check{q}_{t+1}). \tag{22b}$$

Put simply, for each $\check{q} \in [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$, the pair $(\mathsf{x}_t(\check{q}), \mathsf{y}_t(\check{q}))$ denotes the resulting reserve values if the price moves from $e^{\check{q}_t}$ to $e^{\check{q}}$.

The system of differential equations (20) can be solved for a constant product market making framework:

$$\mathsf{x}_t(\check{q})\, \mathsf{y}_t(\check{q}) = x_t\, y_t, \tag{23}$$

[11] As in equation (18), $\mathsf{x}_{t+1}(\check{q}_{t+1}) = \mathsf{x}_t(\check{q}_{t+1})$ and $\mathsf{y}_{t+1}(\check{q}_{t+1}) = \mathsf{y}_t(\check{q}_{t+1})$ hold as well, meaning that the curve update after each swap preserves the token amounts.

Draft - NoFeeSwap

resulting in the following reserve functions:[12]

$$\mathsf{x}_t(\check{q}) = \sqrt{x_t y_t}\; e^{-\frac{\check{q}}{2}}, \tag{26a}$$

$$\mathsf{y}_t(\check{q}) = \sqrt{x_t y_t}\; e^{+\frac{\check{q}}{2}}. \tag{26b}$$

Example 1. *If* $\check{q}_{\text{lower}} = -\infty$*,* $\check{q}_{\text{upper}} = +\infty$*,* $x_t = y_t = 100$*, and* $c_t(x, y) = x\,y$*, then* $\mathsf{x}_t(\check{q}) = 100e^{-\frac{\check{q}}{2}}$ *and* $\mathsf{y}_t(\check{q}) = 100e^{+\frac{\check{q}}{2}}$*.*

In the next section, the notion of liquidity distribution is constructed, which is central to the NoFeeSwap protocol.

2.2 Liquidity Distribution Function

Consider a liquidity interval with a sequence of curves as in (20a) where each function $c_t : \mathbb{R}_+^2 \to \mathbb{R}$ is strictly concave and differentiable. Furthermore, assume that for every t, equations (18) and (19) are satisfied.

For every $t \in \mathbb{Z}_+$, consider a pair of parameterized reserve functions $\mathsf{x}_t, \mathsf{y}_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \to \mathbb{R}_+$ with respect to the natural logarithm of the marginal price, $\check{q}$, which satisfies the system of equations (20).

For every $t \in \mathbb{Z}_+$, define the *liquidity distribution* function $\check{\mathsf{z}}_t : [\check{q}_{\text{lower}}, \check{q}_{\text{upper}}] \to \mathbb{R}_+$ as:

$$\check{\mathsf{z}}_t(\check{q}) \triangleq \sqrt{-\mathsf{x}_t'(\check{q})\mathsf{y}_t'(\check{q})}, \tag{27}$$

which leads to:

$$\mathsf{x}_t(\check{q}) = \int_{\check{q}}^{\check{q}_{\text{upper}}} e^{-\frac{h}{2}}\check{\mathsf{z}}_t(h)dh, \tag{28a}$$

$$\mathsf{y}_t(\check{q}) = \int_{\check{q}_{\text{lower}}}^{\check{q}} e^{+\frac{h}{2}}\check{\mathsf{z}}_t(h)dh. \tag{28b}$$

[12] Another example is the Balancer platform [5], with the formula:

$$\mathsf{x}_t^{\alpha}(\check{q})\mathsf{y}_t^{1-\alpha}(\check{q}) = x_t^{\alpha} y_t^{1-\alpha}, \tag{24}$$

where the constant $\alpha \in (0, 1)$ is set by the pool creator. The reserve functions in this case are:

$$\mathsf{x}_t(\check{q}) = x_t^{\alpha} y_t^{1-\alpha}\Big(\frac{\alpha\, e^{\check{q}}}{1-\alpha}\Big)^{1-\alpha}, \tag{25a}$$

$$\mathsf{y}_t(\check{q}) = x_t^{\alpha} y_t^{1-\alpha}\Big(\frac{\alpha\, e^{\check{q}}}{1-\alpha}\Big)^{-\alpha}. \tag{25b}$$

Draft - NoFeeSwap

These are the amounts of reserve tokens in the active liquidity interval. In order to prove (28a) and (28b), observe that:[13][14]

$$\begin{aligned}
\int_{\check{q}}^{\check{q}_{\text{upper}}} e^{-\frac{h}{2}} \check{\mathsf{z}}_t(h) dh &= \int_{\check{q}}^{\check{q}_{\text{upper}}} \sqrt{-\frac{\mathsf{x}'_t(h)}{\mathsf{y}'_t(h)}} \sqrt{-\mathsf{x}'_t(h)\mathsf{y}'_t(h)}\, dh \\
&= \int_{\check{q}}^{\check{q}_{\text{upper}}} |\mathsf{x}'_t(h)|\, dh = \\
&= \int_{\check{q}}^{\check{q}_{\text{upper}}} -\mathsf{x}'_t(h)\, dh = \mathsf{x}_t(\check{q}) - \mathsf{x}_t(\check{q}_{\text{upper}}) = \mathsf{x}_t(\check{q}), \qquad (29a) \\
\int_{\check{q}_{\text{lower}}}^{\check{q}} e^{+\frac{h}{2}} \check{\mathsf{z}}_t(h) dh &= \int_{\check{q}_{\text{lower}}}^{\check{q}} \sqrt{-\frac{\mathsf{y}'_t(h)}{\mathsf{x}'_t(h)}} \sqrt{-\mathsf{x}'_t(h)\mathsf{y}'_t(h)}\, dh \\
&= \int_{\check{q}_{\text{lower}}}^{\check{q}} |\mathsf{y}'_t(h)|\, dh = \\
&= \int_{\check{q}_{\text{lower}}}^{\check{q}} +\mathsf{y}'_t(h)\, dh = \mathsf{y}_t(\check{q}) - \mathsf{y}_t(\check{q}_{\text{lower}}) = \mathsf{y}_t(\check{q}). \qquad (29b)
\end{aligned}$$

Hence, the pool behavior for the tth swap is fully characterized by the liquidity distribution function $\check{\mathsf{z}}_t$, i.e., the liquidity distribution function is sufficient to determine the token amounts for the tth swap.

Remark 2. *Below are the token exchange amounts through the tth swap:*

$$\mathsf{x}_t(\check{q}_{t+1}) - \mathsf{x}_t(\check{q}_t) = \int_{\check{q}_{t+1}}^{\check{q}_t} e^{-\frac{h}{2}} \check{\mathsf{z}}_t(h) dh, \qquad (30a)$$

$$\mathsf{y}_t(\check{q}_{t+1}) - \mathsf{y}_t(\check{q}_t) = \int_{\check{q}_t}^{\check{q}_{t+1}} e^{+\frac{h}{2}} \check{\mathsf{z}}_t(h) dh. \qquad (30b)$$

Remark 3. *Consider an infinitesimal log-price interval $[\check{q},\ \check{q} + \delta\check{q}]$. We have:*

$$\mathsf{x}_t(\check{q} + \delta\check{q}) - \mathsf{x}_t(\check{q}) = \int_{\check{q}+\delta\check{q}}^{\check{q}} e^{-\frac{h}{2}} \check{\mathsf{z}}_t(h) dh \approx -\check{\mathsf{z}}_t(\check{q}) \times \delta\check{q} \times e^{-\frac{\check{q}}{2}}, \qquad (31a)$$

$$\mathsf{y}_t(\check{q} + \delta\check{q}) - \mathsf{y}_t(\check{q}) = \int_{\check{q}}^{\check{q}+\delta\check{q}} e^{+\frac{h}{2}} \check{\mathsf{z}}_t(h) dh \approx +\check{\mathsf{z}}_t(\check{q}) \times \delta\check{q} \times e^{+\frac{\check{q}}{2}}. \qquad (31b)$$

[13] When the pool price increases, $\mathsf{x}_t(\check{q})$ decreases and $\mathsf{y}_t(\check{q})$ increases, i.e., $|\mathsf{x}'_t(h)| = -\mathsf{x}'_t(h)$ and $|\mathsf{y}'_t(h)| = +\mathsf{y}'_t(h)$.

[14] $\mathsf{x}_t(\check{q}_{\text{upper}}) = \mathsf{y}_t(\check{q}_{\text{lower}}) = 0$ holds as the concentrated liquidity ensures the depletion of one token when a liquidity interval's boundary is reached, i.e., the depletion of $\boldsymbol{x}$ on the upper boundary and $\boldsymbol{y}$ on the lower boundary.

Draft - NoFeeSwap

Hence, $-\check{\mathsf{z}}_t(\check{q}) \times \delta\check{q} \times e^{-\frac{\check{q}}{2}}$ amount of token $\boldsymbol{x}$ and $+\check{\mathsf{z}}_t(\check{q}) \times \delta\check{q} \times e^{+\frac{\check{q}}{2}}$ token amount $\boldsymbol{y}$ are exchanged by crossing $[\check{q},\ \check{q}+\delta\check{q}]$. This means that $\check{\mathsf{z}}_t(\check{q})$ dictates the volume of trade when crossing $[\check{q},\ \check{q}+\delta\check{q}]$, which is why $\check{\mathsf{z}}_t$ is considered the liquidity distribution function.

Remark 4. *The liquidity amount available for trading is proportional to the value of liquidity distribution function at each particular price.*

Example 2 (Constant product DEX – without fee)**.** *Here, the parameterized reserve functions, x_t and y_t, as well as the liquidity distribution function, $\check{\mathsf{z}}_t$, are derived for constant product DEXs, e.g., Uniswap v2 [6], in the hypothetical case of no fee. To this end, the following system of differential equations should be solved:*

$$\mathsf{x}_t(\check{q})\mathsf{y}_t(\check{q}) = x_t y_t, \tag{32a}$$

$$\frac{\mathsf{y}'_t(\check{q})}{\mathsf{x}'_t(\check{q})} = -e^{\check{q}}, \tag{32b}$$

which leads to:[15]

$$\mathsf{x}_t(\check{q}) = \sqrt{x_t y_t}\, e^{-\frac{\check{q}}{2}}, \tag{33a}$$

$$\mathsf{y}_t(\check{q}) = \sqrt{x_t y_t}\, e^{+\frac{\check{q}}{2}}, \tag{33b}$$

$$\check{\mathsf{z}}_t(\check{q}) = \frac{1}{2}\sqrt{x_t y_t}. \tag{33c}$$

However, this structure does not lead to any profit for the LPs and is not practical from the LPs' incentive perspective.

Example 3 (Constant product DEX – with fee)**.** *Here, the parameterized reserve functions x_t and y_t, as well as the liquidity distribution function, $\check{\mathsf{z}}_t$, are derived and illustrated for constant product DEXs, e.g., Uniswap v2, in the presence of fee. To this end, the following system of differential equations should be solved:*[16]

[15] The liquidity in Uniswap v2 is distributed across the entire $(-\infty, +\infty)$ price horizon as there is no concentrated liquidity. With concentrated liquidity, the equations would hold only locally, within the boundaries of the active liquidity interval where the current price of the pool is located.

[16] The case of $\check{q} \geq q_t$ maps to $\mathsf{x}_t(\check{q}) \leq x_t$ and $\mathsf{y}_t(\check{q}) \geq y_t$. The case of $\check{q} \leq q_t$ maps to $\mathsf{x}_t(\check{q}) \geq x_t$ and $\mathsf{y}_t(\check{q}) \leq y_t$.

Draft - NoFeeSwap

$$\mathsf{x}_t(\check{q})[y_t + (1-f)(\mathsf{y}_t(\check{q}) - y_t)] = x_t y_t, \qquad \text{if } \check{q} \geq q_t, \tag{34a}$$

$$[x_t + (1-f)(\mathsf{x}_t(\check{q}) - x_t)]\mathsf{y}_t(\check{q}) = x_t y_t, \qquad \text{if } \check{q} \leq q_t, \tag{34b}$$

$$\frac{\mathsf{y}_t'(\check{q})}{\mathsf{x}_t'(\check{q})} = -e^{\check{q}}, \tag{34c}$$

which leads to:

$$\mathsf{x}_t(\check{q}) = \begin{cases} \sqrt{\frac{x_t y_t}{1-f}} e^{-\frac{\check{q}}{2}} - \frac{f}{1-f} x_t & \check{q} \leq \log_e\left(\frac{(1-f)y_t}{x_t}\right) \\ x_t & \text{o.w.} \\ \sqrt{\frac{x_t y_t}{1-f}} e^{-\frac{\check{q}}{2}} & \log_e\left(\frac{y_t}{(1-f)x_t}\right) \leq \check{q} \end{cases}, \tag{35a}$$

$$\mathsf{y}_t(\check{q}) = \begin{cases} \sqrt{\frac{x_t y_t}{1-f}} e^{+\frac{\check{q}}{2}} & \check{q} \leq \log_e\left(\frac{(1-f)y_t}{x_t}\right) \\ y_t & \text{o.w.} \\ \sqrt{\frac{x_t y_t}{1-f}} e^{+\frac{\check{q}}{2}} - \frac{f}{1-f} y_t & \log_e\left(\frac{y_t}{(1-f)x_t}\right) \leq \check{q} \end{cases}, \tag{35b}$$

$$\check{\mathsf{z}}_t(\check{q}) = \begin{cases} 0 & \log_e\left(\frac{(1-f)y_t}{x_t}\right) \leq \check{q} \leq \log_e\left(\frac{y_t}{(1-f)x_t}\right) \\ \frac{1}{2}\sqrt{\frac{x_t y_t}{1-f}} & \text{o.w.} \end{cases}, \tag{35c}$$

where $f \in (0,1)$ represents the fee ratio. These three functions as well as the resulting curve are illustrated in Figure 5.

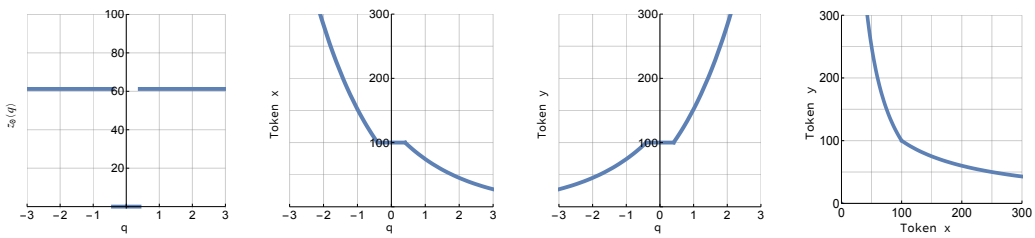
Remark 5. *In Example 2, $c(x,y) = xy$ is strictly concave and differentiable. And, since the curves before and after each swap are the same, equations* (18)–(19) *hold.*

In Example 3, equations (34a)–(34b) *lead to $c_t(x_t, y_t)$'s non-differentiability, for which equation* (19) *does not hold. See Remark 1.*

One of the drawbacks of Uniswap v2 is the spread between the buy and sell marginal prices. In other words, selling a token incurs a different marginal price than buying that token. Without any compromise on profitability for LPs, a key feature of the NoFeeSwap protocol is to address that by avoiding any price range within which the liquidity distribution function is zero.

Remark 6. *In order to avoid spread, the liquidity distribution function must not be zero within any interval. Consider a price range $(\check{q}_1, \check{q}_2)$ where $\check{q}_1, \check{q}_2 \in$*

Draft - NoFeeSwap



(a) (b) (c) (d)

Figure 5: Illustrations corresponding to Example 3 with parameters $x_t = y_t = 100$, and $f = \frac{1}{3}$. Since there is no concentrated liquidity, $\check{q}_{\text{lower}} = -\infty$, $\check{q}_{\text{upper}} = +\infty$. The fee ratio is exaggerated for illustrative purposes. As demonstrated by the figures, no liquidity is devoted to the range $\left[\log_e\left(\frac{(1-f)y_t}{x_t}\right), \log_e\left(\frac{y_t}{(1-f)x_t}\right)\right]$, which results in a marginal spread. (a) Liquidity distribution function, $\check{\mathsf{z}}_t$; (b) Parameterized reserve function, x_t; (c) Parameterized reserve function, y_t; (d) The resulting curve, c_t.

Draft - NoFeeSwap

$[\check{q}_{\text{lower}}, \check{q}_{\text{upper}}]$ *and assume that* $\check{\mathsf{z}}_t(\check{q}) = 0$ *for every* $\check{q} \in (\check{q}_1, \check{q}_2)$. *Hence:*

$$\mathsf{x}_t(\check{q}_2) - \mathsf{x}_t(\check{q}_1) = \int_{\check{q}_2}^{\check{q}_1} e^{-\frac{h}{2}} \check{\mathsf{z}}_t(h) dh = 0, \tag{36a}$$

$$\mathsf{y}_t(\check{q}_2) - \mathsf{y}_t(\check{q}_1) = \int_{\check{q}_1}^{\check{q}_2} e^{+\frac{h}{2}} \check{\mathsf{z}}_t(h) dh = 0. \tag{36b}$$

As a consequence, if $\check{q}_t \in (\check{q}_1, \check{q}_2)$*:*

$$\mathsf{x}_t(\check{q}_1) = \mathsf{x}_t(\check{q}_t) = \mathsf{x}_t(\check{q}_2), \tag{37a}$$

$$\mathsf{y}_t(\check{q}_1) = \mathsf{y}_t(\check{q}_t) = \mathsf{y}_t(\check{q}_2). \tag{37b}$$

which means that for an infinitesimal trade, the trader needs to move the price to either $\check{q}_1 - \delta\check{q}$ *or* $\check{q}_2 + \delta\check{q}$*, where* $\delta\check{q} \to 0^+$*. The marginal prices associated with each of these two trades are:*

$$\frac{\mathsf{y}_t(\check{q}_2 + \delta\check{q}) - \mathsf{y}_t(\check{q}_t)}{\mathsf{x}_t(\check{q}_t) - \mathsf{x}_t(\check{q}_2 + \delta\check{q})} = \frac{\mathsf{y}_t(\check{q}_2 + \delta\check{q}) - \mathsf{y}_t(\check{q}_2)}{\mathsf{x}_t(\check{q}_2) - \mathsf{x}_t(\check{q}_2 + \delta\check{q})} \approx \frac{\check{\mathsf{z}}_t(\check{q}_2) \times \delta\check{q} \times e^{+\frac{\check{q}_2}{2}}}{\check{\mathsf{z}}_t(\check{q}_2) \times \delta\check{q} \times e^{-\frac{\check{q}_2}{2}}} = e^{\check{q}_2}, \tag{38a}$$

$$\frac{\mathsf{y}_t(\check{q}_t) - \mathsf{y}_t(\check{q}_1 - \delta\check{q})}{\mathsf{x}_t(\check{q}_1 - \delta\check{q}) - \mathsf{x}_t(\check{q}_t)} = \frac{\mathsf{y}_t(\check{q}_1) - \mathsf{y}_t(\check{q}_1 - \delta\check{q})}{\mathsf{x}_t(\check{q}_1 - \delta\check{q}) - \mathsf{x}_t(\check{q}_1)} \approx \frac{\check{\mathsf{z}}_t(\check{q}_1) \times \delta\check{q} \times e^{+\frac{\check{q}_1}{2}}}{\check{\mathsf{z}}_t(\check{q}_1) \times \delta\check{q} \times e^{-\frac{\check{q}_1}{2}}} = e^{\check{q}_1}. \tag{38b}$$

Hence, the marginal buy price of the pool is $e^{\check{q}_1}$ and the marginal sell price of the pool is $e^{\check{q}_2}$, which incurs spread.

Remark 7. *In NoFeeSwap, in order to ensure a high precision calculation of the integrals* (28a)-(28b)*, all price movements are restricted to $\check{p} \in (e^{-16}p_{\text{offset}}, e^{+16}p_{\text{offset}})$, and consequently the log-price range $\check{q} \in (q_{\text{offset}} - 16, q_{\text{offset}} + 16)$, where $q_{\text{offset}} = \log_e(p_{\text{offset}})$ is the natural logarithm of the offset price established by the creator of the pool. Therefore, instead of integrals* (28a)-(28b)*, the following are calculated and stored:*[17][18]

$$\begin{aligned}
\sqrt{p_{\text{offset}}}\,\mathsf{x}_t(\check{q}) &= \sqrt{p_{\text{offset}}}\int_{\check{q}}^{\check{q}_{\text{upper}}} e^{-\frac{h}{2}}\check{\mathsf{z}}_t(h)dh \\
&= \int_{\check{q}-q_{\text{offset}}}^{\check{q}_{\text{upper}}-q_{\text{offset}}} e^{-\frac{h}{2}}\check{\mathsf{z}}_t(h+q_{\text{offset}})dh = \int_{q}^{q_{\text{upper}}} e^{-\frac{h}{2}}\mathsf{z}_t(h)dh, \qquad (39a)\\
\frac{1}{\sqrt{p_{\text{offset}}}}\,\mathsf{y}_t(\check{q}) &= \frac{1}{\sqrt{p_{\text{offset}}}}\int_{\check{q}_{\text{lower}}}^{\check{q}} e^{+\frac{h}{2}}\check{\mathsf{z}}_t(h)dh \\
&= \int_{\check{q}_{\text{lower}}-q_{\text{offset}}}^{\check{q}-q_{\text{offset}}} e^{+\frac{h}{2}}\check{\mathsf{z}}_t(h+q_{\text{offset}})dh = \int_{q_{\text{lower}}}^{q} e^{+\frac{h}{2}}\mathsf{z}_t(h)dh, \qquad (39b)
\end{aligned}$$

where

$$q_{\text{upper}} \triangleq \check{q}_{\text{upper}} - q_{\text{offset}}, \qquad (40a)$$

$$q \triangleq \check{q} - q_{\text{offset}}, \qquad (40b)$$

$$q_{\text{lower}} \triangleq \check{q}_{\text{lower}} - q_{\text{offset}}, \qquad (40c)$$

$$\mathsf{z}_t(\cdot) \triangleq \check{\mathsf{z}}_t(\cdot + q_{\text{offset}}), \qquad (40d)$$

are the standardized log-price values and liquidity distribution function.

Remark 8. *The price p and log-price q refer to standardized values, and one needs to perform the following transformations to obtain the real-world values:*

$$\check{p} = p_{\text{offset}} \times p, \qquad (41a)$$

$$\check{q} = q_{\text{offset}} + q, \qquad (41b)$$

$$\sqrt{\check{p}} = \sqrt{p_{\text{offset}}} \times \sqrt{p}. \qquad (41c)$$

[17] It is straightforward to verify these equalities via change of variables.

[18] Similar arguments hold for other token reserve equations, e.g., the integrals in Remark 2.

Draft - NoFeeSwap

Remark 9. *For numerical reasons, all integrals are stored with a multiplier of* $\frac{e^{-8}}{2}$ *in the blockchain storage. This guarantees that the stored values of the integrals remain below* 1*, as shown in equation* (58)*. This multiplier is omitted from the formulations of this document to enhance readability.*

3 NoFeeSwap Protocol

In the NoFeeSwap protocol, there are several aspects to the liquidity distribution function and update algorithms. Section 3.1 explains the structure of the liquidity distribution. Section 3.1.3 explains an algorithm for curve updates after each swap. Sections 3.2 and 3.3 explain the main parameters of the protocol and each liquidity pool. Section 3.4 discusses the calculation of the price and swap amounts, and Section 3.5 provides the calculation of the consequent curve updates and the growth of the LPs. Section 3.6 provides explanations on the components involved in a swap call. Section 3.7 covers Solidity data types. Section 3.8 provides explanations on the protocol's external functions.

Draft - NoFeeSwap

3.1 Liquidity Distribution Function

The management of the liquidity distribution is through the introduction and interactions of two functions: 1) **Kernel**, which is a static liquidity distribution applied to each liquidity interval; 2) **Curve Sequence**, which dynamically updates after each swap and is a mechanism for handling the liquidity distribution in the active liquidity interval.

3.1.1 Kernel

Consider a pool with *log-price spacing* equal to $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} > 0$. Therefore, every concentrated liquidity interval boundary is of the form $q_{\text{lower}} + m\, q_{\text{spacing}}$, for some $m \in \mathbb{Z}$.

At the time of initialization of a pool, the pool owner provides a piecewise linear monotonically non-decreasing function $\mathsf{k} : [0, q_{\text{spacing}}] \to [0, 1]$ which satisfies:

$$\mathsf{k}(0) = 0, \qquad \mathsf{k}(q_{\text{spacing}}) = 1, \tag{42}$$

and is referred to as the kernel.[19] The role of the kernel is to dictate the liquidity distribution function in all liquidity intervals, including the active liquidity interval

[19]Note that breakpoints with equal horizontal coordinates (i.e., $\mathsf{k}(q_0) = h_1$ and $\mathsf{k}(q_0) = h_2$)

that contains the current price of the pool.[20] Figure 6 provides simple examples of the kernel function.

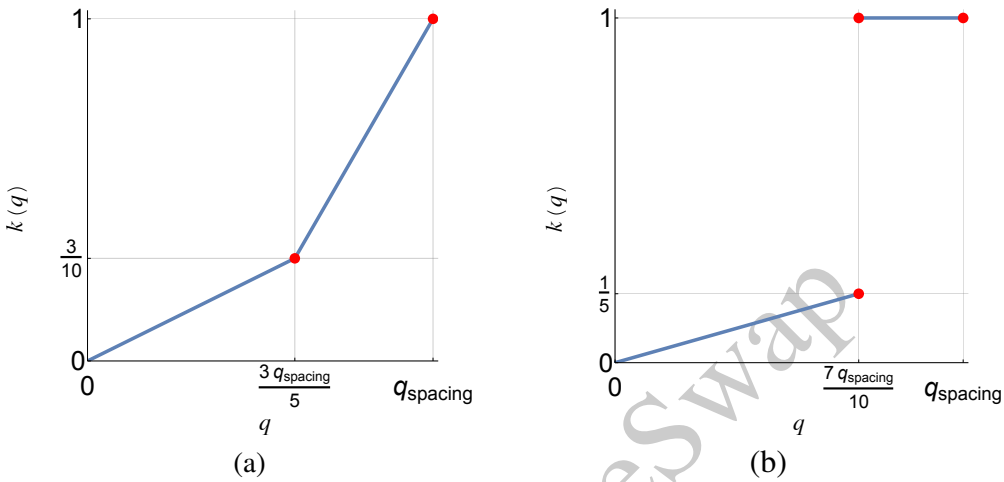


Figure 6: (a) A kernel defined by the set of break-points $\{(0,0),(0.6q_{\text{spacing}},0.3),(q_{\text{spacing}},1)\}$; (b) A kernel defined by the set of break-points $\{(0,0),(0.7q_{\text{spacing}},0.2),(0.7q_{\text{spacing}},1),(q_{\text{spacing}},1)\}$.

Draft - NoFeeSwap

The pool owner selects the kernel. The kernel can also be modifiable by the pool owner if the `isMutableKernel` flag[21] is set. See explanations of `staticParamsStoragePointer` in Section 3.2.2 for kernel modifications.

Remark 10. *A newly specified kernel stays pending and gets activated upon the earliest transition to a new liquidity interval in that pool. At the transition moment, the pending kernel becomes the pool's* `kernel`*, and* `nextKernelLength` *becomes zero.*

Consider an arbitrary integer m and the liquidity interval $[q_{\text{lower}}+mq_{\text{spacing}}, q_{\text{upper}}+$

are permitted. In such cases, we have a kernel discontinuity, i.e., $\mathsf{k}(q_0^-) = \min\{h_1,h_2\}$ and $\mathsf{k}(q_0^+) = \max\{h_1,h_2\}$. The value of the function at a discontinuity is not relevant, and only the integral values in various ranges matter.

[20] Inactive liquidity intervals include every interval except the active liquidity interval.

[21] Flags are discussed in Section 3.2.1.1.

$mq_{\text{spacing}}]$[22]. Define:

$$\begin{aligned}
\text{outgoingMax}_{\mathsf{k}} &\triangleq e^{+\frac{q_{\text{lower}}+mq_{\text{spacing}}}{2}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{-\frac{h}{2}}\mathsf{k}(h - q_{\text{lower}} - mq_{\text{spacing}})dh \\
&= e^{-\frac{q_{\text{upper}}+mq_{\text{spacing}}}{2}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{+\frac{h}{2}}\mathsf{k}(q_{\text{upper}} + mq_{\text{spacing}} - h)dh \\
&= \int_0^{q_{\text{spacing}}} e^{-\frac{h}{2}}\mathsf{k}(h)dh. \qquad (43)
\end{aligned}$$

The value of $\text{outgoingMax}_{\mathsf{k}}$ is a constant associated with the kernel and is independent of the integer m, i.e., the third integral has no m and the first two integrals are invariant to m.[23] This value serves as a normalizer for kernel functions.

3.1.1.1 Inactive liquidity intervals on the right side. The standardized liquidity distribution function over the liquidity interval $q \in [q_{\text{lower}}+mq_{\text{spacing}}, q_{\text{upper}}+mq_{\text{spacing}}]$, for any integer $m > 0$, is equal to:

$$\mathsf{z}(q) = \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_{\mathsf{k}}}\,\mathsf{k}(q - q_{\text{lower}} - mq_{\text{spacing}}), \qquad (44)$$

where sharesTotal is a non-negative integer denoting the total number of LP shares in the corresponding liquidity interval and $\text{growth} \geq 1$ denotes the corresponding liquidity interval's cumulative liquidity growth.[24]

This liquidity interval is inactive, and the pool price is on its left side and hence

[22] which is the same as $[q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{lower}} + (m+1)q_{\text{spacing}}]$

[23] There are exponential terms, $e^{+\frac{q_{\text{lower}}+mq_{\text{spacing}}}{2}}$ and $e^{-\frac{q_{\text{upper}}+mq_{\text{spacing}}}{2}}$, to make the first two integrals invariant for all liquidity intervals.

[24] The protocol keeps track of the share count and the growth value for the active liquidity interval. See `growth` and `sharesTotal` in Section 3.2.2. To retrieve such values for other liquidity intervals, the mapping parameters are introduced. See Section 3.3.

Draft - NoFeeSwap

the reserve of `token1` is zero[25][26], i.e.,

$$
\begin{aligned}
&\texttt{token1reserve} = 0 \\
&\texttt{token0reserve} \\
&= \frac{1}{\sqrt{p_{\text{offset}}}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{-\frac{h}{2}} \mathsf{z}(h) dh \\
&= \frac{1}{\sqrt{p_{\text{offset}}}} \int_{q_{\text{lower}}+mq_{\text{spacing}}}^{q_{\text{upper}}+mq_{\text{spacing}}} e^{-\frac{h}{2}} \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_{\mathsf{k}}} \mathsf{k}(h - q_{\text{lower}} - mq_{\text{spacing}}) dh \\
&= \frac{1}{\sqrt{p_{\text{offset}}}} \text{sharesTotal} \times \text{growth} \times e^{-\frac{q_{\text{lower}}+mq_{\text{spacing}}}{2}}. \qquad (45)
\end{aligned}
$$

3.1.1.2 Inactive liquidity intervals on the left side. The standardized liquidity distribution function over the liquidity interval $q \in [q_{\text{lower}} - mq_{\text{spacing}}, q_{\text{upper}} - mq_{\text{spacing}}]$, for any integer $m > 0$, is equal to:

$$
\mathsf{z}(q) = \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_{\mathsf{k}}} \mathsf{k}(q_{\text{upper}} - mq_{\text{spacing}} - q), \qquad (46)
$$

which its shape is a mirror reflection of equation (44). This liquidity interval is inactive, and the pool price is on its right side, hence:

$$
\begin{aligned}
&\texttt{token0reserve} = 0 \\
&\texttt{token1reserve} \\
&= \sqrt{p_{\text{offset}}} \int_{q_{\text{lower}}-mq_{\text{spacing}}}^{q_{\text{upper}}-mq_{\text{spacing}}} e^{+\frac{h}{2}} \mathsf{z}(h) dh \\
&= \sqrt{p_{\text{offset}}} \int_{q_{\text{lower}}-mq_{\text{spacing}}}^{q_{\text{upper}}-mq_{\text{spacing}}} e^{+\frac{h}{2}} \frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_{\mathsf{k}}} \mathsf{k}(q_{\text{upper}} - mq_{\text{spacing}} - h) dh \\
&= \sqrt{p_{\text{offset}}} \text{sharesTotal} \times \text{growth} \times e^{+\frac{q_{\text{upper}}-mq_{\text{spacing}}}{2}}. \qquad (47)
\end{aligned}
$$

The liquidity distribution function and the token amounts for the *active* liquidity interval are discussed next.

[25] In this document, occasionally, `token0` and token $\boldsymbol{x}$ are used interchangeably; `token1` and token $\boldsymbol{y}$ are used interchangeably.

[26] See equation (39).

Draft - NoFeeSwap

3.1.2 Curve Sequence

In order to keep track of the liquidity distribution function within the active liquidity interval, before the tth swap, a finite sequence of log-price values is considered:[27]

$$\bar{q}_t \triangleq q_t^0, q_t^1, q_t^2, \ldots, q_t^{l-1}, \tag{48}$$

such that:

$$q_{\text{lower}} = \min\{q_t^0, q_t^1\}, \tag{49}$$

$$q_{\text{upper}} = \max\{q_t^0, q_t^1\}, \tag{50}$$

$$q_{\text{current}} = q_t^{l-1}, \tag{51}$$

and for each $i \in \{2, \ldots, l-1\}$,

$$\min\{q_t^{i-1}, q_t^{i-2}\} < q_t^i < \max\{q_t^{i-1}, q_t^{i-2}\}. \tag{52}$$

This is referred to as the *curve sequence*.[28] The liquidity distribution function for the active liquidity interval is equal to:

$$\mathsf{z}_t(q) \triangleq \frac{\text{sharesTotal} \times \text{growth}_\text{t}}{\text{outgoingMax}_\mathsf{k}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(q)), \qquad q_{\text{lower}} \leq q \leq q_{\text{upper}}, \tag{53}$$

where:

$$\mathsf{w}_{\bar{q}_t}(q) \triangleq \sum_{i=1}^{l-1} (q - q_t^i)\big(\mathcal{I}\{q_t^{i+1} < q < q_t^{i-1}\} - \mathcal{I}\{q_t^{i-1} < q < q_t^{i+1}\}\big), \tag{54}$$

and the out-of-range value q_t^l is equal to q_t^{l-1}. The indicator function $\mathcal{I}$ returns 1 if its argument is true and returns 0, otherwise. See Figure 7.

Remark 11. *To facilitate coordinating integrations, each piece of the function* $\mathsf{w}(q)$ *is characterized using three members of the curve sequence, the two ends and the intersection with the horizontal axis.* $\mathsf{w}(q)$ *is defined in such a way that each of these points is a member of the curve sequence.*

[27] The occasional use of time-indexing is to emphasize the evolvement with swaps and to formulate the impact of swaps. For example, z_t, growth_t, and $\bar{q}_t$ change with each swap and are functions of swaps until before t^{th}. However, swaps do not necessarily change the kernel, the number of shares, and $\text{outgoingMax}_\mathsf{k}$.

[28] These equations are for instances when $q_{\text{lower}} < q_{\text{current}} < q_{\text{upper}}$. When $q_{\text{current}} = q_{\text{lower}}$, the curve sequence is $\bar{q}_t = q_{\text{upper}}, q_{\text{lower}}$, and when $q_{\text{current}} = q_{\text{upper}}$, the curve sequence is $\bar{q}_t = q_{\text{lower}}, q_{\text{upper}}$.

Draft - NoFeeSwap

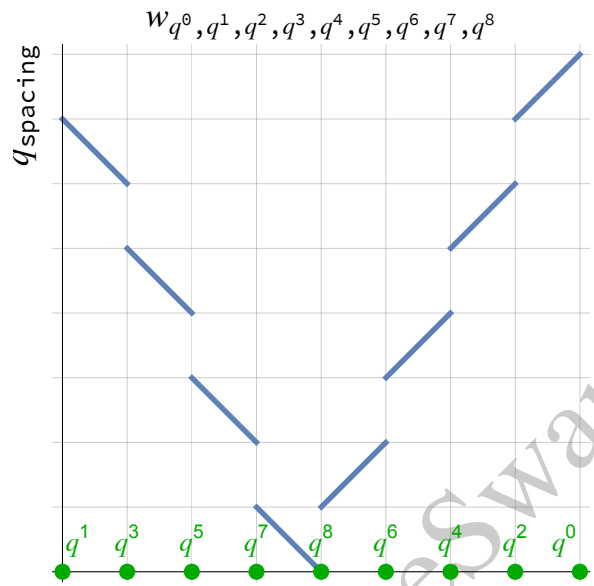


Figure 7: A sample $\mathsf{w}_{\bar{q}_t}(q)$ with a curve sequence of size 9.

Draft - NoFeeSwap

Figure 8(a) shows $\mathsf{w}_{\bar{q}_t}$ for the default curve sequence when the pool price is on the left side of this liquidity interval. Figure 8(b) shows $\mathsf{w}_{\bar{q}_t}$ for the example of $\bar{q}_t = q_{\text{lower}} + q_{\text{spacing}}, q_{\text{lower}}, q_{\text{lower}} + \frac{2}{3}q_{\text{spacing}}, q_{\text{lower}} + \frac{1}{3}q_{\text{spacing}}$. Updates of the curve sequence and $\mathsf{w}_{\bar{q}_t}$ with swaps are discussed in Section 3.1.3.

After each swap, the protocol calculates a composition of $\mathsf{w}_{\bar{q}_t}(\cdot)$ and the kernel function, $\mathsf{k}(\cdot)$, to determine the liquidity distribution function of the active liquidity interval according to equation (53). The token exchange amounts corresponding to each price movement can then be determined by integrating over $\mathsf{z}_t(q)$ throughout the appropriate liquidity intervals, as in Remark 2[29].

When a pool price moves inside a liquidity interval as a consequence of swaps, the curve sequence for that liquidity interval dynamically updates/changes. If the kernel is as in Figure 6(b) and the curve sequence is such that $\mathsf{w}(q)$ is as in Figure 8(b), then, $\mathsf{k}(\mathsf{w}(q))$ can be calculated for every $h \in [q_{\text{lower}}, q_{\text{upper}}]$ as in Figure 9.[30]

For inactive liquidity intervals, $\mathsf{w}(q)$ is always in the form of a shifted identity function, or mirrored. Additionally, the liquidity distribution function in inactive liquidity intervals is proportional to a shifted kernel function, or mirrored. Note

[29] with consideration of the offset as in Remark 7

[30] $\mathsf{z}_t(q)$ can be derived using equation (53).

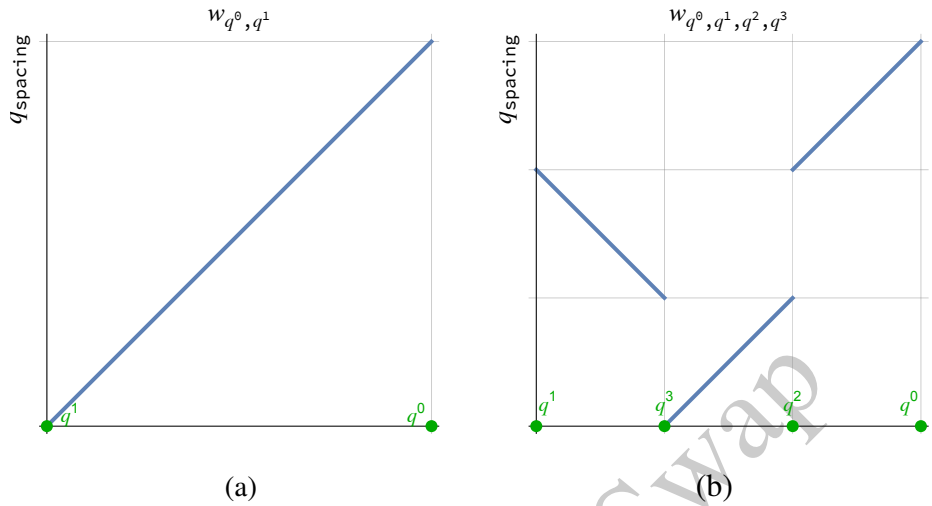


Figure 8: (a) $\mathsf{w}(q)$ and the default curve sequence for an inactive liquidity interval when the pool price is on the left side of it; (b) A sample snapshot of $\mathsf{w}(q)$ in the active liquidity interval after some swaps.

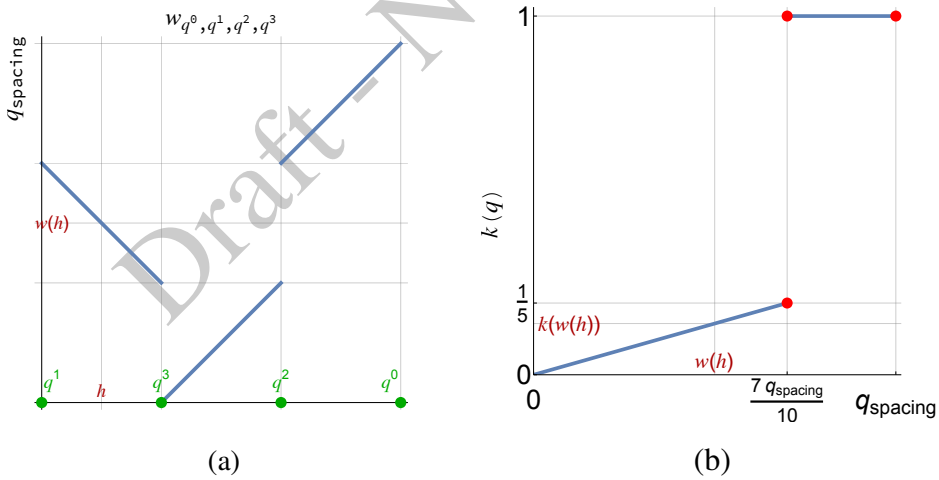


Figure 9: On the left side, $\mathsf{w}(h)$ can be calculated for every h in the range $[q_{\text{lower}}, q_{\text{upper}}]$. Then, on the right side, $\mathsf{w}(h)$ is given to the kernel function as input, and $\mathsf{k}(\mathsf{w}(h))$ is calculated. This illustrates the composition of the curve sequence in Figure 8(b) and the kernel in Figure 6(b), which determines $\mathsf{k}(\mathsf{w}(h))$.

Draft - NoFeeSwap

that the scalar multiplier in the liquidity distribution function, $\frac{\text{sharesTotal} \times \text{growth}}{\text{outgoingMax}_{\mathsf{k}}}$, is different for each liquidity interval (and is zero for intervals with zero liquidity). As depicted in Figure 10, for the liquidity intervals that are on the left side of the current pool price, the liquidity distribution is effectively mirrored.

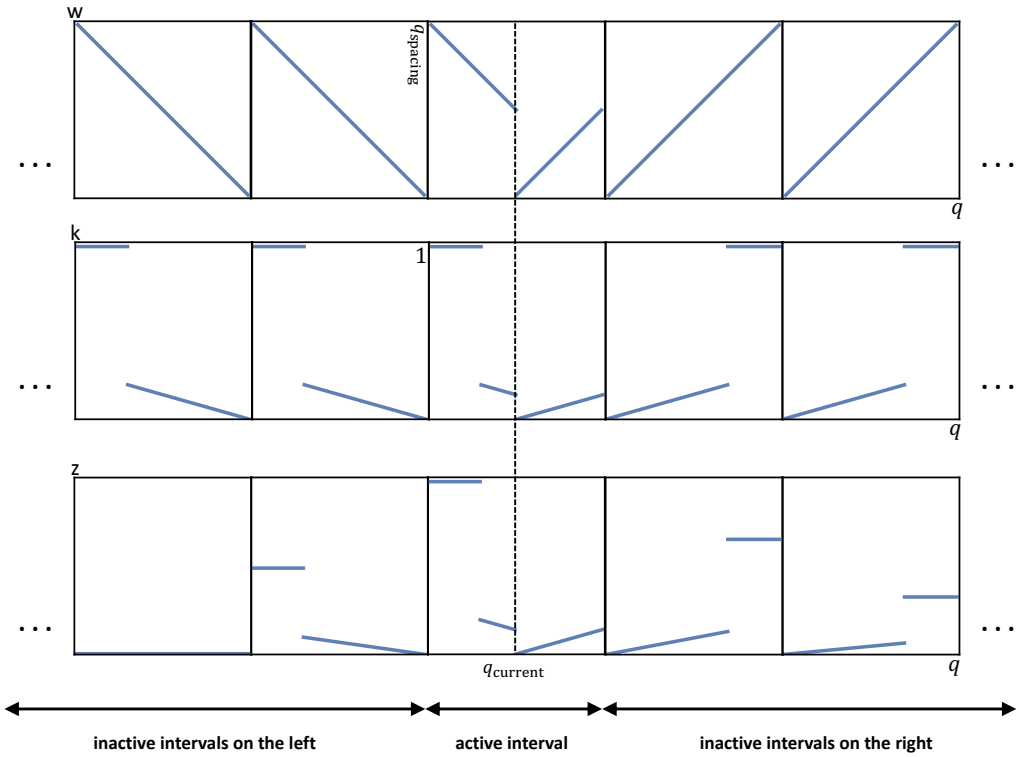


Figure 10: (a) $\mathsf{w}(q)$ [top]; (b) $\mathsf{k}(\mathsf{w}(q))$ [middle]; (c) $\mathsf{z}(q)$ [bottom].

A major component of the NoFeeSwap protocol is how the curve sequence is updated as a result of a swap. This is discussed next.

3.1.3 Curve Sequence Updates after Swaps

Figure 8(b) shows a sample snapshot of $\mathsf{w}_{\bar{q}_t}(q)$ for the active liquidity interval. Observe that $q_t^0, q_t^1, q_t^2, \ldots, q_t^{\ell-1}$ (a curve sequence of size ℓ) approaches the pool price, alternating from left and right, with decreasing heights as in equation (54) and Figure 7. The function $\mathsf{w}_{\bar{q}_t}(q)$ is a rearrangement of the identity function in which every other piece is mirrored. See Figure 11.

The pool price changes through swaps. If the pool price enters a liquidity interval from the left (q_{lower}), the first two members of the curve sequence are $q^0 = q_{\text{upper}}$ and $q^1 = q_{\text{lower}}$. These two and their order remain unchanged as long as the pool price has not reached the other boundary (q_{upper}). Similarly, if the pool price enters a liquidity interval from the right (q_{upper}), the first two members of the curve sequence are $q^0 = q_{\text{lower}}$ and $q^1 = q_{\text{upper}}$.

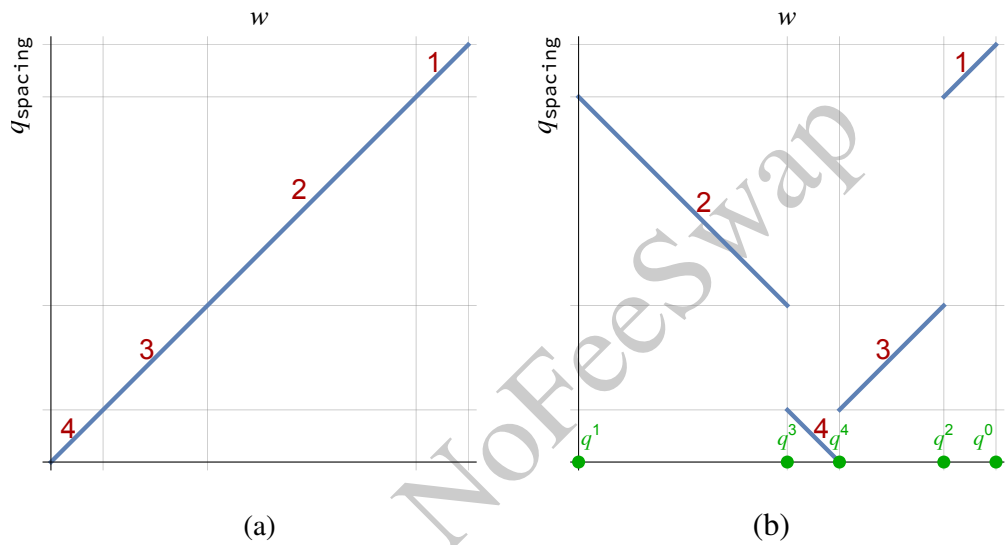


Figure 11: (a) An illustration of $\mathsf{w}(q)$ for an inactive liquidity interval. This shape holds when the pool price is on the left side of this liquidity interval. The pieces to be moved (through swaps) are specified; (b) An updated $\mathsf{w}(q)$ (shifted and every other one is mirrored). Note that all pieces preserve their height after updates.

Draft - NoFeeSwap

After each swap, the curve sequence is updated/amended in preparation for the next swap. An update of the curve sequence can change the number of its members, e.g., it can convert Figure 8(b) to any instance shown in Figure 12. Consider a log-price $q_{\text{new}} \in (q_{\text{lower}}, q_{\text{upper}})$ to be added to a curve sequence of size ℓ. If q_{new} is between $q_t^{\ell-2}$ and $q_t^{\ell-1}$ (the last two members of the curve sequence), then q_{new} is appended to the curve sequence, as in the first case in Figure 12. Otherwise, let ℓ' denote the largest integer such that q_{new} is between $q_t^{\ell'-2}$ and $q_t^{\ell'-1}$ ($2 \leq \ell' \leq \ell - 1$). In this case, the curve sequence is transformed to $q_t^0, q_t^1, q_t^2, \ldots, q_t^{\ell'-1}, q_{\text{new}}$ (a curve sequence of size $\ell' + 1$), as in the second and third cases in Figure 12. The fourth and fifth cases in Figure 12 show instances in which the pool price is

moved to the left and right boundaries of the liquidity interval, respectively[31].

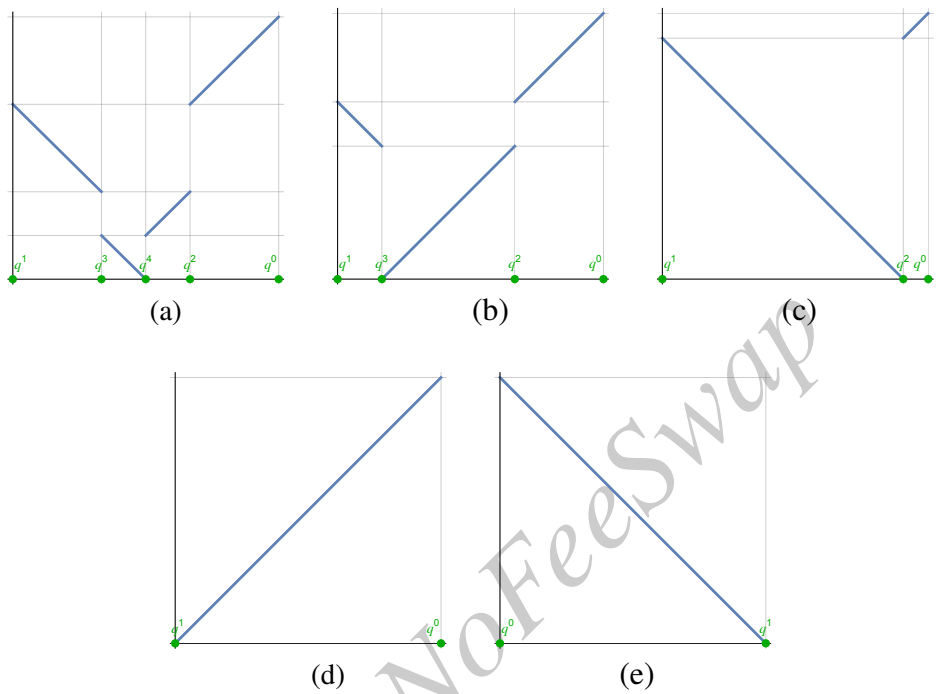


Figure 12: Possible updated $\mathsf{w}(q)$ shapes from Figure 8(b) with an amendment.

Draft - NoFeeSwap

Remark 12. *Section 3.5 explains that after each swap, the curve sequence may undergo up to two updates and details how the amended log-price for each curve update is determined.*[32]

Remark 13. *Members of the curve sequence can be viewed as historical peaks of the price, with the latest members representing the most recent ones, i.e., the curve sequence can be interpreted as the protocol's price memory in the active liquidity interval. By keeping track of the pool price fluctuations through the curve sequence updates, the protocol applies more slippage for moving towards the most recently observed pool prices. This can result in higher profit for LPs at the cost of lower profit for arbitrageurs, as arbitrageurs tend to move the pool price back towards the market price after other traders' transactions.*

[31] and possibly the pool price leaves the liquidity interval after reaching a boundary, through an interval transition

[32] Remark 46 discusses the possibility of having only one such update after certain swaps.

Furthermore, Section 3.5 demonstrates that these updates to the curve sequence result in liquidity growth and profit for LPs.

Remark 14. *The curve sequence enables efficient storage of historical prices on the blockchain that comes with various advantages.*

- *A short stack: The concept of a curve sequence is an innovative and efficient method of keeping track of historical price movements using a short stack. A curve sequence is amended/updated efficiently after each swap by trimming and appending new members to the end through the algorithm discussed in this section.*
- *Pool prices in chronological order: The curve sequence members are the recorded pool prices in chronological order. The value of every member of the curve sequence is in the range between the value of the previous two members. The first two members of the curve sequence are the boundaries of the active liquidity interval. The last member is the most recently appended value.*
- *Similarity to order book charts: The shape of* $\mathsf{w}(q)$ *preserves the desired feature of increasing liquidity as the pool price moves away from the current price toward the boundaries of the liquidity interval.*
- *Historical trading data: It can facilitate:*
  - *For LPs: Higher profits by applying different slippage on the offered liquidity in two directions based on recent trading activity.*
  - *For traders: Maintaining recent trading activity can facilitate strategic trading decisions.*

Draft - NoFeeSwap

3.2 Protocol Parameters [7]

There are various pool-level parameters that are accessed as part of every swap call. These parameters are divided into two categories: i) Dynamic parameters that are written in singleton's storage, ii) Static parameters that are stored as the bytecode of a so-called "storage contract", and can be accessed via the opcode `extcodecopy`. In the following, we explain the aforementioned ones, as well as a few protocol-level parameters:

Section 3.2.1 explains the identification parameters of each liquidity pool. Sections 3.2.2 and 3.2.3 provide the dynamic and static parameters, respectively.[33] Section 3.2.4 presents the parameters related to the splitting of liquidity growth for pools. Finally, Section 3.2.5 briefly discusses the global protocol slot.

3.2.1 poolId

`poolId` is the pool's identifier, a unique and *immutable* 256-bit for each pool that is set at the time of pool initialization. Figure 13 shows its structure.

- *Hook Address:* The least significant 160 bits refer to the hook address.
  - If a hook is intended, these 160 bits should refer to the hook contract's address. In this case, at least one hook flag[34] must be set.
  - If no hook is intended, these 160 bits must be zero. All of the hook flags must be zero, as well.

    Remark 15. *Each pool can select up to one hook contract at the initialization (and use zeros as hook address if no hook is desired). But, each hook address can be used by multiple pools.*

- *Flags:* The next 20 bits are referred to as flags. Each of these flags is discussed in this section.

  Remark 16. *Of the* 20 *flags, the least significant* 17 *flags are for hooks. The* 3 *most significant ones are non-hook flags, i.e.,* `isMutableKernel`*,* `isMutablePoolGrowthPortion` *and* `isDonateAllowed`*.*

- *Log of price offset:* The next 8 bits represent the natural logarithm of the price offset which must be an integer in the `int8` representation in the range $[-89,\ +89]$. This covers the range $[e^{-89},\ e^{+89}]$ for the price offset. The real-world price of a pool can be within the range $(e^{-16}p_{\text{offset}},\ e^{+16}p_{\text{offset}})$, as in Remark 7.

  Remark 17. *A candidate for the logarithmic offset value is the natural logarithm of the intended real-world pool price*[35] *at initialization. This choice*

[33] These are variables that are tracked in storage and are required for every swap call.
[34] See Remark 16.
[35] The real-world pool price is typically around the market price. See equation (11).

Draft - NoFeeSwap

allows the pool to be able to cover real-world prices within $e^{\pm 16}$ of the initial pool price. Alternatively, one may simply input 0 as the logarithm of the offset, in which case, the range of the pool's real-world price is $(e^{-16},\ e^{+16})$.

- *Salt:* The remaining (most significant) 68 bits can be chosen arbitrarily by the pool creator. Since the `saltedPoolId` is a deterministic function of these 68 bits, they can be brute-forced in order to find any desirable pattern for the most significant 68 bits of `poolId`. The precise formula for this will be given later in this document.

3.2.1.1 Flags. Every time a hook is invoked, a snapshot of memory is handed to the hook via calldata. This way, the hook contract can access the parameters in the protocol's memory within its calldata. This includes any input of the call to the protocol.

In the following, each flag is explained. The least significant 17 flags are the permissions/accesses that the pool creator has determined[36] to provide to the hook. Flags, starting from the least significant bit, are as follows:

1. `isPreInitialize`. If this flag is active, the hook is called before the pool initialization, with access to input data.[37]

2. `isPostInitialize`. If this flag is active, the hook is called after the pool initialization, with access to all of the parameters that are used to construct the pool.

   Remark 18. *If there is a hook, at least one of the* `isPreInitialize` *or* `isPostInitialize` *flags is enforced to be active.*

3. `isPreMint`. If this flag is active, the hook is called before minting positions, with access to input data.

4. `isMidMint`. If this flag is active, the hook is called before minting positions, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).

5. `isPostMint`. If this flag is active, the hook is called after minting positions, with access to all of the parameters.

[36] during the pool initialization

[37] However, for all "pre" flags, the pool parameters are not passed to the hook via calldata and can only be accessed externally.

Draft - NoFeeSwap

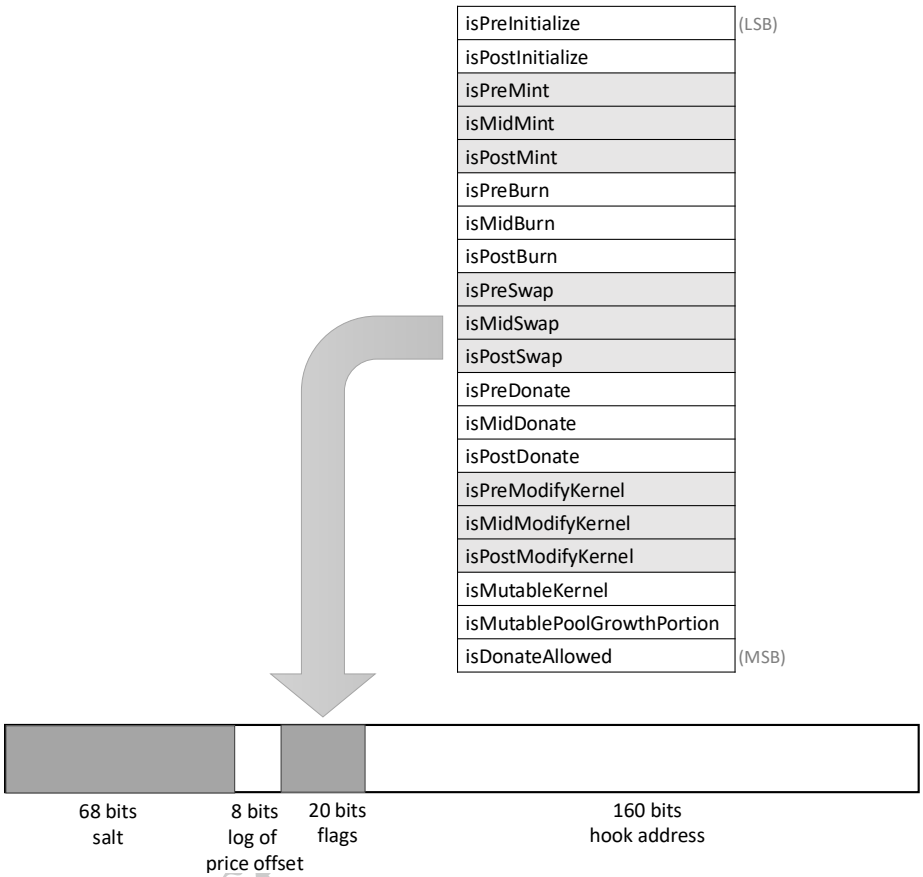


Figure 13: The 256-bit structure of `poolId`.

Draft

6. `isPreBurn`. If this flag is active, the hook is called before burning positions, with access to input data.

7. `isMidBurn`. If this flag is active, the hook is called before burning positions, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).

8. `isPostBurn`. If this flag is active, the hook is called after burning positions, with access to all of the parameters.

9. `isPreSwap`. If this flag is active, the hook is called before swaps, with access to input data.

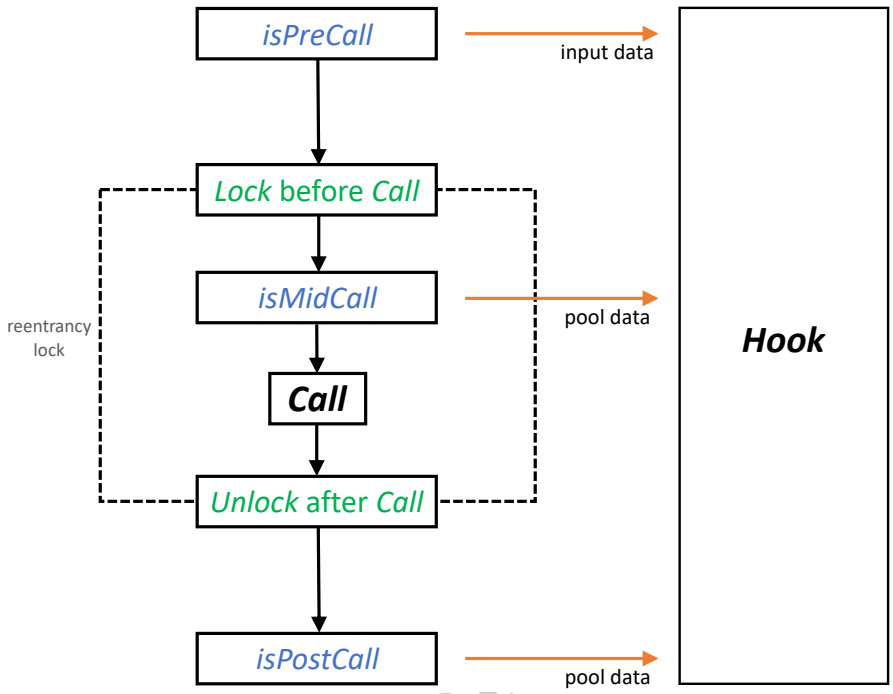


Figure 14: The placement of hooks for `mint`/`burn`/`swap`/`donate`/`modifyKernel` calls.

Draft - No

10. `isMidSwap`. If this flag is active, the hook is called before swaps, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).

11. `isPostSwap`. If this flag is active, the hook is called after swaps, with access to all of the parameters.

12. `isPreDonate`. If this flag is active, the hook is called before donating, with access to input data.

13. `isMidDonate`. If this flag is active, the hook is called before donating, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).

14. `isPostDonate`. If this flag is active, the hook is called after donating, with access to all of the parameters.

15. `isPreModifyKernel`. If this flag is active, the hook is called before a pending kernel is selected, with access to input data.

16. `isMidModifyKernel`. If this flag is active, the hook is called before a pending kernel is selected, with access to all of the parameters. However, the hook cannot interact with the pool, due to a reentrancy lock (see Figure 14).

17. `isPostModifyKernel`. If this flag is active, the hook is called after a pending kernel is selected, with access to all of the parameters.

18. `isMutableKernel`. If this flag is active, pool's `kernel` is modifiable by the pool owner. If this flag is inactive, `isPreModifyKernel`, `isMidModifyKernel` and `isPostModifyKernel` flags should be inactive as well.

19. `isMutablePoolGrowthPortion`. If this flag is active, `poolGrowthPortion` is modifiable by the pool owner. Otherwise, it stays as the amount that is picked at the time of pool initialization.

20. `isDonateAllowed`. If this flag is active, `donate` to the active liquidity interval is permitted.

Draft - NoFeeSwap

Example 4. *Assume that* `poolId` *of a pool is selected as:*

`0x0000000900000000020A0005 2260FAC5E5542A773AA44FBCFEDF7C193BC2C599`.

in hexadecimal representation.

- *The least significant 40 hexadecimal digits*[38] *(blue) represent the address of the hook. So, the hook address is:*

  `0x2260FAC5E5542A773AA44FBCFEDF7C193BC2C599`.

- *The green digits (*`0xA0005`*) indicate the flags with the binary form:*

  `0b10100000000000000101`.

  In this case, `isPreInitialize`*,* `isPreMint`*,* `isMutableKernel` *and* `isDonateAllowed` *flags are activated. These mean that* `isPreInitialize`

[38] Each hexadecimal digit is 4 bits.

and `isPreMint` *hook flags are activated,* `kernel` *can be modified by the pool owner in the future, and that any address is able to donate to the pool's active liquidity interval.*

- *The red digits (*`0x20`*) indicate an* `int8` *representation*[39] *of the offset's natural logarithm with the binary representation of:*

$$\texttt{0b00100000},$$

which correspond to $q_{\text{offset}} = 32$*, and, therefore,* $p_{\text{offset}} = e^{32} \sim 7.9 \times 10^{13}$*. Alternatively, if the* `int8` *representation value is* `0xF2`*, the binary form becomes:*

$$\texttt{0b11110010},$$

and the corresponding values are $q_{\text{offset}} = -14$ *and* $p_{\text{offset}} = e^{-14} \sim 8.3 \times 10^{-7}$.

- *The salt bits are* `0x00000009000000000`*. This is explained further in the next section.*

3.2.1.2 Salt. The most significant 68 bits of the `unsaltedPoolId` do not impact any of the pool's attributes. `unsaltedPoolId` is an `uint256` input parameter of the initialization method of `NofeeswapDelegatee.sol`. The `poolId` is derived from `unsaltedPoolId` and `msg.sender` via the following deterministic formula:

$$\texttt{poolId} = \texttt{unsaltedPoolId} + (\texttt{keccak256}(\texttt{abi.encodePacked}(\texttt{msg.sender}, \texttt{unsaltedPoolId}))) << 188, \tag{55}$$

where '+' denotes addition modulo 2^{256} and the '<<' is equivalent to the EVM opcode `shl`. This way, `poolId` can be determined/mined a priori.

After initialization, `poolId` is used as the sole identifying parameter of every pool for all purposes.

[39] The natural logarithm of price offset must be an integer in the range $[-89, +89]$.

Draft - NoFeeSwap

3.2.2 Dynamic Parameters

These parameters are subject to frequent updates with `swap` calls. In the following, the usage and update process of each is explained.

- `growth`. (128-bit, type `X111`)[40] If a swap call touches an interval, then the amount of liquidity in that interval grows. In other words, a single share in that interval becomes more valuable after the swap. The variable `growth` is the cumulative growth for the active liquidity interval and its value is greater than or equal to 1 (corresponding to 2^{111} in storage). The highest possible growth value is 2^{16} (corresponding to 2^{111+16} in storage). At the time of pool initialization, this value is set to 1 and is non-decreasing over time. The steps of updating `growth` with each swap are discussed in Section 3.5.

  Remark 19. *The type* `X#` *means that the original number is multiplied by* $2^{\#}$ *in storage and preserves a floating value with* `#` *binary places, i.e., the binary representation is shifted to the left by* `#` *digits. See Section 3.7 for more information on data types.*

  Remark 20. *Section 3.3.2 and Figure 18 illustrate how the growth for the liquidity intervals adjacent to the active interval is retrieved, which is used during interval transitions.*

  Remark 21. *The mechanism of* `donate` *gives the deposited amount to the LPs in the active liquidity interval and increases the growth parameter accordingly, such that the deposited amount is effectively divided between the LPs of the active interval proportional to their share counts, without changing the number of shares that each LP owns.*

  Remark 22. *The growth in the active liquidity interval increases by* `donate` *and* `swap` *calls. A portion of this growth goes to the protocol, and a portion of the remaining growth goes to the pool owner. The parameters in Section 3.2.4 are used to track the growth portions of the protocol and the pool owners.*

[40] As in Section 3.7, `X#` types are defined as 256-bit values, however, the parameters here are maintained in the most packed form for efficiency purposes. Only the lowest number of bits required to maintain the values is recorded in storage.

Draft - NoFeeSwap

- `integral0/1`. (216-bit each, type `X216`) These are used for efficient token amount calculations. These values are updated in the storage once after each swap. These integrals are defined as:

$$\text{integral0} \triangleq \int_{q_{\text{current}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))dh, \tag{56a}$$

$$\text{integral1} \triangleq \int_{q_{\text{lower}}}^{q_{\text{current}}} e^{+\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))dh. \tag{56b}$$

Considering equations (28) and (53) and Remark 7, the LPs' total reserve amounts in the active liquidity interval can be calculated for both tokens as:

$$\text{interval reserve } 0 = \frac{1}{\sqrt{p_{\text{offset}}}} \times \frac{\text{sharesTotal} \times \text{growth}_{\text{t}}}{\text{outgoingMax}_{\mathsf{k}}} \times \text{integral0}, \tag{57a}$$

$$\text{interval reserve } 1 = \sqrt{p_{\text{offset}}} \times \frac{\text{sharesTotal} \times \text{growth}_{\text{t}}}{\text{outgoingMax}_{\mathsf{k}}} \times \text{integral1}. \tag{57b}$$

Remark 23. *A multiplier of* $\frac{e^{-8}}{2}$ *scales the integrals in equations* (56a)-(56b) *to stay below* 1.

$$\frac{1}{2}e^{-8} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{\pm\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))dh < \frac{1}{2}e^{-8} \int_{-16}^{16} e^{\frac{h}{2}} dh < 1. \tag{58}$$

Remark 9 explains that this document omits such scaling.

Keeping track of these integrals and updating them in the contract's storage with each swap is more gas-efficient than a full calculation of them with each swap. These integrals are of type `X216` and since they are between 0 and 1, they are represented with exactly 216 bits.

- `sharesTotal`. (128-bit, type `uint256`) It is the total number of shares in the active liquidity interval. It can change only with `modifyPosition` (as shares are added or removed) and `transition` to an adjacent liquidity interval (as the number of shares in another liquidity interval needs to be specified). Section 3.3.1 and Figure 17 explain how the number of shares is retrieved or updated for inactive liquidity intervals.

  Equation (57) shows the relations between `sharesTotal` and the reserve amounts in the active liquidity interval.

Draft - NoFeeSwap

- `staticParamsStoragePointer`. (16-bit, type `uint16`) It is a pointer to the static parameters of the pool.[41] Each time the static parameters of a pool change, the protocol stores those parameters using SSTORE3[42]. And, this pointer is automatically updated to keep track of the latest version of the data.

  Every time the `modifyKernel` function introduces a pending kernel, or an interval transition causes a pending kernel to be activated, or when any of the growth parameters `poolGrowthPortion`, `maxPoolGrowthPortion` or `protocolGrowthPortion` are updated, this reference pointer is updated as well.[43]

  To access the latest version of the static parameters, a 160-bit address of the storage contract is needed. The address is calculated using this pointer, which is only 16 bits (2 bytes) and fits in among the dynamic parameters.

  Remark 24. *The protocol's mechanism for this update is internal to the system and can be skipped for most readers: Each time the static parameters are updated, a proxy contract is deployed with* `create2`*. The proxy's address,* `proxyAddress`*, is calculated by concatenating and hashing the following values, into 20 bytes (160 bits):*

  ```
  0xff (1 byte)
  NofeeswapAddress (20 bytes)
  salt (32 bytes)
  PROXY_CREATION_HASH (32 bytes)
  ```

  where `salt` *is calculated by concatenating and hashing:*

  ```
  staticParamsStoragePointer (2 bytes)
  poolId (32 bytes)
  ```

Draft - NoFeeSwap

[41] See Section 3.2.3.

[42] Inspired by "Philogy" and "Solady"

[43] See the list of static parameters in Section 3.2.3. `maxPoolGrowthPortion` and `protocolGrowthPortion` are the pool's local versions that can be updated by anyone to sync with the `Sentinel` contract (which is modifiable through governance). `poolGrowthPortion` is modifiable only by the pool owner. Each time an interval transition occurs and a pending kernel is activated, `kernel`, `pendingKernelLength`, `outgoingMax`, `outgoingMaxModularInverse` and `incomingMax` are updated. Tags, price offset, and price spacing are immutable and cannot be modified after pool initialization.

The proxy's sole purpose is a one-time deployment of a second contract in which static parameters are encoded. The address of this contract is calculated by concatenating and hashing the following values, into 20 bytes (160 bits):

```
0xd69401 (6 bytes)
proxyAddress (20 bytes)
```

Given `staticParamsStoragePointer`*, which is only 16 bits, one can deterministically calculate the storage contract's address and retrieve static parameters via* `extcodecopy`*.*

- `logPriceCurrent`. (64-bit, type `X59`) It is the latest price of the pool, and therefore, the last member of the curve sequence.

  Remark 25. `sharesTotal`, `staticParamsStoragePointer` *and* `logPriceCurrent` *are placed next to each other in storage, within the third slot of dynamic parameters. Therefore, they can be read on-chain efficiently by reading a single slot of storage.*

- `staticParamsStoragePointerExtension`. (256-bit, type `uint256`) It is a pointer to the pool's static parameters. This is an internal component to facilitate updating the static parameters in the uncommon cases when the number of updates/counter reaches $2^{16}-1$, i.e., `staticParamsStoragePointer` reaches its maximum. More precisely:

  - If `staticParamsStoragePointer < type(uint16).max`, this pointer's value effectively is `staticParamsStoragePointer`.
  - `staticParamsStoragePointer == type(uint16).max` indicates that the pointer occupies more than 16 bits and for which the singleton's storage slot `getDynamicParamsSlot(getPoolId()) - 1` is used.

- `curve`. It is a sequence of 64-bit values of type `X59`. Each 256-bit slot fits 4 members of the `curve` sequence that are log-prices of type `X59` in offset-binary representation, i.e., 0 corresponds to the real-world price of $p_{\text{offset}} \times e^{-16}$ and `0x8000000000000000` corresponds to the real-world price of p_{offset}. See Table 1 for more details. `curve` sequence should

Draft - NoFeeSwap

have at least two log-price members. The first and second 64-bit members are the boundaries of the active liquidity interval. The last 64-bit member of the `curve` sequence is the current log-price of the pool, i.e., `logPriceCurrent`. The ordering rules discussed in Section 3.1.2 are verified upon the initialization of a pool and are preserved by each update of the `curve` sequence.

3.2.3 Static Parameters

These are pool parameters that typically do not change with frequent operations such as `swap` and `modifyPosition`. Some are immutable, some are modifiable only by the pool owner, some are syncable with the protocol by any address, and some are updated when a new kernel is provided or activated. SSTORE3 is used to read `kernel` and other static parameters efficiently.

In this document, occasionally, *tags* and *tokens* are used interchangeably. Tag emphasizes the fact that trading can be beyond ERC-20 tokens and can include various other tradables, a.k.a., multi-tokens.

- `tag0/1`. (256-bit each, type `Tag`) These refer to the two tokens in a pool, and traders can swap from one to the other.[44] There is a lot of flexibility on what can be traded in the NoFeeSwap protocol.[45] Type of each tag may refer to any of the followings:
  - **Native token**. For example, ETH is Ethereum's native token. The native token's tag value is zero.
  - **ERC-**20. The 160-bit token address is upcast to a 256-bit tag.
  - **ERC-**1155. The value of the tag is determined by hashing `tokenAddress` (160 bits) and `tokenId` (256 bits):
    `keccak256(encodePacked(tokenAddress, tokenId))`.
  - **ERC-**6909. The value of the tag is determined by hashing `tokenAddress` (160 bits) and `tokenId` (256 bits):
    `keccak256(encodePacked(tokenAddress, tokenId))`.

[44] In this document, occasionally, *tags* and *tokens* are used interchangeably. Tag emphasizes the fact that trading can be beyond ERC-20 tokens and can include various other tradables, aka, multi-tokens.

[45] The accounting of tags in NoFeeSwap's singleton is compliant with ERC-6909 multi-token specifications.

Draft - NoFeeSwap

- **NoFeeSwap LP token**. The tag value is determined by hashing `poolId` (256 bits) and min/max log-prices of the position (each 256 bits): `keccak256(encodePacked(qmin, qmax, poolId)).qmin` and `qmax`, here, are in the `X59` two's complement representation, as in Table 2. See `X59` in Section 3.7 and Remark 50.

  Remark 26. *LP tokens from pools with no hook can be traded as above. However, this is not the case in some situations, depending on the hook. For example, incentive hooks discussed in Section 6.1 hold on to this LP token and, in return, give out an NFT that cannot be traded as a tag.*

The hashing here allows `tag` to fit in a 256-bit as an identifier. The tags of a pool are immutable and cannot be modified after the pool initialization.

Remark 27. *In every pool, the tag with an arithmetically smaller* 256*-bit is* `tag0` *and the other one is* `tag1`.

Remark 28. *The first two tag types from the above list are available in other DEXs. NoFeeSwap provides more flexibility and facilitates trading beyond ERC-*20 *tokens.*

Remark 29. *The concept of tag enables trading beyond ERC-*20 *in a DEX, i.e., it facilitates trading the protocol's LP tokens as well as ERC-*1155 *and ERC-*6909 *multi-tokens. This feature enables the extension of the DEX capability to manage combinations of NFTs and semi-fungible tokens.*

- `sqrtOffset`/`sqrtInverseOffset`. (256-bit each, type `X127`) These are immutable and cannot be modified after the initialization. See Remark 17 for relevant explanations on picking offset. The justification for adding these parameters is discussed in Remark 7.

  Remark 30. *Section 3.2.1 explains that the price offset's natural logarithm is encoded in* `poolId`*. See Example 4. The purpose of including these two additional values in the static parameters is to optimize the calculations.*

- `spacing`. (496-bit) Define $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}}$ as the width of each liquidity interval, which is immutable and cannot be modified after initialization. This 496-bit memory space occupies:

  - `logSpacing` (64-bit)

Draft - NoFeeSwap

- `sqrtSpacing` (216-bit, type `X216`)
- `sqrtInverseSpacing` (216-bit, type `X216`)

which can be accessed via `PriceLibrary`. These, respectively, represent:

$$q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}}, \tag{59a}$$

$$\frac{1}{\sqrt{p_{\text{spacing}}}} = e^{-\frac{q_{\text{spacing}}}{2}} = e^{-\frac{q_{\text{upper}} - q_{\text{lower}}}{2}}, \tag{59b}$$

$$\frac{\sqrt{p_{\text{spacing}}}}{e^{+16}} = e^{-16+\frac{q_{\text{spacing}}}{2}} = e^{-16+\frac{q_{\text{upper}} - q_{\text{lower}}}{2}}. \tag{59c}$$

It is straightforward to verify that $-16 < q_{\text{lower}} < q_{\text{upper}} < 16$[46] assures that the last two parameters stay between 0 and 1.

Remark 31. *q_{upper} and q_{lower} refer to the boundaries of the active liquidity interval and should not be confused with q_{min} and q_{max}, which are used to refer to the boundaries of a liquidity position in this document.*

- `outgoingMax`. (216-bit, type `X216`) It is calculated as in equation (43). It is updated when a new `kernel` is activated. There are two usages for this parameter: 1) When a pending `kernel` is activated, this is recalculated, as a normalizer, to keep the rest of the calculations intact and invariant with respect to updates of `kernel`; 2) For large trades, the liquidity intervals can be efficiently crossed by using this value as opposed to calculating the integrals.

  Since the protocol performs division by `outgoingMax` frequently, its modular inverse is included as well.

- `outgoingMaxModularInverse`. (256-bit, type `uint256`) It is the modular inverse of `outgoingMax`$/2^k$ modulo 2^{256}, where k is the largest power of 2 that divides `outgoingMax`.

- `incomingMax`. (216-bit, type `X216`) It is similar to `outgoingMax` but for the incoming token (the one that the pool receives from a swapper) as

[46] as in Remark 7

Draft - NoFeeSwap

opposed to the outgoing token. This parameter is defined as follows:

$$\begin{aligned}\text{incomingMax}_{\mathsf{k}} &\triangleq e^{-\frac{q_{\text{upper}}}{2}} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{+\frac{h}{2}} k(h - q_{\text{lower}}) \\ &= e^{+\frac{q_{\text{lower}}}{2}} \int_{q_{\text{lower}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} k(q_{\text{upper}} - h) \\ &= \int_{0}^{q_{\text{spacing}}} e^{+\frac{h}{2}} \mathsf{k}(h) dh, \end{aligned} \tag{60}$$

which is a constant associated with the kernel. Similarly to `outgoingMax`, in (43), the value of this parameter is equivalent for all liquidity intervals. Here, for simplicity, it is written only for the active interval.

Similarly to `outgoingMax`, this is updated when a new `kernel` is activated. The usage of this is that for large trades, the liquidity intervals can be efficiently crossed by using this as opposed to calculating the integrals.

- `poolGrowthPortion`. (48-bit, type `X47`) It is the portion of liquidity growth which the pool owner takes. This portion is capped by the value `maxPoolGrowthPortion` (discussed next), determined by the owners/governance of the protocol token.

  If the `isMutablePoolGrowthPortion` flag is on during the pool initialization, the pool owners can change this pool parameter through the `modifyPoolGrowthPortion` function in the `NofeeswapDelegatee` contract.

- `maxPoolGrowthPortion`. (48-bit, type `X47`) It is an upper limit on the portion that the pool owner can take from the liquidity growth. It is the pool's local copy of this parameter. See Section 4.2.

  Remark 32. *Trading professional pool owners can regularly design and update kernels, benefiting other LPs in the pool, and receive a portion of the pool's growth in return. Note that the protocol governance can set any cap on the pool owner's portion.*

- `protocolGrowthPortion`. (48-bit, type `X47`) It is the portion of liquidity growth that goes to the protocol. It is the pool's local copy of this parameter.

Draft - NoFeeSwap

Remark 33. *The global versions of parameters* `maxPoolGrowthPortion` *and* `protocolGrowthPortion`*, for every pool, are under the control of the protocol and are modifiable by the protocol's governance.*

After a change of those global parameters, any address can run the function `updateGrowthPortions` *of* `NofeeswapDelegatee` *for any pool to update growth portion parameters in that pool's storage and make it in sync with the protocol's global version. See Section 3.2.5 and Section 4.2.*

Remark 34. *Currently,* `protocolGrowthPortion` *is set to zero for all pools. Protocol governance can increase this to a non-zero amount.*

Remark 35. *The growth portions of the protocol and the pool owner are kept track of and can be withdrawn as mentioned in Section 3.2.4. The protocol first applies the protocol's growth portion, and then, the pool's growth portion, i.e., the growth is split into three components as below:*

<u>LPs′ Portion</u>

$$\begin{aligned}\text{growth}_{\text{post}} - \text{growth}_{\text{pre}} = &(\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times (1 - \text{protocolGrowthPortion}) \times (1 - \text{poolGrowthPortion[capped]}),\end{aligned} \tag{61a}$$

<u>Protocol′s Portion</u>

$$\begin{aligned}\text{marginalGrowth}_{\text{protocol}} = &(\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times \text{protocolGrowthPortion},\end{aligned} \tag{61b}$$

<u>Pool Owner′s Portion</u>

$$\begin{aligned}\text{marginalGrowth}_{\text{pool}} = &(\text{growth}_{\text{postNoSplit}} - \text{growth}_{\text{pre}}) \\ &\times (1 - \text{protocolGrowthPortion}) \times \text{poolGrowthPortion[capped]},\end{aligned} \tag{61c}$$

where

- $\text{growth}_{\text{pre}}$ *represents the accumulated growth for LPs before the current swap;*
- $\text{growth}_{\text{postNoSplit}}$ *represents the accumulated growth for LPs after the current swap if there no growth splitting is applied, i.e., it is the LPs' growth under the assumptions of* $\text{protocolGrowthPortion} = 0$ *and* $\text{poolGrowthPortion[capped]} = 0$*, which is the growth derived in Section 3.5;*

Draft - NoFeeSwap

- $\mathrm{growth}_{\mathrm{post}}$ *represents the accumulated growth for LPs after the current swap (where growth splitting is taken into account);*
- $\mathrm{poolGrowthPortion}[\mathrm{capped}]$ *is the effective growth portion for the pool owner, i.e.,* $\min\{\mathrm{maxPoolGrowthPortion}, \mathrm{poolGrowthPortion}\}$*;*
- $\mathrm{marginalGrowth}_{\mathrm{protocol}}$ *represents the marginal growth for the protocol from the current swap;*
- $\mathrm{marginalGrowth}_{\mathrm{pool}}$ *represents the marginal growth for the pool owner from the current swap.*

It is straightforward to verify that the amount of growth from the current swap which LPs do not receive due to splitting, $\mathrm{growth}_{\mathrm{postNoSplit}} - \mathrm{growth}_{\mathrm{post}}$, is equal to the combined growth of the protocol and the pool owner from the current swap, $\mathrm{marginalGrowth}_{\mathrm{protocol}} + \mathrm{marginalGrowth}_{\mathrm{pool}}$.

- `pendingKernelLength`. (16-bit, type `Index`) Keeps track of the number of break-points of the pending kernel. If it is 0, it means that there are no pending kernels.
- `kernel`. It is defined in Section 3.1.1. It governs the distribution of liquidity across the price horizon. We store kernels by listing their break-points *except for the origin*[47]. For example, a kernel stored with a single break-point, $\{(q_{\mathrm{spacing}}, 1)\}$, represents a simple ramp that connects the origin, $(0, 0)$, to the point $(q_{\mathrm{spacing}}, 1)$. In addition, $\{(\frac{q_{\mathrm{spacing}}}{2}, 0), (\frac{q_{\mathrm{spacing}}}{2}, 1), (q_{\mathrm{spacing}}, 1)\}$ represents a step function with a jump in $\frac{q_{\mathrm{spacing}}}{2}$.

  Remark 36. *A wide variety of kernel functions can be constructed. The break-points are provided through a* `kernelCompactArray` *on initialization of a pool or when selecting a new pending kernel. A pool owner can provide a new kernel in compact form through the* `modifyKernel` *function. Figure 15 shows the structure for one break-point. In the compact form, which is how a pool owner provides the input, for each break-point, a* 64*-bit horizontal coordinate (log-price,* q*) is given in* `X59` *representation, and a* 16*-bit vertical coordinate, corresponding to kernel's intensity,* $\mathsf{k}(q)$*, is provided in* `X15` *representation. Hence, each break-point takes* 80 *bits in the compact form. Break-points should be tightly packed within a* `uint256[]` *array with their height appearing first. Examples of kernels with a few break-points are provided in Figure 6.*

Draft: NoFeeSwap

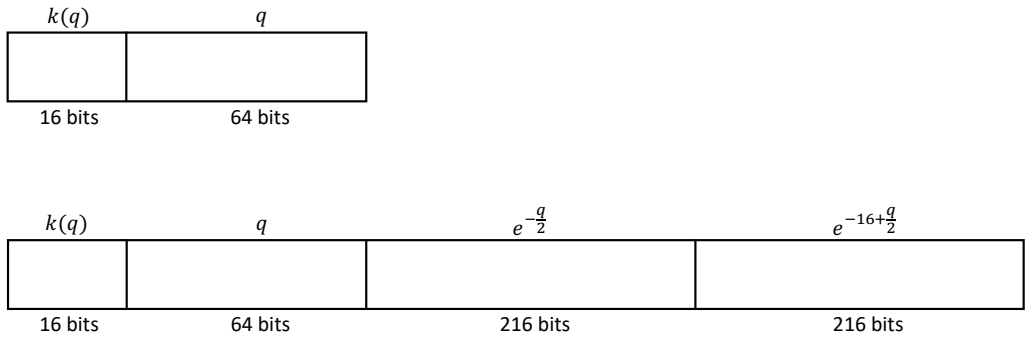


Figure 15: (a) Structure of each break-point in kernel's compact form, `kernelCompactArray`, 80 bits (top); (b) Structure of each break-point in kernel's expanded form, `kernel`, 512 bits (bottom).

Draft - NoFreeSwap

The expanded version of the kernel includes two additional terms for each break-point (for computational efficiency), i.e., $e^{-\frac{q}{2}}$ and $e^{-16+\frac{q}{2}}$. These are always less than 1 as $0 < q < \frac{32}{3}$.[48] Since they are always less than 1, they are stored in `X216` representation with 216 bits. Hence, each `kernel` break-point takes 512 bits ($16 + 64 + 216 + 216$).

Remark 37. *Assume that the pool owner inputs* $\mathsf{k}(q_1), q_1, \mathsf{k}(q_2), q_2, \mathsf{k}(q_3), q_3, \ldots, \mathsf{k}(q_N), q_N$ *for* `kernelCompactArray`. *Note that* $(\mathsf{k}(q_0), q_0) = (0, 0)$ *is always considered the first member of* `kernel` *and is not entered. Below are the input requirements:*[49]

- $q_{i-1} = q_i = q_{i+1}$ *must be avoided for any* $1 \leq i \leq N-1$.
- $\mathsf{k}(q_{i-1}) = \mathsf{k}(q_i) = \mathsf{k}(q_{i+1})$ *must be avoided for any* $1 \leq i \leq N-1$.
- $(\mathsf{k}(q_i), q_i) = (\mathsf{k}(q_{i+1}), q_{i+1})$ *must be avoided for any* $0 \leq i \leq N-1$.
- $q_{i+1} < q_i$ *must be avoided for any* $0 \leq i \leq N-1$.
- $\mathsf{k}(q_{i+1}) < \mathsf{k}(q_i)$ *must be avoided for any* $0 \leq i \leq N-1$.
- $q_1 = 0$ *must be avoided, i.e., a vertical jump from the origin is not permitted.*

[47] The exclusion is only for efficiency purposes as the origin is always the first member.

[48] As in Section 3.1.1, all q's of the kernel are between 0 and q_{spacing}. Also, as in Remark 55, $q_{\text{spacing}} < \frac{32}{3}$.

[49] Otherwise, the initialization or the kernel update call is **reverted**.

- $q_{N-1} = q_N$ *must be avoided, i.e., a vertical jump to the last point is not permitted.*
- $\mathsf{k}(q_N) = 1^{50}$ *and* $q_N = q_{\text{spacing}}$ *must hold.*
- $q_{i+1} - q_i \geq 2^{-27}$ *should hold*[51] *for each* $0 \leq i \leq N-1$ *where* $q_{i+1} \neq q_i$*, to ensure the robustness of the numerical calculation and to avoid catastrophic cancellation.*

3.2.4 Growth Sharing Parameters

Growth in the active liquidity interval increases by `donate` and `swap` calls, and the below parameters are used for keeping track of the portions for the protocol and the pool owners.

- `accrued0/1`. (104-bit each) These values account for the total accrued amounts of growth portions with respect to `tag0/1`, owed to the pool owner and the protocol, collectively.

  The equations in Remark 35 are in terms of growth. They should be multiplied by the number of shares and other parameters for determining the token amounts, similar to equations (45), (47), (57).

- `poolRatio0/1`. (24-bit each, type `X23`) These are the ratios of accumulated values belonging to the pool (and the rest belonging to the protocol).

These 4 parameters are packed in one 256-bit slot, as $(104 + 24) \times 2 = 256$.

Remark 38. *To avoid overflows,* `sharesGross` *is a storage parameter that represents the sum of share counts across all liquidity intervals of a pool, with the maximum permitted amount of* $2^{127} - 1$ *for each pool.*

3.2.5 Protocol Slot

Protocol slot is a 256-bit space in storage that stores the global growth portion parameters that are subject to change through protocol governance, i.e., the default values for `maxPoolGrowthPortion` and `protocolGrowthPortion`. These

[50] `X15` representation for this value is 2^{15}.

[51] The `X59` representation for this difference value is $2^{-27+59} = 2^{32}$. This limit corresponds to $\frac{p_{i+1}}{p_i} \geq e^{(2^{-27})} \sim 1.00000000745$.

Draft - NoFeeSwap

can also be overridden by the `Sentinel` contract[52] (which is also modifiable through protocol governance) to allow customized growth portions for each pool or token pair. The protocol slot also stores the address of the protocol owner, which is currently `Timelock`. See Figure 16.

This is not read in every call. There is a local version of the growth portion parameters kept for each pool, as in Section 3.2.3.

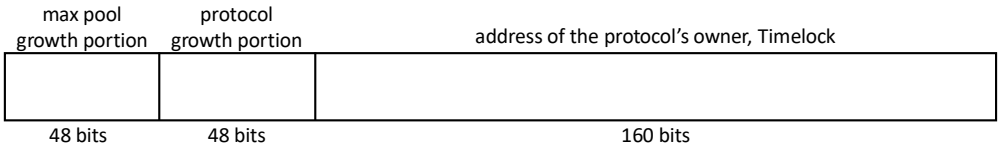


Figure 16: Protocol slot (48+48+160=256 bits).

Draft - NoFeeSwap

3.3 Mappings

There are two sets of mappings on the boundaries of the liquidity interval, `sharesDelta` and `growthMultiplier`, enabling efficient updates of the share and growth amounts when there is a transition to an adjacent liquidity interval. They are also used for handling `modifyPosition` calls.

3.3.1 sharesDelta

`sharesDelta` is defined on the boundaries of adjacent liquidity intervals and is the net difference of the number of shares in the two intervals. It is of type `int256`. See Figure 17.

3.3.1.1 `sharesTotal` updates with interval transitions. As the active liquidity interval transitions, `sharesDelta` is used to update `sharesTotal`.

- With an interval transition to the left (through q_{lower}), `sharesDelta`$[q_{\text{lower}}]$ should be subtracted from the number of shares in the active interval to derive the number of shares in the new active interval.
- With an interval transition to the right (through q_{upper}), `sharesDelta`$[q_{\text{upper}}]$ should be added to the number of shares in the active interval to derive the number of shares in the new active interval.

[52] Section 4.2

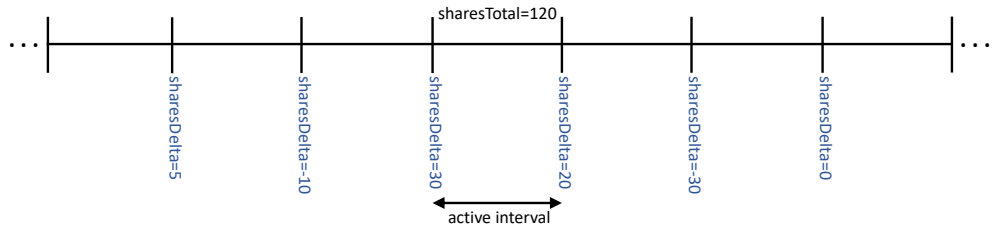


Figure 17: Depicting `sharesTotal` and `sharesDelta` where share counts for these 7 liquidity intervals are $(95, 100, 90, 120, 140, 110, 110)$.

Draft - NoFeeSwap

3.3.1.2 `sharesDelta` updates with `modifyPosition`.

The mappings at q_{min} and q_{max} (the two boundaries of the position) are updated.

3.3.2 growthMultiplier

Each `growthMultiplier` is a slot and is stored in the `X208` representation. For each integer m, let $g(m) \geq 1$ denote the growth value for the liquidity interval:[53]

$$[q_{\text{lower}} + mq_{\text{spacing}}, q_{\text{upper}} + mq_{\text{spacing}}]. \tag{62}$$

The mapping can be calculated as:

$$\text{growthMultiplier}[q_{\text{upper}} + |m|q_{\text{spacing}}] = \sum_{n=0}^{+\infty} g(+n + |m| + 1)e^{-\frac{q_{\text{upper}}+(n+|m|)q_{\text{spacing}}}{2}}, \tag{63a}$$

$$\text{growthMultiplier}[q_{\text{lower}} - |m|q_{\text{spacing}}] = \sum_{n=0}^{+\infty} g(-n - |m| - 1)e^{+\frac{q_{\text{lower}}-(n+|m|)q_{\text{spacing}}}{2}}. \tag{63b}$$

It can be observed that this mapping is defined on every spaced log-price. Note that:

- The mapping terms on the right side of the active liquidity interval record a weighted sum of the growth values for all the liquidity intervals on their right side;

[53] It is $\text{growth}_{\text{post}}$ in Remark 35. The growth updates through swaps are shown in equation (73).

- The mapping terms on the left side of the active liquidity interval record a weighted sum of the growth values for all the liquidity intervals on their left side.

3.3.2.1 Facilitating `modifyPosition`. These values make calculations of the token amounts straightforward. If an LP wants to withdraw a single share from every liquidity interval within spaced log-price $q_{\text{upper}} + mq_{\text{spacing}}$ ($m \geq 0$) to $+\infty$, the number of `token0` owed is equal to:[54]

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \text{growthMultiplier}[q_{\text{upper}} + mq_{\text{spacing}}]. \tag{64}$$

Similarly, the amount of `token1` corresponding to a single share from every liquidity interval within $-\infty$ to spaced log-price $q_{\text{lower}} - mq_{\text{spacing}}$ ($m \geq 0$) is equal to:

$$\sqrt{p_{\text{offset}}} \times \text{growthMultiplier}[q_{\text{lower}} - mq_{\text{spacing}}]. \tag{65}$$

Remark 60 shows how the token amounts for `modifyPosition` are derived from this mapping using (64)-(65).

3.3.2.2 Keeping track of the growth values. The `growthMultiplier` mapping enables keeping track of the growth values for all liquidity intervals. An interval transition requires calculation for the new active interval's growth and updating the transitioned boundary's mapping. Figure 18 shows an interval transition towards the left, including:

1. The new active liquidity interval's growth value is derived as:

$$g(-1) = e^{+\frac{q_{\text{lower}}}{2}}(\text{growthMultiplier}[q_{\text{lower}}] - \text{growthMultiplier}[q_{\text{lower}} - q_{\text{spacing}}]). \tag{66}$$

2. The transitioned boundary's mapping, $\text{growthMultiplier}[q_{\text{lower}}]$, is updated to track the cumulative growth on the right side of the new active liquidity interval:

$$\text{growthMultiplier}[q_{\text{lower}}] = \text{growthMultiplier}[q_{\text{upper}}] + e^{-\frac{q_{\text{lower}}}{2}} g(0). \tag{67}$$

Similarly, after an interval transition towards the right:

[54] This can be shown using equations (63a) and (45).

Draft – NoFeeSwap

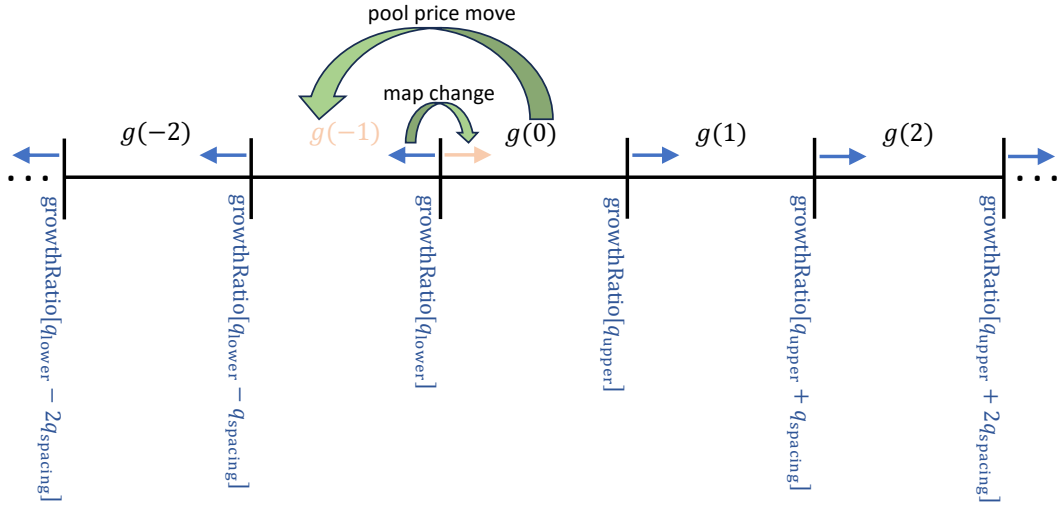


Figure 18: Interaction of growth and growthMultiplier with an interval transition towards left.

Draft - NoFeeS

1. The new active liquidity interval's growth value is derived as:

$$g(1) = e^{+\frac{q_{\text{upper}}}{2}}(\text{growthMultiplier}[q_{\text{upper}}] - \text{growthMultiplier}[q_{\text{upper}} + q_{\text{spacing}}]), \tag{68}$$

2. The transitioned boundary's mapping, $\text{growthMultiplier}[q_{\text{upper}}]$, is updated to track the cumulative growth on the left side of the new active liquidity interval:

$$\text{growthMultiplier}[q_{\text{upper}}] = \text{growthMultiplier}[q_{\text{lower}}] + e^{+\frac{q_{\text{upper}}}{2}} g(0). \tag{69}$$

3.4 Calculation of q_{target} and Swap Amounts

The main input parameters for a swap call are `logPriceLimit`, `amountSpecified`, and `zeroForOne`.

1. `logPriceLimit`: q_{limit} is a limit that the swap does not move the pool's price beyond. If `zeroForOne`[LSB128] $\notin \{0, 1\}$, the swap's direction is implicitly determined from q_{limit}.[55] `logPriceLimit` is in `X59` representation. See Table 2.

[55] If `zeroForOne`[LSB128] $\in \{0, 1\}$, it determines the swap's direction.

2. amountSpecified: This is a specified limit target to a token amount of the swap. Also, its sign determines if the incoming token's amount is specified or the outgoing token's amount.[56] The incoming token refers to the token that the swapper gives to the pool, and the outgoing token refers to the token that the swapper receives from the pool.

   exactInput is a parameter, internal to the protocol, which specifies if amountSpecified determines the value of the incoming token or the outgoing token. Its determination is as follows. Note that the incoming token is received by the pool, and therefore, its amount is positive. Hence, if amountSpecified > 0, it refers to the incoming token's amount, and if amountSpecified < 0, it refers to the outgoing token's amount.

   (a) exactInput $= 0 \iff$ amountSpecified < 0,

   (b) exactInput $= 1 \iff$ amountSpecified > 0.

3. zeroForOne: It is a 256-bit and each half of it is reserved for a separate purpose.

   (a) zeroForOne[MSB128]:[57] It is a threshold on the number of shares that should be available in a liquidity interval for the swap function to transact in it. For example, if zeroForOne$[\text{MSB128}] = 0$, which is the default, no minimum number of shares is imposed.

   For example, if this value is 5 in a swap call, it means that at least 5 LP shares should exist in an interval for the swapper to be willing to trade in that interval.

   Remark 39. *A novel feature is the existence of an optional threshold, picked by swappers, which allows them to avoid trading in low liquidity intervals and to avoid the consequent undesired high slippage and gas costs.*

   (b) zeroForOne$[\text{LSB128}]$:[58]

      i. zeroForOne$[\text{LSB128}] = 0$ means swapper gives token1 and receives token0. Therefore, the pool gives token0 and receives token1.

[56] The incoming token refers to the token that the swapper gives to the pool, and the outgoing token refers to the token that the swapper receives from the pool.

[57] zeroForOne[MSB128] is defined as: $(128 >> \texttt{zeroForOne})$.

[58] zeroForOne[LSB128] is defined as: $((1 << 128) - 1) \wedge \texttt{zeroForOne}$.

Draft - NoFeeSwap

ii. `zeroForOne`[LSB128] $= 1$ means swapper gives `token0` and receives `token1`. Therefore, the pool gives `token1` and receives `token0`.

iii. `zeroForOne`[LSB128] $\notin \{0, 1\}$ means that the trade is "zero for one" if $q_{\text{limit}} < q_{\text{current}}$ and it is "one for zero" if $q_{\text{current}} < q_{\text{limit}}$, i.e., swaps are in the direction of reaching q_{limit}.

Remark 40. *For* `zeroForOne`[LSB128]*, the values of* 0 *and* 1 *are reserved for specifying the swap's direction. In case* `zeroForOne`*[LSB128]*$\in \{0, 1\}$*, then* `logPriceLimit` *should be picked in the direction that* `zeroForOne`*[LSB128] implies, and the swap gets reverted otherwise. Having only these* 2 *choices can be inconvenient for arbitrageurs. A novelty here is to allow swappers to pick* `zeroForOne`*[LSB128]* $\notin \{0, 1\}$*, which signals the swapper's lack of intention to specify a direction. In this case, the swap happens such that the pool price moves towards* `logPriceLimit`*, with no specific direction enforced. This is particularly attractive for arbitrageurs who look for mispricing in various pools, as opposed to swapping in a specific direction.*

Remark 41. *Swaps in arbitrary directions are enabled, which facilitates arbitrage actions that intend to sync the pool price with a target price, e.g., an outside source, with no regard for the direction of the swap.*

Remark 42. *The swap calls can be categorized as follows for their directions:*

- **Price-increasing trades.** *(*`tag1` *is received by the pool)*
  - `zeroForOne`[LSB128] $= 0$*: If* $q_{\text{limit}} > q_{\text{current}}$*, the swap proceeds and otherwise it gets reverted.*
  - `zeroForOne`[LSB128] $\notin \{0, 1\}$ & $q_{\text{limit}} > q_{\text{current}}$.
- **Price-decreasing trades.** *(*`tag0` *is received by the pool)*
  - `zeroForOne`[LSB128] $= 1$*: If* $q_{\text{limit}} < q_{\text{current}}$*, the swap proceeds and otherwise it gets reverted.*
  - `zeroForOne`[LSB128] $\notin \{0, 1\}$ & $q_{\text{limit}} < q_{\text{current}}$.

Numerical Formulation. Depending on the swapper's inputs, one of the following equations is solved for each swap. This determines the size of the swap. The

Draft - NoFeeSwap

corresponding functions are `searchIncoming`[59] and `searchOutgoing`[60]. The derivation of q_{target} for various sets of parameters is as follows:

1. If price-increasing trade & $\texttt{exactInput} = 0$, a specified amount of `tag0` is taken from the protocol in swap. Derive $q_{\text{target}} = \min\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \int_{q_{\text{current}}}^{q_{\text{search}}} e^{-\frac{h}{2}} \mathtt{z}_t(h)dh = |\texttt{amountSpecified}|. \tag{70a}$$

2. If price-increasing trade & $\texttt{exactInput} = 1$, a specified amount of `tag1` is given to the protocol in swap. Derive $q_{\text{target}} = \min\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\sqrt{p_{\text{offset}}} \times \int_{q_{\text{current}}}^{q_{\text{search}}} e^{+\frac{h}{2}} \mathtt{z}_t(h)dh = |\texttt{amountSpecified}|. \tag{70b}$$

3. If price-decreasing trade & $\texttt{exactInput} = 1$, a specified amount of `tag0` is given to the protocol in swap. Derive $q_{\text{target}} = \max\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\frac{1}{\sqrt{p_{\text{offset}}}} \times \int_{q_{\text{search}}}^{q_{\text{current}}} e^{-\frac{h}{2}} \mathtt{z}_t(h)dh = |\texttt{amountSpecified}|. \tag{70c}$$

4. If price-decreasing trade & $\texttt{exactInput} = 0$, a specified amount of `tag1` is taken from the protocol in swap. Derive $q_{\text{target}} = \max\{q_{\text{search}}, q_{\text{limit}}\}$, where q_{search} is calculated through numerical methods from:

$$\sqrt{p_{\text{offset}}} \times \int_{q_{\text{search}}}^{q_{\text{current}}} e^{+\frac{h}{2}} \mathtt{z}_t(h)dh = |\texttt{amountSpecified}|. \tag{70d}$$

Remark 43. `swapWithin` *and* `cross` *are discussed in Section 3.6. Each* `cross` *reduces the absolute value of* `amountSpecified` *to the remaining specified amount. The above equations for finding* q_{search} *are part of* `swapWithin`.

Remark 44. *The above steps determine* q_{target}. *The corresponding swap token amounts are derived using Remarks 2 and 7.*

[59] when $\texttt{exactInput} = 0$
[60] when $\texttt{exactInput} = 1$

Draft - NoFeeSwap

3.5 Calculation of Overshoot & Growth

As demonstrated in Figure 20, there are typically two updates for `curve` sequence after each `swapWithin`. The `curve` sequence amends an overshoot from the target log-price to $q_{\text{overshoot}}$, and then comes back and amends q_{target}.[61] Note that these are parts of the process for updating the `curve` sequence and growth after each `swapWithin`, as explained in the following.[62]

Let $\bar{q}_t$ represent the `curve` sequence before the pool's t^{th} swap, and $\bar{q}_{t+1}$ represent the `curve` sequence after this swap, i.e., after amending it with t^{th} swap's $q_{\text{overshoot}}$ and q_{target}.

Remark 45. *When the pool's price is on q_{target} and* `curve` *sequence is changed from $\bar{q}_t$ to $\bar{q}_{t+1}$ (by appending $q_{\text{overshoot}}$ and q_{target}), the number of shares and tokens does not change in the transition from the old curve to the amended one. Therefore,* $\text{growth} \times \text{integral0}$ *and* $\text{growth} \times \text{integral1}$ *must stay unchanged, according to equation* (57)*, i.e.,*

$$\frac{\text{growth}_{\text{t}}}{\text{growth}_{\text{t+1}}} = \frac{\text{integral0}_{t+1}}{\text{integral0}_t} = \frac{\text{integral1}_{t+1}}{\text{integral1}_t}. \tag{71}$$

Define:

$$f_{\text{left}}(q_{\text{overshoot}}) \triangleq \frac{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}\mathsf{k}(\mathsf{w}_{\bar{q}_{t+1}}(h))}}{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}\mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))}}, \tag{72a}$$

$$f_{\text{right}}(q_{\text{overshoot}}) \triangleq \frac{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}\mathsf{k}(\mathsf{w}_{\bar{q}_{t+1}}(h))}}{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}\mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))}}. \tag{72b}$$

f_{left} and f_{right} represent the inverse of marginal growth of two sides, as in equation (71). The equation $f_{\text{left}} = f_{\text{right}}$ should be solved with respect to $q_{\text{overshoot}}$ to equate the growth values implied from these two fractions.[63]

Assume that the trade is such that the pool price increases, i.e., $q_{\text{current}} \leq q_{\text{target}}$.[64] Then, $q_{\text{overshoot}} \in [q_{\text{target}}, q_{\text{upper}}]$. The following hold due to the specific structure of `curve` sequence updates described in Section 3.1.3.

[61] Therefore, either $q_{\text{current}} \leq q_{\text{target}} \leq q_{\text{overshoot}} \leq q_{\text{upper}}$ or $q_{\text{lower}} \leq q_{\text{overshoot}} \leq q_{\text{target}} \leq q_{\text{current}}$, depending on the swap's direction.

[62] The exchange amounts in the swap are already calculated before such updates, in the first step of Figure 20.

[63] Note that q_{target} is already determined from Section 3.4 and the only remaining variable in `curve` sequence is $q_{\text{overshoot}}$.

[64] The other case can be argued similarly.

Draft - NoFeeSwap

- The extreme end where $q_{\text{overshoot}}$ is equal to q_{target}:
  - $\underline{f_{\text{left}}(q_{\text{target}})} \leq 1$, due to the Hardy–Littlewood inequality, as $\mathsf{w}_{\bar{q}_{t+1}}$ is a decreasing rearrangement of $\mathsf{w}_{\bar{q}_t}$ on the left side of q_{target}, and the exponential term in the integral is monotonically increasing.
  - $\underline{f_{\text{right}}(q_{\text{target}})} = 1$, as $\mathsf{w}_{\bar{q}_{t+1}}$ matches $\mathsf{w}_{\bar{q}_t}$ on the right side of q_{target}. In this case, there is no overshoot and the two liquidity distribution functions are identical within the liquidity interval $[q_{\text{target}}, q_{\text{upper}}]$.
- The other extreme end where $q_{\text{overshoot}}$ is equal to q_{upper}:
  - $\underline{f_{\text{left}}(q_{\text{upper}})} \geq f_{\text{right}}(q_{\text{upper}})$, because $\mathsf{w}_{\bar{q}_{t+1}}$ takes its lowest amounts between q_{target} and q_{upper}, and the kernel is a non-decreasing function.
  - $\underline{f_{\text{right}}(q_{\text{upper}})} \leq 1$, as $\mathsf{w}_{\bar{q}_{t+1}}$ falls below $\mathsf{w}_{\bar{q}_t}$ on the right side of q_{target}. In this case, `curve` sequence is transformed to $[q_{\text{lower}}, q_{\text{upper}}, q_{\text{target}}]$, and therefore, $\mathsf{w}_{\bar{q}_{t+1}}$ on the right side of q_{target} is on the lowest levels after the swap.

$q_{\text{overshoot}}$ is calculated such that $f_{\text{left}} = f_{\text{right}}$ holds for its corresponding $\bar{q}_{t+1}$. The above bullet points along with the Intermediate-Value theorem prove that there is an $q^*_{\text{overshoot}}$ value, $q_{\text{target}} \leq q^*_{\text{overshoot}} \leq q_{\text{upper}}$ such that $f_{\text{left}}(q^*_{\text{overshoot}}) = f_{\text{right}}(q^*_{\text{overshoot}})$.[65][66] This equation is solved iteratively by minimizing a mismatch term between f_{left} and f_{right}.

If $\bar{q}_{t+1}$ is derived by amending $q^*_{\text{overshoot}}$ and q_{target}, the below equation holds according to Remark 45.

$$\frac{\text{growth}_{\text{t}}}{\text{growth}_{\text{t+1}}} = \frac{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_{t+1}}(h))}{\int_{q_{\text{lower}}}^{q_{\text{target}}} e^{+\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))} = \frac{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_{t+1}}(h))}{\int_{q_{\text{target}}}^{q_{\text{upper}}} e^{-\frac{h}{2}} \mathsf{k}(\mathsf{w}_{\bar{q}_t}(h))} \leq 1, \tag{73}$$

and, therefore, the shrinkage of integrals caused by updates of `curve` sequence leads to growth.

[65] f_{right} goes from 1 to a value ≤ 1 over range $[q_{\text{target}}, q_{\text{upper}}]$. And, f_{left} goes from a value ≤ 1 to a value $\geq f_{\text{right}}$ over this range. Given the continuousness of these functions, they intersect somewhere in this range.

[66] f_{right} is a non-increasing function of $q_{\text{overshoot}}$ and f_{left} is a non-decreasing function of $q_{\text{overshoot}}$, over range $[q_{\text{target}}, q_{\text{upper}}]$.

Draft - NoFeeSwap

Remark 46. *There are typically two updates of the* `curve` *sequence after each swap, one to* `overshoot` *and one to* `target`*. However, if the overshoot and target match, there will be only one update. For example, they match when the swap hits one of the two boundaries of a liquidity interval.*

Please refer to the section `_overshoot_` in `Memory.sol` for further details.

3.6 Swap Flowcharts

There are three types of operations that can be executed as part of a swap. Figure 19 shows the sequence for any swap.

1. **SwapWithin.** Swaps inside a liquidity interval. The previous sections focused on the algorithms involved in the execution of this component.

   Figure 20 is a high-level demonstration of a `swapWithin` operation. Of the three steps in this flowchart, Section 3.4 discusses the first step. Section 3.5 discusses the calculation in the middle step. The last step is straightforward using the method discussed in Section 3.1.3.

2. **Cross a liquidity interval.** Cross an entire liquidity interval from one boundary to the next. This is carried out efficiently using the values `outgoingMax` and `incomingMax`.

3. **Transition to another liquidity interval.** Transition through a boundary from one liquidity interval to an adjacent one, which requires updating dynamic parameters (Section 3.2.2), shares/growth mappings (Section 3.3), and, if there is a pending `kernel`, static parameters (Section 3.2.3).

Draft - NoFeeSwap

3.7 Data Types

Solidity's custom types are widely used by the protocol. The types `X#` are created to specify values with # decimals.

- `X15`. (uint256) This type is dedicated to the liquidity distribution. The distribution of liquidity within the interval is governed by a piecewise-linear `kernel` function. The `kernel` function is characterized by a list of breakpoints inputted by the pool owner. The vertical coordinate of each breakpoint is a number within the range $[0, 1]$ which is stored as `X15`, with 2^{15} representing 1. See $\mathsf{k}(q)$ in Figure 15.

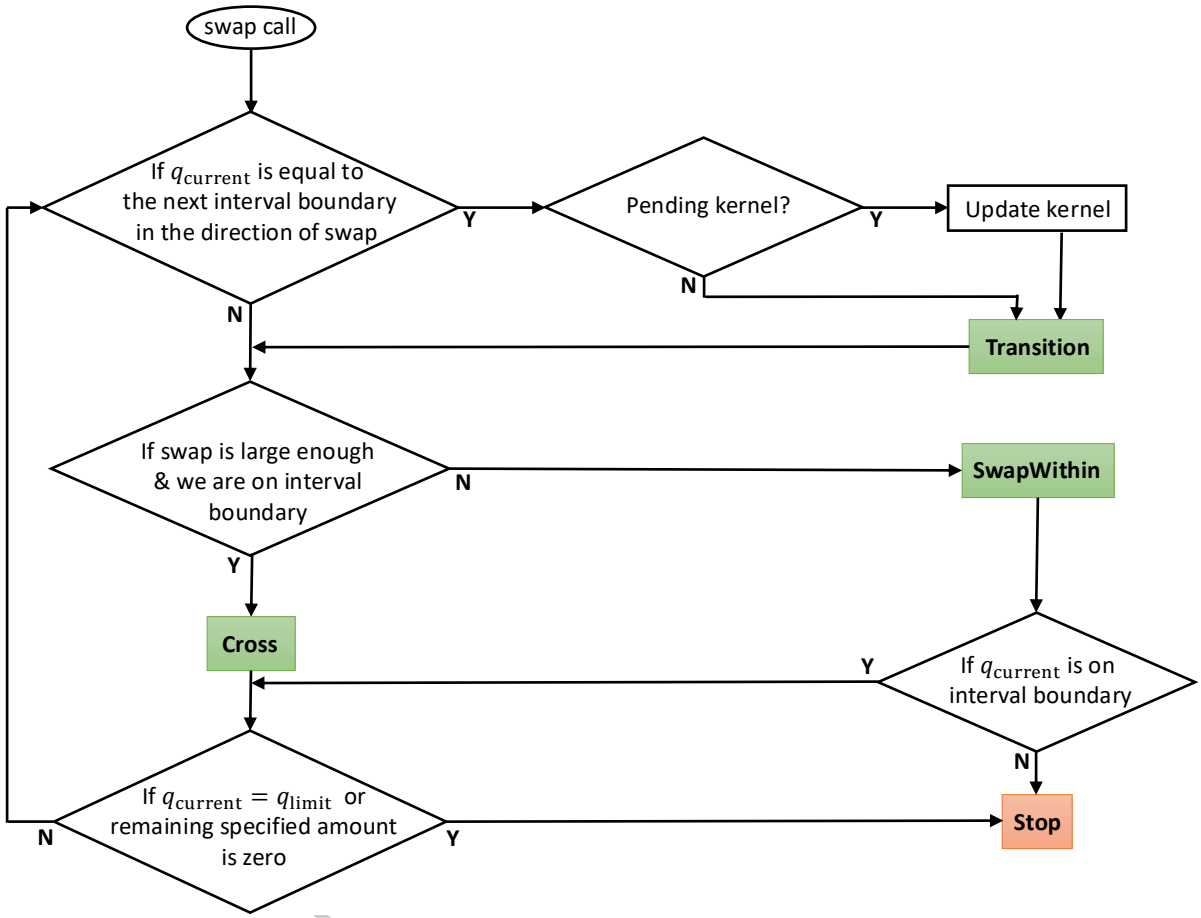


Figure 19: Swap flowchart determining how `swapWithin`, `transition` and `cross` operations are called.

Draft

- `X23`. (uint256) This type is dedicated to the growth ratio parameters between the protocol and the pool owner. As in Section 3.2.4, the pool ratio parameters are stored as `X23` with 2^{23} representing 100%.
- `X47`. (uint256) This type is dedicated to the growth portion parameters. As in Section 3.2.3, the growth portion parameters are stored as `X47` with 2^{47} representing 100%.
- `X59`. (int256) This type is dedicated to the natural logarithm of prices. There are two distinct representation formats:

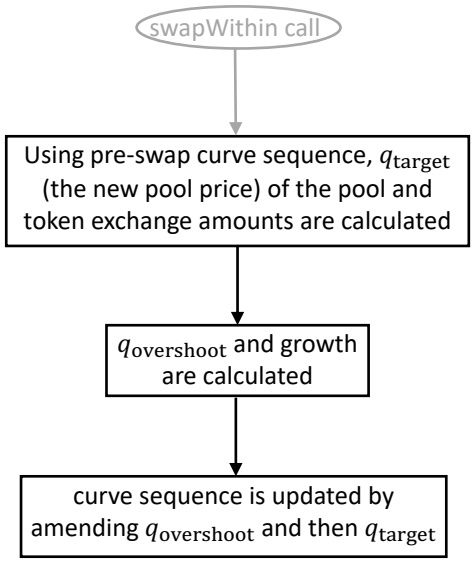


Figure 20: `swapWithin` flowchart.

Draft - NoFee.ap

- **Offset-binary representation (offsetted):**[67] It is used for members of `curve` sequence. The red terms in Table 1 cause the offset-binary format. As in Remark 7, the log-prices of a `curve` sequence are normalized to stay within ± 16. As in Table 1, q_{offset} is included to convert it to the real-world price.

  In this format, 0 refers to $q = -16$ and $p = e^{-16}$, and 2^{64} refers to $q = +16$ and $p = e^{+16}$.

  Offset-binary representation is selected for several reasons, including its properties due to symmetry over the range $q \in [-16, 16]$ and compatibility with exponential.

- **Two's complement representation (real-world):** For other log-prices, this representation is used.[68] See Table 2. Also, note that these are real-world values and not offsetted ones.

  $*$ *Real-world prices for the purpose of simplicity for the users.*

[67] "Offset-binary" refers to the method for signed number representation and "offsetted" refers to applying q_{offset}.

[68] There are various reasons, including simplicity and the fact that summing one offset-binary with two's complement numbers preserves the offset-binary representation.

- For `logPriceMin` and `logPriceMax` (in modifyPosition) and `logPriceLimit` (in swap), any signed value can be selected in a slot, i.e., $-2^{255} \leq h \leq 2^{255} - 1$.
- For the purpose of tag creation for LP tokens, 256-bit `X59` representation is used for specifying the position's boundaries. See tags for NoFeeSwap's LP tokens in Section 3.2.3.

∗ *Relative and non-negative measures where offset is not relevant.*

- q_{spacing} fits in 64 bits, as $0 < h < 2^{63}$. Note that $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} > 0$ and $q_{\text{spacing}} < 16\ (2^{63-59})$ according to Remark 55.
- For `kernel`, each q fits in 64 bits, as $0 \leq h < 2^{63}$. For every break-point of the kernel, $q \in [0, q_{\text{spacing}}]$. See q in Figure 15.
- q_{offset} is in `X59` representation in various calculations. However, the one included in `poolId` is not in `X59` representation.

Table 1: The relationship between the real-world prices and the corresponding offset-binary representation.

`X59` for `curve` sequence members	standardized log-price, within ± 16	real-world log-price, see (41b)	standardized price, within $e^{\pm 16}$	real-world price, see (41a)
$0 < h < 2^{64}$	$\frac{h-2^{63}}{2^{59}}$	$\frac{h-2^{63}}{2^{59}} + q_{\text{offset}}$	$e^{\frac{h-2^{63}}{2^{59}}}$	$e^{\frac{h-2^{63}}{2^{59}}+q_{\text{offset}}}$

Table 2: The relationship between the real-world prices and the corresponding two's complement representation.

`X59`	log-price	price	sqrt-price
h	$\frac{h}{2^{59}}$	$e^{\frac{h}{2^{59}}}$	$e^{\frac{h}{2^{60}}}$

Remark 47. q_{offset}*, in storage, is not in* `X59` *representation. It is maintained in* `int8` *representation in* `poolId`*, as in Section 3.2.1. In addition,* `sqrtOffset` *is maintained in static parameters in* `X127`*. However, they are converted to the* `X59` *representation in various internal calculations.*

Draft - NoFeeSwap

Remark 48. *The storage of the* `curve` *sequence using* `X59` *representation in offset-binary is innovative.*

- *Due to offset-binary representation, the exponential of each member of the* `curve` *sequence can be calculated efficiently, i.e., the corresponding two's complement representation would have required conditioning on the sign and extra addition and inversion operations, which increases the gas cost and decreases the accuracy.*
- *A benefit of* $e^{\frac{-2^{63}}{2^{59}}}$ *in Table 1, resulting from the offset-binary representation, is that for the sqrt-price it becomes a* e^{-8} *multiplier which is efficiently used for another purpose discussed in Remark 9.*
- *The offset-binary's addition to non-negative two's complement log-prices preserves offset-binary.*

Remark 49. *For exponential calculations using the Padé Approximant, usually the multipliers are too large for storage in EVM and a large number of terms are needed for sufficient accuracy. Here, the inputs are shrunk by a certain amount before the exponent calculation, to make sure that a small number of terms with parameters storable with only a few digits are sufficient. Then, after the exponent calculation, the output is squared a sufficient number of times to net the effect of shrinkage and provide the expected value with full precision.*

- `X74`. (int256) It is used as an intermediary type when searching for precise log-price movements corresponding to the specified outgoing/incoming token amounts.
- `X111`. (int256) It is dedicated to growth and liquidity values.
- `X127`. (int256) It is used for token amounts and `sqrtOffset`.
- `X208`. (uint256) It is used for growth multipliers.
- `X216`. (int256) It is dedicated to integrals and the sqrt-prices.

There are additional data types:

- Type `Index` (uint16) is used for enumerations, e.g., for members of the `curve` sequence and `kernel`.

Draft - NoFeeSwap

- Type `Tag` (uint256) is used for referring to items in Section 3.2.3's `tag` discussion.
- Types `Curve`/`Kernel`/`KernelCompact` (uint256) are memory pointers for handling `curve` sequence and `kernel`.
- There are two data types for price, one includes three members, `logPrice` +`sqrtPrice`+`sqrtInversePrice`, and the other one includes four members, `heightPrice`+`logPrice`+`sqrtPrice`+`sqrtInversePrice`.

3.8 External Functions

Here, a description of the external functions in the protocol and the corresponding inputs/outputs is provided.[69]

3.8.1 Nofeeswap Contract

This contract is the core protocol and contains all swap-related functions.

- `modifyBalance`. Mints/burns a singleton balance of `tag`. This results in a transient storage balance that must be settled. This is more gas efficient than external transfers from/to the sender's wallet.[70] See Figure 21.
  - `owner`: Balance owner.
  - `tag`: The tag whose balance is to be modified.
  - `amount`: The amount of tag to be added (positive) or removed (negative) to/from the owner's singleton balances.
- `modifyBalance`. Mints/burns singleton double balance of `tag0/1`. This results in transient storage balances that must be settled. See Figure 21. Can be used for gas-efficient transactions by eliminating the need to update two storage slots.[71]
  - `owner`: Balance owner.

[69] The requirement in Remark 51 is a safeguard against reentrancy.

[70] Frequent users benefit from lower gas costs by depositing a certain amount of `tag` to NoFeeSwap, for future activities, as opposed to using an external wallet for every transaction.

[71] A trader who exchanges or provides liquidity for pools of token pair X-Y frequently benefits from lower gas costs by depositing certain amounts of tokens X and Y as double balance for future activities.

Draft - NoFeeSwap

- `tag0/1`: Identifiers of the two NoFeeSwap tokens. The arithmetically smaller one is `tag0` and the other one is `tag1`.
- `amount0/1`: The amounts of each tag to be added (positive) or removed (negative) to/from the owner's singleton balances.

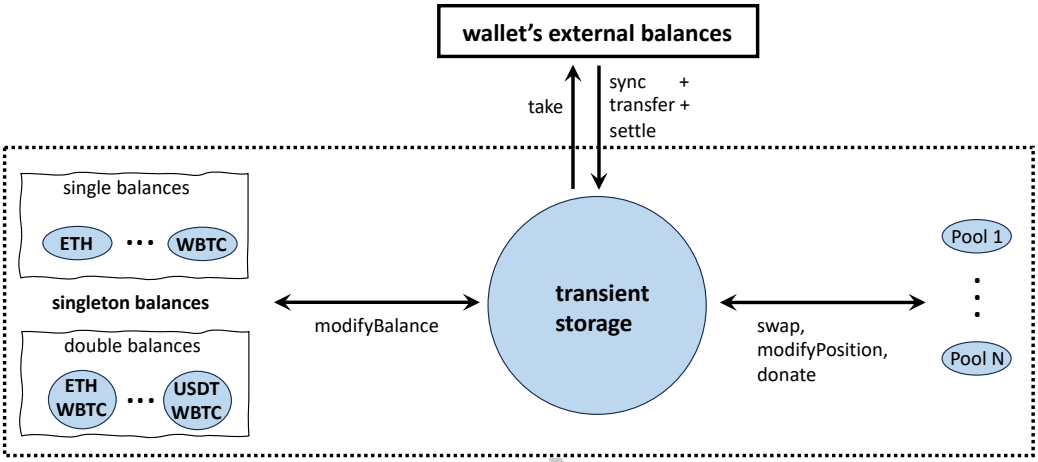


Figure 21: A visualization of the fund transfer structure.

Draft - Not

Remark 50. *In order to eliminate the need for expensive token and multi-token transfers, NoFeeSwap's core singleton offers an ERC-6909 compliant accounting structure. Users have the option to deposit their native, ERC-20, ERC-1155, and ERC-6909 tokens in return for the singleton's ERC-6909 balance. As explained in Section 3.2.3, the corresponding singleton multi-token identifiers are regarded as* `tag` *and are calculated as follows:*

$$\begin{aligned} \texttt{Native} &\rightarrow \texttt{tag} = \texttt{Tag.wrap}(0) \\ \texttt{ERC}-20 &\rightarrow \texttt{tag} = \texttt{Tag.wrap}(\texttt{uint256}(\texttt{uint160}(\\ &\qquad \texttt{underlyingERC20Address}))) \end{aligned} \tag{74}$$

$$\begin{aligned} \texttt{ERC}-1155 &\rightarrow \texttt{tag} = \texttt{Tag.wrap}(\texttt{uint256}(\texttt{keccak256}(\\ &\qquad \texttt{encodePacked}(\\ &\qquad\quad \texttt{underlyingERC1155Address}, \\ &\qquad\quad \texttt{underlyingERC1155TokenId})))) \end{aligned} \tag{75}$$

$$\texttt{ERC} - \texttt{6909} \rightarrow \texttt{tag} = \texttt{Tag.wrap}(\texttt{uint256}(\texttt{keccak256}(\texttt{encodePacked}(\texttt{underlyingERC6909Address}, \texttt{underlyingERC6909TokenId})))) \tag{76}$$

$$\texttt{LP} - \texttt{Position} \rightarrow \texttt{tag} = \texttt{Tag.wrap}(\texttt{uint256}(\texttt{keccak256}(\texttt{encodePacked}(\texttt{qMin}, \texttt{qMax}, \texttt{poolId})))) \tag{77}$$

The ownership of these `tags` *is accounted for via the singleton's internal ERC-*6909 *mechanism.*

Note that one may also use the NoFeeSwap singleton address and any of the aforementioned tags as a pair of `underlyingERC6909Address` *and* `underlyingERC6909TokenId` *in equation* (76)*, in order to derive a different* `Tag` *for the same asset. However, users are discouraged from doing this in order to avoid unnecessary fragmentation.*

*In addition to the standard ERC-*6909 *interface, NoFeeSwap offers a double-balance feature which allows users to store the balances associated with any pair of tags in a single storage slot.*

- `unlock`. This function gives access to NoFeeSwap functionalities that are safeguarded against reentrancy (the calls that require a change in transient balances).
  - `unlockTarget`: The target contract address which must implement `IUnlockCallback`, i.e., the callback address. See Remark 51.
  - `data`: The data/instructions to be passed to the target contract.
  - [output] `result`: The output from the target contract.

Remark 51. `unlockCallback`*: In order to interact with any of the following functions of* `Nofeeswap` *and* `NofeeswapDelegatee`*, an account must first invoke* `INofeeswap.unlock(unlockTarget,data)` *to unlock the protocol.*

Draft – NoFeeSwap

- `INofeeswap.clear(Tag,uint256)`
- `INofeeswap.take(address,address,uint256)`
- `INofeeswap.take(address,uint256,address,uint256)`
- `INofeeswap.take(address,uint256,address,uint256,bytes)`
- `INofeeswap.settle(address)`
- `INofeeswap.transferTransientBalanceFrom(address,address,Tag,uint256)`
- `INofeeswap.modifyBalance(address,Tag,int256)`
- `INofeeswap.modifyBalance(address,Tag,Tag,int256,int256)`
- `INofeeswap.swap(uint256,int256,X59,uint256,bytes)`
- `INofeeswapDelegatee.modifyPosition(uint256,X59,X59,int256,bytes)`
- `INofeeswapDelegatee.donate(uint256,uint256,bytes)`

The function `INofeeswap.unlock` *then invokes*

`IUnlockCallback(unlockTarget).unlockCallback(caller,data)` *through which any account may interact with the above methods.*

The default target which implements `unlockCallback` *is* `Operator.sol`*, for which a sequence of instructions should be encoded as* `data`*. See Section 5.*

All transient balances must be settled by the `unlockCallback` *function to prevent the unlock call from reverting. In other words, all of the balances in transient storage for all accounts should net to zero for this call to go through.*

- `caller`*: The* `caller` *of the protocol's* `unlock` *method.*
- `data`*: The input data that was passed to the* `unlock` *method.*
- *[output]* `returnData`*: The output that should be returned to the* `caller`*.*

- `clear`. Clears positive transient balance.

Draft - NoFeeSwap

- `tag`: The tag whose transient balance is to be cleared.
- `amount`: The amount to be cleared, which must be equal to the current transient balance of the caller.

Remark 52. *Some notes before discussing the next functions:*

- `take` *function is used for withdrawals from the transient storage to an external wallet. (There is no need for using a* `transfer` *function with it as it is implicitly done in it.)*
- `sync`/`transfer`/`settle` *functions are used (in order) for deposits to the transient storage from an external wallet.*
- `transfer` *function for NoFeeSwap LP tokens is included here. For other tokens, this function from its corresponding contract is needed.*

- `take`. Pays native or ERC-20 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). It can be used for flash loans as well. See Figure 21.
  - `token`: ERC-20 or native tokens to be taken from the protocol's wallet. `address(0)` refers to the native token.
  - `to`: The target address.
  - `amount`: The token amount to be taken from the protocol.
- `take`. Pays ERC-6909 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). See Figure 21.
  - `token`: ERC-6909 address.
  - `tokenId`: Multi-token id to be taken from the protocol's wallet.
  - `to`: The target address.
  - `amount`: The token amount to be taken from the protocol.
- `take`. Pays ERC-1155 assets from NoFeeSwap to `to` (transient storage balance to a target wallet address). See Figure 21.
  - `token`: ERC-1155 address.
  - `tokenId`: Multi-token id to be taken from the protocol's wallet.
  - `to`: The target address.

Draft - NoFeeSwap

  - `amount`: The token amount to be taken from the protocol.
  - `transferData`: Data to be used from `IERC1155.safeTransferFrom`.

- `sync`. Synchronizes NoFeeSwap ERC-20. Should be called prior to ERC-20 transfers to NoFeeSwap.
  - `token`: ERC-20 tokens to be synced.

- `sync`. Synchronizes NoFeeSwap ERC-1155 or ERC-6909 balance. Should be called prior to ERC-1155 or ERC-6909 transfers to NoFeeSwap.
  - `token`: ERC-1155 or ERC-6909 address to be synced.
  - `tokenId`: Multi-token id to be synced.

- `settle`. Should be called after native, ERC-20, ERC-1155 or ERC-6909 transfers to NoFeeSwap to update transient balances. Can also be used to send native tokens to the protocol to settle native transient balance of `msg.sender`.
  - [output] `paid`: The amount paid.

- `transferTransientBalanceFrom`. Transfers transient balance from `sender` to `receiver`. Can also be used to settle on behalf of other accounts.
  - `sender`: The sender's address.
  - `receiver`: The recipient's address.
  - `tag`: The tag whose balance is to be transferred.
  - `amount`: The amount to be transferred.

- `dispatch`. Executes delegate calls to the `NofeeswapDelegatee` contract. See Section 3.8.2.
  - `input`: Encoded input via `abi.encodeWithSelector` to be passed as a delegate call to `NofeeswapDelegatee` contract.
  - [output] `output0/1`: Two output slots.

- `swap`. Performs a swap. See Section 3.4 for the explanations of the input parameters.

Draft - NoFeeSwap

  - `poolId`: The identifier of the target pool. See Section 3.2.1.
  - `amountSpecified`: The amount to be given to (positive) or taken from (negative) the pool.
  - `logPriceLimit`: See Section 3.4 and Table 2 for explanations.
  - `zeroForOne`: See Section 3.4 for explanations.
  - `hookData`: The data to be passed to the hook.
  - [output] `amount0/1`: The amount of `tag0/1` swapped. Positive amounts are incoming to the protocol, and negative amounts are outgoing from the protocol.

- `storageAccess`. Provides access to NoFeeSwap storage (one slot)
  - `slot`: The slot to be read.
  - [output] Data read from storage.

- `storageAccess`. Provides access to NoFeeSwap storage (a consecutive sequence of slots)
  - `startSlot`: The starting slot to be read.
  - `nSlots`: Number of slots to be read.
  - [output] Data read from storage.

- `storageAccess`. Provides access to NoFeeSwap storage (a sparse sequence of slots)
  - `slots`: An array of storage slots to be read.
  - [output] Data read from storage.

- `transientAccess`. Provides access to NoFeeSwap transient storage (one slot)
  - `slot`: The slot to be read.
  - [output] Data read from transient storage.

- `transientAccess`. Provides access to NoFeeSwap transient storage (a consecutive sequence of slots)
  - `startSlot`: The starting slot to be read.

Draft - NoFeeSwap

- `nSlots`: Number of slots to be read.
- [output] Data read from transient storage.

- `transientAccess`. Provides access to NoFeeSwap transient storage (a sparse sequence of slots)
  - `slots`: An array of transient storage slots to be read.
  - [output] Data read from transient storage.

See ERC-6909 specifications for the description of the remaining functions from the ERC-6909 standard, i.e., `supportsInterface`, `balanceOf`, `allowance`, `isOperator`, `transfer`, `transferFrom`, `approve`, `setOperator`.

Draft - NoFeeSwap

3.8.2 NofeeswapDelegatee Contract

This contract is the second part of the protocol's core, and most operations except for swap are included here. Every functionality of this contract is executed through the `dispatch` function of `Nofeeswap` contract, as mentioned in Section 3.8.1.

- `initialize`. Initializes a new NoFeeSwap pool.
  - `unsaltedPoolId`: The unique pool identifier to be initialized. It encapsulates the hook address, the flags, and the price offset, as explained in Section 3.2.1. Please see Section 3.2.1.2 for explanations on `unsaltedPoolId`.
  - `tag0/1`: The two tags for which this pool is created. The arithmetically smaller one is `tag0` and the other one is `tag1`.

    Example 5. *Assume that it is intended to create a new pool for ETH-WBTC. In this case,* `tag0` *must be* 0 *and* `tag1` *must be equal to WBTC's address with additional* 96 *zeros as the most significant bits, i.e.,*

    `0x000000000000000000000000002260fac5e5542a773aa44fbcfedf7c193bc2c599`

  - `poolGrowthPortion`: The initial value for the pool growth portion in the `X47` representation. `zeroX47` and `oneX47` refer to 0% and 100%, respectively. This value dictates the portion of the growth that goes to the pool owner followed by the protocol.

– `kernelCompactArray`: An array of type `uint256[]` whose elements consist of `kernel` break-points, 80 bits each. They are packed in 256-bit slots. See Section 3.1.1, Table 2, and Remarks 36 and 37.

Example 6. *If* $q_{\text{spacing}} = 2^{72}$*, the kernel can be as in Table 3, which is depicted in Figure 22. Note that the first* 0 *entry is always omitted, as mentioned in Section 3.2.3. According to Table 2, the corresponding value of* $q_{\text{spacing}} = 2$ *in* `X59` *is* 2^{60}.

Table 3: The kernel break-points of Example 6.

Kernel Break-point q	q in X59	Corresponding height, $k(q)$	$k(q)$ in X15
0	0	0	0
0.5	2^{58}	0.125	2^{12}
1	2^{59}	0.5	2^{14}
1	2^{59}	1	2^{15}
2	2^{60}	1	2^{15}

LoFeeSwap

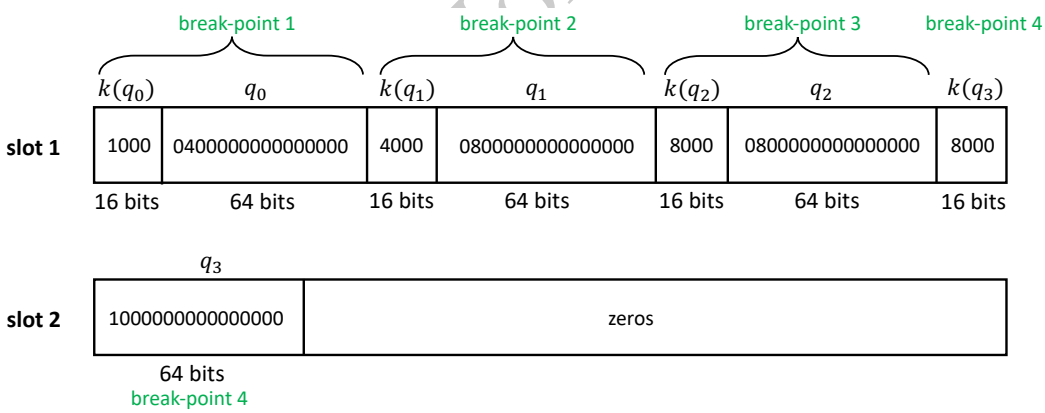


Figure 22: `compactKernel` of Example 6.

– `curveArray`: The initial curve sequence array. It is an array of type `uint256[]` whose each element can contain four members of the initial `curve` sequence. The current price of the pool is the last member of the `curve` sequence. q_{spacing} can be determined from the first

[72] i.e., $q_{\text{upper}} - q_{\text{lower}} = 2$ and $\frac{p_{\text{upper}}}{p_{\text{lower}}} = e^2$.

two members of the `curve` sequence, as $q_{\text{spacing}} = q_{\text{upper}} - q_{\text{lower}} = |q^0 - q^1|$. See Sections 3.1.2 and 3.1.3 for details on the `curve` sequence, and Table 1 for the `X59` representation of it.

Remark 53. *For a valid initialization, the q_{spacing} implied from* `curveArray` *must match the one implied from* `kernelCompactArray`*.*

– `hookData`: Data to be passed to the initialize hooks.

Remark 54. *The initial* `curve` *sequence is supplied by the pool creator and determines q_{spacing} which should not be less than 2^{40} in the* `X59` *representation.*[73]

Remark 55. *When a* `curve` *sequence is used to initialize a pool, $-16 < q^0, q^1 < 16$ holds as in Remark 7 and Table 1. The rightmost and leftmost liquidity intervals of this entire horizon are regarded as blank liquidity intervals and should be avoided, i.e., there should be at least 3 intervals. Therefore, $q_{\text{spacing}} = |q^0 - q^1| < \frac{32}{3}$.*

Remark 56. *One candidate for the initial* `curve` *sequence (consistent with Remark 13) is q^0, q^1, q^2, where:*

– **Pool price:** *q^2 is the market log-price at the time of initialization.*

– **q_{spacing} selection (permanent impactful influence):** *$|q^1 - q^0|$, which determines q_{spacing}, is selected based on the economic characteristics of the tokens, such as price volatility, as well as the view and strategy of the creator of the pool.*

– **Curve shape (temporary influence):**

  ∗ *Select $q^0 < q^2 < q^1$ if the historical price has reached q^2 from the above*

  ∗ *Select $q^1 < q^2 < q^0$ if the historical price has reached q^2 from below*

Remark 57. *The selection of q_{spacing} is generally more material than the initial* `curve` *sequence because the initial* `curve` *sequence is not permanent. However,*

[73] The corresponding log-price value is $q_{\text{spacing}} = \frac{2^{40}}{2^{59}}$. The corresponding price value is $p_{\text{spacing}} = \exp\left(\frac{2^{40}}{2^{59}}\right) \approx 1.0000019$, which is equal to $\frac{p_{\text{upper}}}{p_{\text{lower}}}$ for each liquidity interval.

Draft - NoFeeSwap

- *For pools that q_{spacing} is relatively small and it is expected that the liquidity intervals are transitioned frequently, some $q^0, q^1, q^2 (= q_{\text{current}})$ can be simply selected as the* `curve` *sequence, at initialization, where $|q^0 - q^1|$ is selected equal to the desired q_{spacing}. The* `curve` *sequence is expected to reset shortly after an interval transition.*
- *For pools that q_{spacing} is relatively large, the choice of initial* `curve` *sequence is more material, as the* `curve` *sequence resets infrequently and the implications of its initial value last longer. For example, if $q_{\text{spacing}} = 5$, assuming that the historical market log-prices over the last few weeks peaked at 7 (up), 2 (down), and 4 (up), in chronological order, with the current log-price of 3, the* `curve` *sequence can be chosen as $7, 2, 4, 3$.*

Example 7. *Assume that it is intended to create a new pool for ETH-WBTC, when the market price of WBTC is exactly 20 times that of ETH. Note that WBTC has 8 decimals and ETH has 18 decimals. Hence, the smallest unit of WBTC is 10^{-8} and the smallest unit for ETH is 10^{-18}. Therefore, the price in the protocol should be around:*

$$\frac{\text{price of each unit of WBTC (tag1)}}{\text{price of each unit of ETH (tag0)}} = 20 \times 10^{18-8} = 2 \times 10^{11}. \quad (78)$$

If the pool creator intends to make p_{offset} around 2×10^{11}, then the corresponding `int8` *representation of* `logOffset` *can be calculated using SymPy with command* `floor(log(2*(10**11)))` *which is equal to 26, i.e., $q_{\text{offset}} = 26$ and $p_{\text{offset}} = e^{26}$.*[74]

Therefore, $q_{\text{current}} = \log_e (2 \times 10^{11}) - q_{\text{offset}} \sim 0.02$, with `X59` *offset-binary representation of (according to Table 1):*

```
hex(floor((2**63)+(2**59)*(log(2*(10**11))-26))),
```

which is equal to `0x802c33d08936686c`*. This is selected as the last member of the* `curve` *sequence.*

According to Remark 7, price of the real-world pool can move to any value within $e^{-16} p_{\text{offset}}$ and $e^{+16} p_{\text{offset}}$, where in this example, p_{offset} is a number close to the market price at initialization.

[74] See Remark 47.

Draft - NoFeeSwap

Example 8. *Each member of the* `curve` *sequence is a* 64*-bit representing a log-price value in offset-binary* `X59` *representation. Four members are packed in each slot of the input array. In Example 7, if there are* 5 *members,*

- $q^0 = q_{\text{current}} + 1 \implies$ `0x882c33d08936686c` $\implies p = e^{+1} p_{\text{current}}$
- $q^1 = q_{\text{current}} - 1 \implies$ `0x782c33d08936686c` $\implies p = e^{-1} p_{\text{current}}$
- $q^2 = q_{\text{current}} + \frac{1}{2} \implies$ `0x842c33d08936686c` $\implies p = e^{+\frac{1}{2}} p_{\text{current}}$
- $q^3 = q_{\text{current}} - \frac{1}{2} \implies$ `0x7c2c33d08936686c` $\implies p = e^{-\frac{1}{2}} p_{\text{current}}$
- $q^4 = q_{\text{current}} \implies$ `0x802c33d08936686c` $\implies p = p_{\text{current}}$

These can be derived using price relationships in Table 1. Note that this example also has an offset multiplier of $p_{\text{offset}} = e^{26}$*. The corresponding slots of this* `curve` *sequence are illustrated in Figure 23.*

Observe that $q_{\text{spacing}} = |q_{\text{upper}} - q_{\text{lower}}| = |q^0 - q^1| = 2$*, and, the kernel's last log-price must be consistent.*

FeeSwap

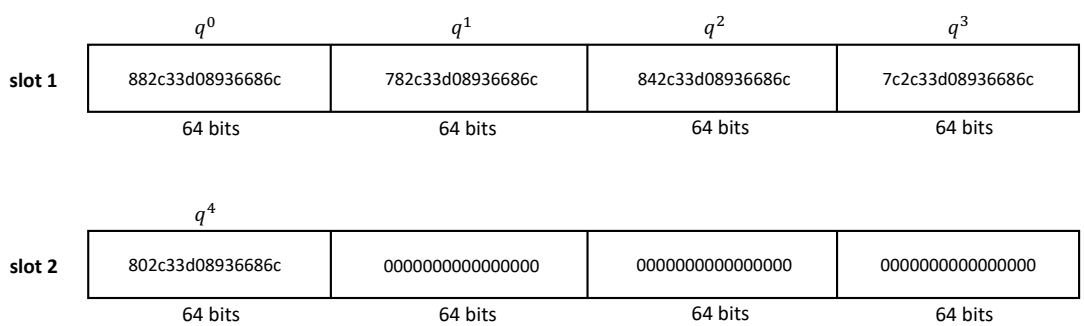


Figure 23: The `curve` sequence of Example 8 in initialization.

Example 9. q_{spacing} *in Example 8 is too large for most practical cases. Its value should be derived based on the historical fluctuations of that token pair's market price, and the pool creator's preference on how often it is desired to have transitions between liquidity intervals and reset* `curve` *sequence.*

Assume a case quite similar to Example 8, but where the pool creator intends to make $\frac{p_{\text{upper}}}{p_{\text{lower}}} \sim 1.004$*, which means that the price moves from*

an interval boundary to the other one lead to 0.4% *price change. Then,* $q_{\text{spacing}} = |q^0 - q^1| = 2^{-8}$ *can be selected*[75]*, and therefore,* $2^{-8+59} = 2^{51}$ *in* `X59` *representation, as in Table 2. If there are* 5 *members in* `curve` *sequence as before,*

- $q^0 = q_{\text{current}} + 2^{-9} \implies p = e^{+2^{-9}} p_{\text{current}}$
- $q^1 = q_{\text{current}} - 2^{-9} \implies p = e^{-2^{-9}} p_{\text{current}}$
- $q^2 = q_{\text{current}} + 2^{-10} \implies p = e^{+2^{-10}} p_{\text{current}}$
- $q^3 = q_{\text{current}} - 2^{-10} \implies p = e^{-2^{-10}} p_{\text{current}}$
- $q^4 = q_{\text{current}} \implies p = p_{\text{current}}$

The price ratio of the two boundaries of a liquidity interval is $e^{(2^{-8})} \sim 1.004$*, leading to* $\sim 0.4\%$ *price difference between the two.*

- `modifyPosition`. Mints/burns within a concentrated liquidity interval.
  - `poolId`: Identifier of the target pool.
  - `logPriceMin`: The left (smaller) position boundary which must be equal to the boundaries of the active liquidity interval modulo q_{spacing}.
  - `logPriceMax`: The right (larger) position boundary which must be equal to the boundaries of the active liquidity interval modulo q_{spacing}.
  - `shares`: The number of shares to be minted (positive)/burned (negative) per interval. The call applies `shares` count to **each** of $k = \frac{q_{\max} - q_{\min}}{q_{\text{spacing}}}$ intervals.
  - `hookData`: Data to be passed to hooks.
  - [output] `amount0/1`: The amount of `tag0/1`. Positive means they are deposited into the pool and negative means they are withdrawn from the pool.

Remark 58. *To create a new position, the selected left and right log-price boundaries,* $q_{\min}$ *and* $q_{\max}$*, must be in the form of* $q_{\text{lower}} + kq_{\text{spacing}}$*, each for an integer* k*. Additionally,* $-16 + q_{\text{spacing}} < q_{\min} < q_{\max} < 16 - q_{\text{spacing}}$ *must hold, since the rightmost and leftmost liquidity intervals of this entire horizon are regarded as blank liquidity intervals and should be avoided, as in Remark 55.*

[75] since $2^{-8} \sim \log_e(1.004)$

Draft - NoFeeSwap

Tables 1 and 2 show the differences of different representations. Table 1 shows how to convert offsetted offset-binary representation of `curve` *sequence log-prices to the real-world log-prices.*

Remark 59. *If the first two members of* `curve` *sequence are q^0 and q^1, then, $q_{\text{lower}} = \min\{q^0, q^1\}$, $q_{\text{upper}} = \max\{q^0, q^1\}$ and $q_{\text{spacing}} = |q^0 - q^1|$. The first two members of the curve sequence can be retrieved for this purpose from functions like* `readBoundariesExternally` *in* `storage.sol`.

Remark 60. *The incoming and outgoing token amounts due to* `modifyPosition` *in any arbitrary range can be calculated as follows. Note that q_{lower} and q_{upper} represent the boundaries of the active liquidity interval.*

- *Assuming $q_{\text{upper}} \leq q_{\text{min}}$, for a single share in every interval from q_{min} to q_{max}, the corresponding amount of* `token0` *is equal to:*[76]

$$\texttt{amount0} = \frac{1}{\sqrt{p_{\text{offset}}}}(\texttt{growthMultiplier}[q_{\text{min}}] - \texttt{growthMultiplier}[q_{\text{max}}]).$$

- *Similarly, assuming $q_{\text{max}} \leq q_{\text{lower}}$, the amount of* `token1` *corresponding to a single share in every interval from q_{min} to q_{max} is equal to:*[77]

$$\texttt{amount1} = \sqrt{p_{\text{offset}}}(\texttt{growthMultiplier}[q_{\text{max}}] - \texttt{growthMultiplier}[q_{\text{min}}]).$$

- *Lastly, the token amount in the active liquidity interval corresponding to any specific number of shares is derived through equation* (57).

- `donate`. Donates the token amounts equivalent to a number of shares to be distributed proportionally among all the LPs in the current active liquidity interval. See Remark 21.
  - `poolId`: Target pool's identifier.
  - `shares`: The number of shares to be donated.
  - `hookData`: Data to be passed to donate hooks.

[76] See equation (64).
[77] See equation (65).

Draft - NoFeeSwap

  - [output] `amount0/1`: These are the donated amounts of `tag0/1`.

- `modifyProtocol`. Sets `maxpoolGrowthPortion`, `protocolGrowthPortion`, and the address of the protocol owner which are all in one slot. See Section 3.2.5. Must be called by the current protocol owner, which is `timelock` contract, i.e., through the protocol's governance.

  - `protocol`: The new value for the protocol slot.

- `modifySentinel`. Sets `Sentinel` contract. Must be called by the current protocol owner. See Section 4.2.

  - `sentinel`: New sentinel contract.

- `modifyPoolOwner`. Sets a new pool owner for the given `poolId`. It must be called by the current pool owner.

  - `poolId`: The identifier of the target pool.
  - `newOwner`: The new owner's address.

- `modifyKernel`. Provides a new pending kernel for a given pool, to be activated when the next liquidity interval transition happens. If there is already a pending kernel in that pool, it replaces it. It must be called by the pool owner.

  - `poolId`: The identifier of the target pool.
  - `kernelCompactArray`: The new kernel array in its compact form.
  - `hookData`: Data to be passed to the hook.

- `modifyPoolGrowthPortion`. Changes `poolGrowthPortion` to a proposed amount. This is governed by the `Sentinel` contract, as in Section 4.2.1, i.e., the protocol can receive some tokens to permit increasing this. It must be called by the pool owner.[78]

  - `poolId`: The identifier of the target pool.
  - `poolGrowthPortion`: The new pool growth portion.

[78] Note that the effect of `poolGrowthPortion` is capped by `maxPoolGrowthPortion`.

Draft - NoFeeSwap

- `updateGrowthPortions`. Syncs `protocolGrowthPortion` and `maxPoolGrowthPortion` for a pool to the latest values set by the `Sentinel` contract or protocol owner. It can be called by any address.
  - `poolId`: The identifier of the target pool.
- `collectPool`. Collects pool owner's accrued growth portions. The collected amounts are transferred to the pool owner's singleton single balances. It can be called by any address.
  - `poolId`: The target pool's identifier.
  - [output] `amount0/1`: The total amount of `tag0/1` collected.
- `collectProtocol`. Collects protocol owner's accrued growth portions. The collected amounts are transferred to the protocol owner's singleton single balances. It can be called by any address.
  - `poolId`: The target pool's identifier.
  - [output] `amount0/1`: The total amount of `tag0/1` collected.

4 Governance

4.1 Parameters

The standard `GovernorBravo` contract of Compound [8] is used for governance, with the following parameters:

- `Nofee` token contract:
  - **Constants:**
    - `name`: Nofeeswap
    - `symbol`: NOFEE
    - `decimals`: 18
    - `minimumTimeBetweenMints`: 1 year[79]
    - `mintCap`: 2%
  - **Mutable:**

[79] `minter` is `Timelock` contract. `mintingAllowedAfter` is set to 4 years after the deployment.

Draft - NoFeeSwap

    * `totalSupply`: 10 billion NOFEE

- `GovernorBravo` is upgradable by governance:[80]
  - **Constants:**
    * `MIN_PROPOSAL_THRESHOLD`: 0.01% of NOFEE
    * `MAX_PROPOSAL_THRESHOLD`: 1% of NOFEE
    * `MIN_VOTING_PERIOD`: ∼1 day
    * `MAX_VOTING_PERIOD`: ∼2 weeks
    * `MIN_VOTING_DELAY`: ∼15 seconds
    * `MAX_VOTING_DELAY`: ∼1 week
    * `quorumVotes`: 5% of NOFEE
    * `proposalMaxOperations`: 10
  - **Mutable:**
    * `votingPeriod`: 1 week
    * `votingDelay`: 2 days[81]
    * `proposalThreshold`: 0.1% of NOFEE
    * `admin`: `Timelock`
- `Timelock` contract is upgradable by governance:
  - **Constants:**
    * `GRACE_PERIOD`: 14 days[82]
    * `MINIMUM_DELAY`: 2 days
    * `MAXIMUM_DELAY`: 30 days
  - **Mutable:**
    * `delay`: 2 days[83]
    * `admin`: `GovernorBravo`[84]

[80] It contains the main parameters and functionalities of the governance.

[81] It is the period between a proposal and the automatic start of voting.

[82] The maximum permitted period between a voting termination and the execution is `delay+GRACE_PERIOD`.

[83] It is the minimum permitted period between a voting termination and the execution.

[84] The admin can run `queueTransaction` (when proposal comes), `cancelTransaction` (when voting rejects) and `executeTransaction` (when voting approves).

Draft - NoFeeSwap

4.2 `Sentinel` Contract

`Sentinel` contract has two components that are explained in the following. It is upgradable by governance.

4.2.1 Manage Pool Initialization & PoolGrowthPortion Parameter Change

`Sentinel` contract can manage pool creation and modifying poolGrowthPortion parameter. Currently:

- It requires non-refundable 0 NOFEE tokens for the protocol to permit a pool creation;
- It requires non-refundable 1000 NOFEE tokens for the protocol to permit an *increase* of a pool growth portion parameter.[85]

Remark 61. *The collected tokens from the above operations remain with* `Timelock` *and can be spent by governance voting.*

Remark 62. *As in Section 6.1.5.1, the creators of pools with an incentive hook do not receive a portion of the pools' growth. Also, the pools initiated by the* `incentivePoolFactory` *are exempt from the costs mentioned in this section.*

4.2.2 Customize ProtocolGrowthPortion & MaxPoolGrowthPortion Parameters

`Sentinel` contract is used for maintaining customized max pool growth portion and protocol growth portion parameters. As mentioned in Section 3.2.5, the `Sentinel` contract overwrites the default values of these parameters from the protocol slot.

Initially, the `Sentinel` contract maintains the maxPoolGrowthPortion parameter of 50% for certain pools with stable prices[86] and 20% for all other pools. In addition, it maintains the default protocolGrowthPortion parameter of 0. Both of these parameters are modifiable by governance at pair-level.[87]

[85] If a pool initialization is with a non-zero pool growth portion parameter, this additional cost is applied.

[86] initially, for several major pools with stable prices

[87] These are global records that are maintained in the storage. These parameters can be modified for each pair by governance. Local versions of such parameters for each pool are maintained locally and are updatable to global versions by a call from any address. See `updateGrowthPortions` function from `NofeeswapDelegatee`.

Draft - NoFeeSwap

An upgrade of the `Sentinel` contract by governance can allow modifying these growth portion parameters for any pool, as opposed to the current token-pair level.

4.2.3 External Functions

Part 1: Below are the external functions that are called by the protocol and are available at the `Sentinel` contract interface:

- `getGrowthPortions`. Gives the values of `maxPoolGrowthPortion` and `protocolGrowthPortion` given a snapshot of memory when called by the protocol.[88]
  - `sentinelInput`: Data passed to sentinel contract (a snapshot of memory).
  - [output] `maxPoolGrowthPortion`: The value for `maxPoolGrowthPortion`. If it returns any value greater than `oneX47`, then the protocol slot's default values are used.
  - [output] `protocolGrowthPortion`: The value for `protocolGrowthPortion`. If it returns any value greater than `oneX47`, then the protocol slot's default values are used.
- `authorizeInitialization`. Called by the protocol to obtain authorization for a pool initialization.
  - `sentinelInput`: Data passed to sentinel contract (a snapshot of memory).
- `authorizeModificationOfPoolGrowthPortion`. Called by the protocol to get authorization for modifying a pool growth portion parameter.
  - `sentinelInput`: Data passed to sentinel contract (a snapshot of memory).

Part 2: Below are the other external functions.

- `setAdmin`. Sets a new admin for this contract. It should be called by the admin.

[88]The current implementation of this function does not use `poolId`. The current `Sentinel` contract has the same growth portions for all pools of the same tag pair.

Draft - NoFeeSwap

- `admin`: Address of the new admin.

- `setGrowthPortions`. Sets new values for `maxPoolGrowthPortion` and `protocolGrowthPortion` for a given tag pair. It should be called by the admin.
  - `tag0/1`: Each is an array of tags, for any number of tag pairs (arithmetically smaller and larger ones, respectively).
  - `maxPoolGrowthPortion`: Given values for `maxPoolGrowthPortion`.
  - `protocolGrowthPortion`: Given values for `protocolGrowthPortion`.

- `setInitializationCost`. Sets a new value for the number of tokens to be claimed to authorize a pool initialization. It should be called by the admin. It is currently 0 NOFEE.
  - `initializationCost`: The new value for initialization cost.

- `setGrowthPortionCost`. Sets a new value for the number of tokens to be claimed to authorize increasing a pool growth portion or setting an initial non-zero amount for it. It should be called by the admin. It is currently 1000 NOFEE.
  - `growthPortionCost`: The new cost value.

Remark 63. *The following public parameters are in* `Sentinel` *contract as well.*

- *The address of* `Nofeeswap` *contract which is called to authorize functions.*

- *The address of* `IncentivePoolFactory` *contract which is exempt from the costs of pool creation and modifying the pool growth portion parameter. See Remark 62.*

- `token`*: NOFEE address, which is the ERC-*20 *token to be claimed for authorizations.*

- `admin`*: The address of the contract admin.*

- `initializationCost`. *The amount of tokens to be claimed to authorize pool initialization.*

Draft - NoFeeSwap

- `growthPortionCost`*: The amount of tokens to be claimed to authorize increasing a pool growth portion or setting an initial non-zero amount for it.*

- `growthPortions`*: A mapping of each token pair to maxPoolGrowthPortion and protocolGrowthPortion.*

4.3 NOFEE Token

NOFEE is the governance token. Section 4.1 provides information on this ERC-20 token's contract parameters. The distribution is as follows:

- 40% of the NOFEE tokens are reserved for the team, developers, investors, and future employees, with a commitment, for the foreseeable future, to remain hands-off, without any voting participation, to avoid disrupting community governance and to prevent token conversion to xNOFEE, ensuring that the community receives all distributed rewards.

- 40% is at the protocol's treasury with a linear vesting schedule of 4 years. These tokens are accessible for community governance.

  - First year:
    - Vesting amount: 1.6 billion NOFEE
    - Vesting begin: @ date T
    - Vesting cliff: @ date T
    - Vesting end: @ date $T + 1$ year
  - Second year:
    - Vesting amount: 1.2 billion NOFEE
    - Vesting begin: @ date $T + 1$ year
    - Vesting cliff: @ date $T + 1$ year
    - Vesting end: @ date $T + 2$ year
  - Third year:
    - Vesting amount: 0.8 billion NOFEE
    - Vesting begin: @ date $T + 2$ year
    - Vesting cliff: @ date $T + 2$ year
    - Vesting end: @ date $T + 3$ year

Draft - NoFeeSwap

- Fourth year:
  - Vesting amount: 0.4 billion NOFEE
  - Vesting begin: @ date $T + 3$ year
  - Vesting cliff: @ date $T + 3$ year
  - Vesting end: @ date $T + 4$ year

- 20% is reserved for the incentive program and is distributed over 5 years. See Figure 24.

4.4 xNOFEE Token

A core idea behind the xNOFEE token is quite similar to SushiBar in SushiSwap [9] in the sense that the xNOFEE ERC-20[89] token is minted by locking the NOFEE ERC-20 token in a vault.

Section 4.4.1 explains the rewards from the `Incentive` hook program that come to xNOFEE holders. Section 4.4.2 explains the mechanism for converting such rewards to NOFEE. Sections 4.4.3–4.4.6 explain the position handling mechanisms for the vault contract, as well as the rewards handling while the voting delegation is allowed, and provide the relevant contracts.

Draft - NoFeeSwap

4.4.1 Pool Growth Portion for xNOFEE Holders

A portion of the growth in pools with an `Incentive` hook goes to xNOFEE holders, i.e., 50% of the growth portion from the pools where both tokens are stable coins, i.e., USDC-USDT pairs, and, 20% of the growth portion from the other pools, i.e., ETH-WBTC & ETH-USDC & ETH-USDT & WBTC-USDC & WBTC-USDT pairs.[90] See Section 6.1.5.1 for information on the ownership and pool growth portion of pools with an `Incentive` hook.

Remark 64. *In return, the* `Incentive` *hook program distributes a large portion of the NOFEE tokens among the LP participants in these pools.*

[89] following ERC-4626 standard

[90] Governance/Timelock can change the specific portion amounts. See `modifyPoolGrowthPortion` function from `IncentivePoolFactory`.

4.4.2 Reward Conversion

Such growth is accumulated and converted to NOFEE through 4 initial pools:[91]

- NOFEE-ETH,
- NOFEE-WBTC,
- NOFEE-USDC,
- NOFEE-USDT,

Remark 65. *The pool specification for these initial conversion pools is:*

- *They have a ramp kernel, i.e, with kernel break-points* $\{(0,0),(q_{\text{spacing}},1)\}$.
- *They are owned by* `Timelock` *and their kernels are modifiable by governance.*
- *The* `incentivePoolFactory` *acts as a hook to temporarily disallow trades in the opposite direction of the NOFEE disbursements for the remainder of that block. See Section 6.2.*
- *Donate to them is not allowed.*
- *Their pool growth portion parameter is zero and cannot be modified in the future.*
- $q_{\text{spacing}} \sim 0.03$.[92]

Remark 66. *Any address can cover the gas costs to: a) call* `collectPool` *function from* `NofeeswapDelegatee` *to collect accumulated pool growth portion tokens of any number of pools with an* `Incentive` *hook, and then, b) call* `disburse` *function from* `IncentivePoolFactory` *to convert the accumulated tokens of a particular tag to NOFEE and disburse them to the* `XNofee` *contract.*

[91] Governance/Timelock can change the specific pools that are used for conversions. See `modifyConversionPools` function from `IncentivePoolFactory`.

[92] approximately 3% between the prices of consecutive ticks

Draft - NoFeeSwap

4.4.3 `XNofee` Contract

This contract locks NOFEE and mints xNOFEE for the owner of the deposited tokens. Upon withdrawal, the owner receives more NOFEE than initially deposited due to NOFEE disbursements from the `IncentivePoolFactory` contract to this contract.

`XNofee` contract follows ERC-4626 [10] vault. xNOFEE tokens are the shares and NOFEE tokens are the underlying assets. xNOFEE's decimal is set to 24, as opposed to NOFEE's decimal, which is 18. Therefore, in the ERC-4626 token [10], `decimalsOffset`$= 6$, and there are 1 virtual xNOFEE shares and 10^6 virtual NOFEE assets in the vault at the deployment.

Below are the new functions and overrides to ERC-4626:

- `trusteeOf`. Calculates the deterministic address of the trustee contract corresponding to each owner.
  - `owner`: The owner of the corresponding xNOFEEs.
  - [output] `trustee`: The trustee contract associated with the owner.
- `delegate`. Transfers the accrued assets of the given address to the corresponding trustee and delegates voting power to the given address.
  - `delegatee`: The address to be appointed as the `delegatee`.
- `transferFromTrustee`. Transfers all of the assets from the trustee contract back to `this`.
- `_update`. Transfers a `value` amount of tokens from `from` to `to`, or alternatively mints (or burns) if `from` (or `to`) is the zero address.

  Remark 67. *It is overridden in the ERC-20's implementation of this function to include a potential transfer from a* `XNofeeTrustee` *contract. It assures that the number of NOFEE in the* `XNofeeTrustee` *contract of the address*[93] *will not exceed the amount corresponding to their xNOFEE ownership.*

Remark 68. *When any account other than the* `XNofeePortal` *contract attempts to* `deposit`/`mint`*, it is thrown. The reason is that any newly minted xNOFEE should be withheld within the* `XNofeePortal` *contract for an initial*

[93] who is transferring xNOFEE to another address, or burning xNOFEE

Draft - NoFeeSwap

period and should not be rewarded by the NOFEE disbursements if they haven't waited for the entire initial period.

Remark 69. *If there are N deposited NOFEE and X minted xNOFEE in the ERC-4626 [10] vault, there are 3 possible modifications to the total amounts of it:*

- *Deposit n NOFEE & mint $x = n\frac{X}{N}$ xNOFEE:* (`deposit`/`mint`) *In this case, the depositor receives minted xNOFEE proportionally, i.e., $x = n\frac{X}{N}$, and the number of existing xNOFEE is increased to $X + x$. The price ratio stays unchanged.*
- *Burn x xNOFEE & withdraw $n = x\frac{N}{X}$ NOFEE:* (`withdraw`/`redeem`) *In this case, they withdraw NOFEE proportionally, i.e., $n = x\frac{N}{X}$, and the number of existing xNOFEE is reduced to $X - x$. The price ratio stays unchanged.*
- *Protocol to add n_{reward} NOFEE: When n_{reward} NOFEE reward is added, xNOFEE becomes more valuable, and its value changes from $\frac{N}{X}$ NOFEE per xNOFEE to $\frac{N+n_{\text{reward}}}{X}$ NOFEE per xNOFEE, i.e., the price ratio changes to a more valuable xNOFEE.*

Draft - NoFeeSwap

4.4.4 `XNofeeTrustee` Contract

If an xNOFEE owner decides to vote with the underlying NOFEE tokens that they are owed, a new instance of the `XNofeeTrustee` contract is deployed[94] to hold their underlying NOFEE, enabling the xNOFEE owner to delegate the voting power.

4.4.5 `XNofeePortal` Contract

It is an ERC-6909 token [11]. A feature of the `XNofee` vault is that nobody can `mint`/`deposit` in it directly except for the `XNofeePortal` contract. The purpose is that it holds onto the minted xNOFEE for around 2 months[95], which is regarded as the "maturity period". After this period, the owner can claim their xNOFEE and interact with the `XNofee` vault. However, before the end of the

[94] by `XNofee` contract on their behalf

[95] `portalCliff`$= 450,000$ blocks ($\sim$ 2 months)

maturity period, `XNofeePortal` keeps track of ownership via the ERC-6909 standard. The ERC-6909 `tokenId` encodes the following information:

- The "block number" at the time of minting is included in order to keep track of the maturity period.
- The numerator and denominator of the "NOFEE $<->$ xNOFEE" conversion rate at the time of minting are included to exclude any rewards in case of withdrawing/redeeming prior to the end of the maturity period.

A `XNofeePortal` ERC-6909 owner is able to partially burn their ownership, but cannot add to it. Any new minting will be a new separate position.

If the 2-month maturity period is over, the user cannot directly burn the ERC-6909 position. In that case, a `XNofeePortal` owner is required to return their ERC6909 token and receive `XNofee` ERC-20 using `transform` function.

It has the following external functions:

- `trusteeOf`. Calculates the deterministic address of the `XNofeePortalTrustee` contract corresponding to each owner.
  - `owner`: The owner associated with the trustee contract.
  - [output] `trustee`: The trustee contract associated with the owner.
- `previewDeposit`. Allows an on-chain or off-chain user to simulate the effects of their deposit at the current block, given current on-chain conditions.
  - `assets`: The number of assets to be deposited.
  - [output] `shares`: The resulting number of shares.
- `previewMint`. Allows an on-chain or off-chain user to simulate the effects of their mint at the current block, given current on-chain conditions.
  - `shares`: The number of shares to mint.
  - `assets`: The corresponding number of assets.
- `previewWithdraw`. Returns the number of shares burnt corresponding to an "early" withdrawal.
  - `id`: The `tokenId` to be withdrawn.

Draft - NoFeeSwap

  - `assets`: The number of assets to be deposited.
  - [output] `shares`: The resulting number of shares.

- `previewRedeem`. Returns the number of assets burnt corresponding to an "early" redeem.
  - `id`: The `tokenId` to redeem.
  - `shares`: The number of shares to be minted.
  - [output] `assets`: The corresponding amount of assets.

- `deposit`. Transfers the accrued assets of the given address to the corresponding trustee and delegates voting power to the given address.
  - `assets`: The number of assets to be deposited.
  - `receiver`: The recipient of the resulting multi-tokens.
  - [output] `id`: The resulting `tokenId`.
  - [output] `shares`: The resulting number of shares.

- `mint`. Mints exactly shares Vault `xNofee` shares by depositing `nofee`. The resulting `xNofee` are locked by this contract.
  - `shares`: The number of shares to be minted.
  - `receiver`: The recipient of the resulting multi-tokens.
  - [output] `id`: The resulting `tokenId`.
  - [output] `assets`: The corresponding amount of assets.

- `withdraw`. Burns shares from owner and sends exactly assets of `nofee` to receiver. The conversion rate at the time of minting is used. ("early" withdraw)
  - `id`: The tokenId to be withdrawn.
  - `assets`: The amount of assets to be withdrawn.
  - `receiver`: The recipient of the resulting assets.
  - `owner`: The owner of the multi-tokens to be withdrawn.
  - [output] `shares`: The number of shares to be withdrawn.

Draft - NoFeeSwap

- `redeem`. Burns exactly `xNofee` shares from owner and sends assets of `nofee` to receiver. The conversion rate at the time of minting is used. ("early" redeem)
  - `id`: The `tokenId` to be redeemed.
  - `shares`: The number of shares to be redeemed.
  - `receiver`: The recipient of the resulting assets.
  - `owner`: The owner of the multi-tokens to be redeemed.
  - [output] `assets`: The resulting amount of assets.

  Remark 70. *Early* `withdraw`/`redeem` *is performed through the* `XNofeePortal` *contract and the other ones are performed through* `XNofee` *contract.*

- `transform`. Unlocks `xNofee` from this contract's custody and sends them to the receiver.
  - `id`: The `tokenId` to be transformed to xNOFEE.
  - `shares`: The number of shares to be transformed.
  - `receiver`: The recipient of the resulting xNOFEE.
  - `owner`: The owner of the multi-tokens to be transformed.

  Remark 71. *When an account attempts to* `withdraw`/`redeem` *xNOFEE from* `XNofeePortal` *contract after the cliff period, it is thrown. It should release the xNOFEE tokens from* `XNofeePortal` *using* `transform` *function.*

- `delegate`. Transfers the accrued assets of the given address to the corresponding `XNofeePortalTrustee` contract and delegates voting power to the given address.
  - `delegatee`: The address to be appointed as the delegatee.

- `transferFromTrustee`. Transfers the `xNofee` from the trustee back to `address(this)`.

Draft - NoFeeSwap

4.4.6 `XNofeePortalTrustee` Contract

The voting power should be maintainable at all times, even during the initial period. To this end, ERC-6909 token owners have the option to deploy an `XNofeePortalTrustee` instance through which they can vote. An `XNofeePortalTrustee` contract associated with an ERC-6909 owner can possess all of the locked xNOFEE across all `tokenId`s of a particular owner.

The `XNofeePortal` contract is given full access to the xNOFEE withheld in instances of the `XNofeePortalTrustee` contract.

The xNOFEE are locked in this contract so that the owner can delegate their voting power to themselves or any other account. An instance of this contract can be deployed for every account whose xNOFEE are locked by the portal. The corresponding instance of this contract holds the locked xNOFEE amount and enables the owner to delegate their NOFEE voting power.

Draft - NoFeeSwap

5 Operator (optional tool)

An operator is provided to make a sequence of calls to the protocol. See `Operator.sol`. Any alternative operator can be used for the calls as well.

A sequence of compactly encoded instructions from the following list can be given to the operator contract to be executed in order. It is designed to give high flexibility to the users.

0. `PUSH0`: Places `0x0` in the transient storage slot `valueSlot` (clears it).

   `abi.encodePacked(uint8 action, uint8 valueSlot)`

1. `PUSH10`: Places `signextend(9, value)` in the transient storage slot `valueSlot`.

   `abi.encodePacked(uint8 action, int80 value, uint8 valueSlot)`

2. `PUSH16`: Places `signextend(15, value)` in the transient storage slot `valueSlot`.

   `abi.encodePacked(uint8 action, int128 value, uint8 valueSlot)`

3. `PUSH32`: Places `value` in the transient storage slot `valueSlot`.

   `abi.encodePacked(uint8 action, int256 value, uint8 valueSlot)`

4. `NEG`: Negates the content of the given transient storage slot, `valueSlot`, and inserts the outcome in `resultSlot`.

   `abi.encodePacked(uint8 action, uint8 valueSlot, uint8 resultSlot)`

5. `ADD`: Calculates the addition of two values, `value0Slot` and `value1Slot`, and inserts the outcome in `resultSlot`. Throws in case of overflow or underflow.

   `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

6. `SUB`: Calculates the subtraction of two values, `value0Slot` and `value1Slot`, and inserts the outcome in `resultSlot`. Throws in case of overflow or underflow.

   `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

7. `MIN`: Calculates the minimum of two signed integer values, `value0Slot` and `value1Slot`, and inserts the outcome in `resultSlot`.

   `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

8. `MAX`: Calculates the maximum of two signed integers, `value0Slot` and `value1Slot`, and inserts the outcome in `resultSlot`.

   `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

9. `MUL`: Calculates the multiplication of two values, `value0Slot` and `value1Slot`, and inserts the outcome in `resultSlot`. Throws in case of overflow or underflow.

   `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

Draft - NoFeeSwap

10. `DIV`: Calculates the division of `value0Slot` to `value1Slot`, and inserts the outcome in `resultSlot`. It is rounded towards the origin.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

11. `DIV_ROUND_DOWN`: Calculates the (round-down) division of `value0Slot` to `value1Slot`, and inserts the outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

12. `DIV_ROUND_UP`: Calculates the (round-up) division of `value0Slot` to `value1Slot`, and inserts the outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

13. `LT`: Checks if `value0Slot` is less than `value1Slot`, and inserts the boolean outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

14. `EQ`: Checks if `value0Slot` is equal to `value1Slot`, and inserts the boolean outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

15. `LTEQ`: Checks if `value0Slot` is less than or equal to `value1Slot`, and inserts the boolean outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

16. `ISZERO`: Checks if `valueSlot` is equal to zero, and inserts the boolean outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, int256 valueSlot, uint8 resultSlot)`

Draft - NoFeeSwap

17. `AND`: Calculates AND of two booleans, `value0Slot` and `value1Slot`, and inserts the resulting bits in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

18. `OR`: Calculates OR of two booleans, `value0Slot` and `value1Slot`, and inserts the resulting bits in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

19. `XOR`: Calculates XOR of two booleans, `value0Slot` and `value1Slot`, and inserts the resulting bits in `resultSlot`.

    `abi.encodePacked(uint8 action, uint8 value0Slot, uint8 value1Slot, uint8 resultSlot)`

20. `JUMP`: If the content of `conditionSlot` is non-zero, it jumps to the operation at byte `destination`. Throws if `destination` refers to anything but a `JUMPDEST`.

    `abi.encodePacked(uint8 action, uint16 destination, uint8 conditionSlot)`

21. `JUMPDEST`: Marks a valid destination for `JUMP` and has no other effect.

    `abi.encodePacked(uint8 action)`

22. `READ_TRANSIENT_BALANCE`: Reads a tag's transient balance of an address in the protocol, and inserts the outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, Tag tag, address owner, uint8 resultSlot)`

23. `READ_BALANCE_OF_NATIVE`: Reads the native balance of an address in the protocol, and inserts the outcome in `resultSlot`.

    `abi.encodePacked(uint8 action, address owner, uint8 resultSlot)`

Draft - NoFeesSwap

24. `READ_BALANCE_OF_ERC20`: Reads an ERC-20 balance of an address in the protocol, and inserts the outcome in `resultSlot` (together with the `successSlot` flag that shows if the action succeeded).[96]

    `abi.encodePacked(uint8 action, address token, address owner, uint8 successSlot, uint8 resultSlot)`

25. `READ_BALANCE_OF_MULTITOKEN`: Reads a multi-token balance of an address in the protocol, and inserts the outcome in `resultSlot` (together with the `successSlot` flag that shows if the action succeeded).

    `abi.encodePacked(uint8 action, address token, uint256 id, address owner, uint8 successSlot, uint8 resultSlot)`

26. `READ_ALLOWANCE_ERC20`: Reads the allowance of an ERC-20 of an address given to a spender in the protocol.

    `abi.encodePacked(uint8 action, address token, address owner, address spender, uint8 successSlot, uint8 resultSlot)`

27. `READ_ALLOWANCE_PERMIT2`: Reads the ERC-20 allowance granted by `owner` to `spender` via `permit2`.

    `abi.encodePacked(uint8 action, address token, address owner, address spender, uint8 resultSlot)`

28. `READ_ALLOWANCE_ERC6909`: Reads the allowance of an ERC-6909 of an address given to a spender in the protocol.

    `abi.encodePacked(uint8 action, address token, uint256 id, address owner, address spender, uint8 successSlot, uint8 resultSlot)`

29. `READ_IS_OPERATOR_ERC6909`: Reads a spender's operator status for an ERC-6909 token of the owner.

    `abi.encodePacked(uint8 action, address token, address owner, address spender, uint8 successSlot, uint8 resultSlot)`

[96] The `resultSlot` is from the contract, and the `successSlot` is from EVM.

Draft - NoFeeSwap

30. `READ_IS_APPROVED_FOR_ALL_ERC1155`: Reads if a spender is an approved spender for an ERC-1155 token of the owner.

    `abi.encodePacked(uint8 action, address token, address owner, address spender, uint8 successSlot, uint8 resultSlot)`

31. `READ_DOUBLE_BALANCE`: Read the singleton `(tag0, tag1)` double balance of an owner.

    `abi.encodePacked(uint8 action, Tag tag0, Tag tag1, address owner, uint8 value0Slot, uint8 value1Slot)`

32. `WRAP_NATIVE`: Wraps `ETH` to `WETH`.

    `abi.encodePacked(uint8 action, uint8 valueSlot, uint8 successSlot)`

33. `UNWRAP_NATIVE`: Unwraps `WETH` to `ETH`.

    `abi.encodePacked(uint8 action, uint8 amountSlot, uint8 successSlot)`

34. `PERMIT_PERMIT2`: Grants `permit2` allowance to `spender` for a given `amount` of the owner's token via the owner's EIP-712 signature.

35. `PERMIT_BATCH_PERMIT2`: Grants `permit2` allowance for a batch of tokens owned by `owner` to `spender` via the owner's EIP-712 signature.

36. `TRANSFER_NATIVE`: Transfers an amount of the native token to an address.

    `abi.encodePacked(uint8 action, address to, uint8 amountSlot, uint8 successSlot)`

37. `TRANSFER_FROM_PAYER_ERC20`: Transfers an amount of an ERC-20 token to an address, together with a flag that shows if the action succeeded.

    `abi.encodePacked(uint8 action, address token, uint8 amountSlot, address to, uint8 successSlot, uint8 resultSlot)`

Draft - NoFeeSwap

38. `TRANSFER_FROM_PAYER_PERMIT2`: Transfers an amount of an ERC-20 token to an address through Permit2.

    `abi.encodePacked(uint8 action, address to, uint8 amountSlot, address token, uint8 successSlot)`

39. `TRANSFER_FROM_PAYER_ERC6909`: Transfers an amount of an ERC-6909 token to an address.

    `abi.encodePacked(uint8 action, address token, uint256 id, address to, uint8 amountSlot, uint8 successSlot, uint8 resultSlot)`

40. `SAFE_TRANSFER_FROM_PAYER_ERC1155`: Transfers an amount of an ERC-1155 token to an address.

    `abi.encodePacked(uint8 action, address token, uint256 id, address to, uint8 amountSlot, uint8 successSlot, uint24dataByteCount, bytesdata)`

41. `CLEAR`: Clears an amount of the operator's positive transient balance in tag, i.e., gives up that amount without taking it back.

    `abi.encodePacked(uint8 action, Tag tag, uint8 amountSlot, uint8 successSlot)`

42. `TAKE_TOKEN`: Takes an amount of an ERC-20 token from the protocol (transient balance) and gives it to a specified address.

    `abi.encodePacked(uint8 action, address token, address to, uint8 amountSlot, uint8 successSlot)`

43. `TAKE_ERC6909`: Takes an amount of an ERC-6909 token from the protocol (transient balance) and gives it to a specified address.

    `abi.encodePacked(uint8 action, address token, uint256 id, address to, uint8 amountSlot, uint8 successSlot)`

Draft - NoFeeSwap

44. `TAKE_ERC1155`: Takes an amount of an ERC-1155 token from the protocol (transient balance) and gives it to a specified address.

    `abi.encodePacked(uint8 action, address token, uint256 id, address to, uint8 amountSlot, uint8 successSlot, uint24dataByteCount, bytesdata)`

45. `SYNC_TOKEN`: Syncs an ERC-20 token.

    `abi.encodePacked(uint8 action, address token)`

46. `SYNC_MULTITOKEN`: Syncs a multi-token.

    `abi.encodePacked(uint8 action, address token, uint256 id)`

47. `SETTLE`: Should be used after ERC-20, ERC-6909, or ERC-1155 transfers to `Nofeeswap` in order to update the operator's transient balances. Can be used to send a value of native tokens to the protocol to settle the native transient balance of the operator.

    `abi.encodePacked(uint8 action, uint8 valueSlot, uint8 successSlot, uint8 resultSlot)`

48. `TRANSFER_TRANSIENT_BALANCE`: Transfers a positive amount of transient balances of the operator to a receiver.

    `abi.encodePacked(uint8 action, Tag tag, address receiver, uint8 amountSlot, uint8 successSlot)`

49. `TRANSFER_TRANSIENT_BALANCE_FROM_PAYER`: Transfers a positive amount of transient balances of the payer to a receiver. Can be used to settle on behalf of other accounts.

    `abi.encodePacked(uint8 action, Tag tag, address receiver, uint8 amountSlot, uint8 successSlot)`

50. `MODIFY_SINGLE_BALANCE`: Modifies (in either direction) the corresponding singleton balance of a tag by a specified amount.

    `abi.encodePacked(uint8 action, Tag tag, uint8 amountSlot, uint8 successSlot)`

Draft - NoFeeSwap

51. `MODIFY_DOUBLE_BALANCE`: Modifies (in either direction) the corresponding singleton double balance of a tag-pair by a specified pair of amounts.

    `abi.encodePacked(uint8 action, Tag tag0, Tag tag1, uint8 amountSlot0, uint8 amountSlot1, uint8 successSlot)`

52. `SWAP`: Performs a `swap` by taking the inputs.

    `abi.encodePacked(uint8 action, uint256 poolId, uint8 amountSpecifiedSlot, uint64 limitOffsetted, uint8 zeroForOne, uint8 crossThresholdSlot, uint8 successSlot, uint8 amount0Slot, uint8 amount1Slot, uint16 hookDataBytesCount, bytes hookData)`

53. `MODIFY_POSITION`: Performs a `modifyPosition` by taking the inputs.

    `abi.encodePacked(uint8 action, uint256 poolId, uint64 qMinOffsetted, uint64 qMaxOffsetted, uint8 sharesSlot, uint8 successSlot, uint8 amount0Slot, uint8 amount1Slot, uint16 hookDataBytesCount, bytes hookData)`

54. `DONATE`: Performs a `donate` by taking the inputs.

    `abi.encodePacked(uint8 action, uint256 poolId, uint8 sharesSlot, uint8 successSlot, uint8 amount0Slot, uint8 amount1Slot, uint16 hookDataBytesCount, bytes hookData)`

55. `QUOTE_SWAP`: Similar to swap, but it only quotes.

56. `QUOTE_MODIFY_POSITION`: Similar to modifyPosition, but it only quotes.

57. `QUOTE_DONATE`: Similar to donate, but it only quotes.

58. `QUOTE_TRANSIENT_ACCESS`: Gets access to the quoter transient storage.

    `encodePacked(uint8 action, bytes32 transientSlot, uint8 resultSlot)`

Draft - NoFeeSwap

59. `REVERT`: Reverts with the current content of `returndata` as the reason.

Remark 72. *When* `INofeeswap.unlock` *is called with* `Operator.sol` *as* `unlockTarget`*, a sequence of instructions is encoded as* `data` *and passed to* `IUnlockCallback(operator).unlockCallback(caller,data)`*, such as* `swap` *and* `modifyPosition` *actions. See Remark 51.*

Among those instructions, one may include a `Callback` *action that invokes* `IOperatorCallback(caller).operatorCallback(previousReturnData)` *by* `Operator.sol` *where* `previousReturnData` *are the return data of the previous call among the sequence of calls.*

For example, `caller` *may encode a* `QuoteModifyPosition` *action and a* `Callback` *action immediately after that to receive the output of the quote as a callback. To do this,* `caller` *must implement the following* `operatorCallback` *function.*

Draft - NoFeeSwap

6 Pre-built Hooks

The first in the following is the `Incentive` hook contract to reward the LPs of the subscribing pools with NOFEE tokens.

6.1 `Incentive` Hook Contract

During pool initialization, the pools of token pairs mentioned in Section 6.1.1 have the choice to use the incentive hook of that specific token pair.

6.1.1 NOFEE Token Rewards by Hook/Pair

An initial distribution of 20% of the total 10 billion NOFEE tokens is performed through several incentive hooks, each distributing to pools of a particular pair. The reward is divided into 6 separate parts:

- ETH - USDC $\rightarrow$ 6% of NOFEE supply (600 million NOFEE)
- ETH - USDT $\rightarrow$ 4% of NOFEE supply (400 million NOFEE)
- WBTC - USDC $\rightarrow$ 4% of NOFEE supply (400 million NOFEE)
- ETH - WBTC $\rightarrow$ 2% of NOFEE supply (200 million NOFEE)

- WBTC - USDT $\rightarrow 2\%$ of NOFEE supply (200 million NOFEE)
- USDC - USDT $\rightarrow 2\%$ of NOFEE supply (200 million NOFEE)

Figure 24 shows how the total reward for each pair of tokens is distributed over 60 months, and the periodic rewards decrease linearly over time. The first payment is around 1.5 months after the incentive program's start date. The final incentive distribution date is around 5 years after the incentive program's start date.

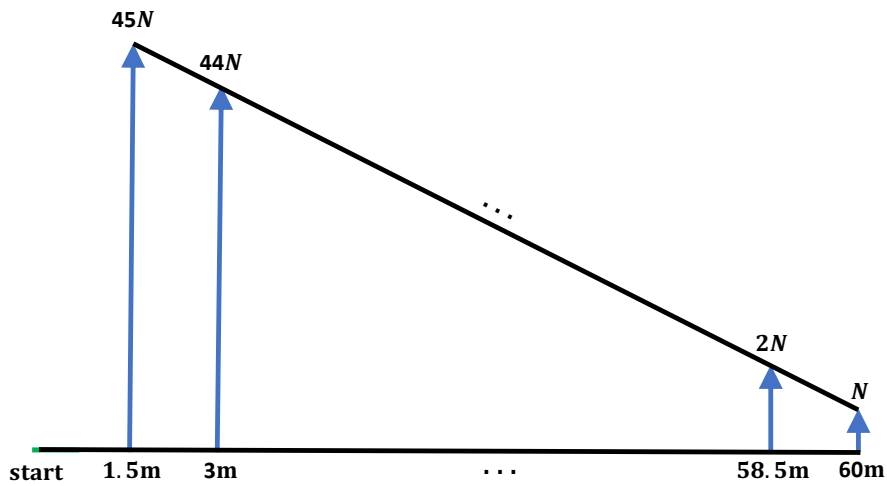


Figure 24: Reward ratios of each hook over 60 months. For the hook (token pairs) that the total reward is 600 million NOFEE tokens, $N + 2N + 3N + \cdots + 45N = 1035N = 6 \times 10^8$, and therefore, $N \sim 580$ thousand NOFEE tokens. For hooks (token pairs) that the total reward is 400 million NOFEE tokens, $N + 2N + 3N + \cdots + 45N = 1035N = 4 \times 10^8$, and therefore, $N \sim 386$ thousand NOFEE tokens. For hooks (token pairs) that the total reward is 200 million NOFEE tokens, $N+2N+3N+\cdots+45N = 1035N = 2\times10^8$, and therefore, $N \sim 193$ thousand NOFEE tokens.

Draft

6.1.2 Each LP's Weight in Earning Evanescent Points

Each incentive hook distributes NOFEE rewards to LPs of their pools. *Evanescent point* is the criteria for how much NOFEE rewards each LP can collect. At any point in time, if an address has $x\%$ of a particular incentive hook's outstanding

evanescent points, they can collect $x\%$ of NOFEE rewards that are available for collection.

For each block, LPs of the active liquidity interval for each incentive hook earn rewards with evanescent points proportional to the following value per share:[97]

$$\frac{\texttt{growth} \times \sqrt{\texttt{integral0} \times \texttt{integral1}}}{\texttt{outgoingMax}}. \tag{79}$$

Remark 73. *For each LP, the parameters in equation* (79) *are from their corresponding pool. For example, if at some block, the value of this parameter for the pool X is twice that of the pool Y, the LPs in the active interval of the pool X earn twice as many rewards per share per block compared to the LPs in the active interval of pool Y.*

Remark 74. *As in Figure 24, NOFEE distribution is initially higher (and distributed among a smaller number of LPs as the number of LPs is smaller early on). Therefore, there is more incentive to participate as an LP in the earlier stages. The weight of the evanescent points distributed does not change with time.*

Remark 75. *These rewards are on the hook contract level. All LPs of a particular incentive hook (token-pair), from all such pools, earn evanescent points of that hook contract.*

Draft - NoFeeSwap

6.1.3 Each LP's Total Evanescent Points

At any moment, the total number of evanescent points owed to a position is equal to:

$$\begin{aligned}&\texttt{evanescentPointsOwed} + \texttt{shares} \times \\ &\quad (\texttt{evanescentPointsPerShareCurrent} - \\ &\qquad\qquad \texttt{evanescentPointsPerShareSubtrahend}),\end{aligned} \tag{80}$$

where:

- `evanescentPointsOwed` is evanescent points accrued since the creation of this incentive position until the last time it is touched. Collecting rewards and modifying positions affects this value.

[97] See pool's dynamic parameters in Section 3.2.2 and static parameters in Section 3.2.3.

- `evanescentPointsPerShareSubtrahend` is the snapshot of evanescent points per share (a cumulative measure) at the last time that the position is touched, and if it is not touched since its initial mint, at the moment when it is created, summed up over the position's every interval.

- `evanescentPointsPerShareCurrent` is evanescent points per share (a cumulative measure) at the moment when equation (80) is calculated, summed up over the position's all intervals.

- `shares` represents the position's number of shares per liquidity interval. For example, for a position that has 5 shares in each of several consecutive liquidity intervals, $\texttt{shares} = 5$.

See Section 6.1.5.2 for information on the ownership handling of LP tokens.

6.1.4 Each LP's NOFEE Reward Amount

Rewards can be collected at any time. However, the timing of withdrawals can affect the collected amount. LPs who collect their rewards immediately after a NOFEE deposit to the corresponding hook contract are typically rewarded more.[98] This is because as time passes, incentive LPs earn more evanescent points and each evanescent point corresponds to a lower amount of NOFEE token. However, when there is a NOFEE deposit from the incentive program, each evanescent point mapping jumps to a higher amount of NOFEE token.

Remark 76. *The name evanescent for these rewards is to illustrate the benefit of early withdrawals, as they are earned by the community continuously and LPs should take their corresponding NOFEE when there is a NOFEE deposit by an incentive hook before the dilution of evanescent points.*

Any LP can take NOFEE at any time in return for their evanescent points. NOFEE rewards for an LP are calculated as follows:

$$\texttt{rewardAmount} = \texttt{totalNofeeAllowance} \times \frac{\texttt{evanescentPointsCollected}}{\texttt{totalEvanescentPointsOwed}}, \tag{81}$$

where:

[98] Each incentive hook is scheduled with receiving several NOFEE reward deposits as in Figure 24.

Draft - NoFeeSwap

- `totalNofeeAllowance` is the total NOFEE amount available up for grabs for evanescent point holders of this hook contract.
- `evanescentPointsCollected` is the number of burned evanescent points.
- `totalEvanescentPointsOwed` is the total number of evanescent points owed to the community by this hook.

6.1.5 Other Design Considerations

Incentive pools have additional calculations beyond the ones by the protocol, and therefore, require modified functions for some of their calls. See Section 6.1.7 for specific explanations on each function.

`IncentivePoolFactory` contract maintains the pools, and actions such as `initialize` and `modifyKernel` are through them. As in Figure 25, for each token-pair incentive program, there is an `Incentive` contract that manages the rewards.

FeeSwap

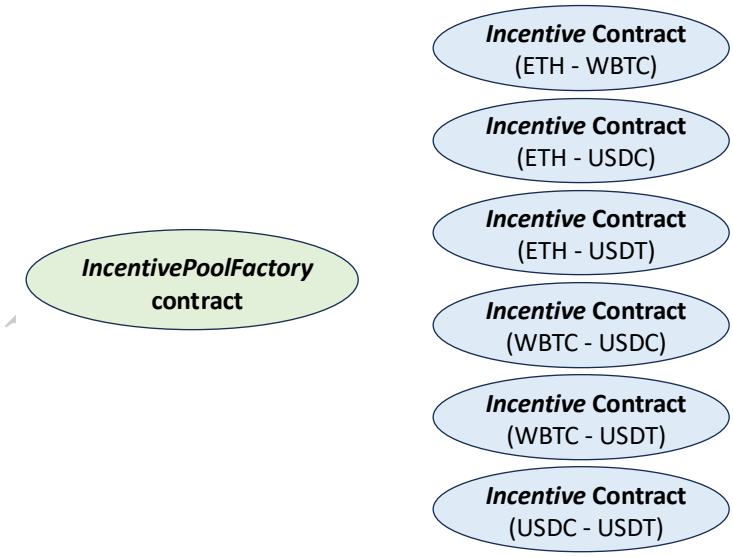


Figure 25: The contracts that are specifically for the incentive programs.

6.1.5.1 Ownership of Incentive Pools. `IncentivePoolFactory` contract maintains the ownership of the pools with an incentive hook, and the creators of the pools cannot set or collect the pool growth portion.

Section 4.4.1 explains that poolGrowthPortion is taken for distribution to xNOFEE holders. This is done through `IncentivePoolFactory` that maintains the ownership of the pools with an incentive hook.

However, through an NFT that pool creators receive at pool creation, they can maintain the right to modify the kernel in the future if the corresponding flag in `poolId` is set in initialization.

6.1.5.2 Ownership of LP tokens. When there is no hook, the LP position's corresponding single balance token is given to the caller as in Remark 50.

However, the ownership of LP tokens of pools with an incentive hook stays with `Incentive` contract. And, in return, it issues an NFT for the LP, such that the liquidity position together with the evanescent points, discussed in Section 6.1.2, is assigned to the LP.

This mechanism facilitates keeping track of the evanescent points in the incentive contract.[99]

Draft - NoFeeSwap

6.1.6 `Incentive` Contract's Parameters

6.1.6.1 Pool-level Parameters. They are subject to updates with the pool's activity, e.g., swap, modifyPosition, and donate.

- `block`. Last block number where the pool is touched.
- `lower`. Most recent `lower` boundary of the active liquidity interval.
- `upper`. Most recent `upper` boundary of the active liquidity interval.
- `activeEvanescentPointsPerShare`. Evanescent points per share for the active liquidity interval.

6.1.6.2 Position-level Parameters. They are subject to update with the position's change, i.e., mint, burn, and collection of NOFEE rewards.

- `poolId`. The `poolId` for the corresponding position.

[99] Note that these NFTs cannot be traded in the protocol. See Remark 26.

- `logPriceMinOffsetted`. Left boundary of the corresponding position.
- `logPriceMaxOffsetted`. Right boundary of the corresponding position.
- `shares`. The number of shares per liquidity interval for the corresponding position.
- `evanescentPointsPerShareSubtrahend`. The evanescent points per share at the time when this position is created or the last time it is touched (collected or modified).
- `evanescentPointsOwed`. Evanescent points accrued since the creation of this incentive position until the last time it is touched (collected or modified).

6.1.6.3 Evanescent points per share. The value of `evanescentPointsPerShare(q)` mapping in a pool is equal to:[100]

- If a spaced log-price q is less than or equal to q_{lower}: The total evanescent points per single share for a position from $-\infty$ to q.
- If a spaced log-price q is more than or equal to q_{upper}: The total evanescent points per single share for a position from q to $+\infty$.

6.1.7 External Functions

For most functionalities, the external functions in the core[101] should also be used for pools with incentive hooks. However, for two pool management functions there are differences, i.e., for `initialize`, `modifyKernel` and `modifyPoolGrowthPortion` calls `IncentivePoolFactory` contract should be used. We will explain these in the following.

[100] This mapping is quite similar to the growth multiplier in Section 3.3.2.
[101] i.e., `Nofeeswap.sol` and `NofeeswapDelegatee.sol`

Draft - NoFeeSwap

6.1.7.1 External functions of `IncentivePoolFactory` contract.

- `modifyPoolGrowthPortion`. Sets new value for `poolGrowthPortion` for each of the given tag pairs. It should be run by the admin, currently `Timelock`, i.e., it is modifiable by governance.
  - `tag0/1`: The tag-pair to modify the parameter for.
  - `newPoolGrowthPortion`: The new pool growth portion.

  Remark 77. *For pools with an incentive hook, the* `IncentivePoolFactory` *contract maintains the pool ownership, and the poolGrowthPortion parameter can be modified for each tag-pair through* `modifyPoolGrowthPortion` *of the* `IncentivePoolFactory` *contract by governance.*

- `initialize`. Initializes a pool and mints an NFT.
  - `unpepperedPoolId`
  - `tag0/1`
  - `kernelCompactArray`
  - `curveArray`

  Remark 78. *The most significant 68 bits of the* `unsaltedPoolId` *do not impact any of the pool's attributes.* `unsaltedPoolId` *is an* `uint256` *input parameter of the initialization method of* `NofeeswapDelegatee.sol`*. The* `poolId` *is derived from* `unsaltedPoolId` *and* `msg.sender` *via the following deterministic formula:*[102]

$$
\begin{aligned}
\texttt{poolId} = \texttt{unsaltedPoolId} + (&\\
&\texttt{keccak256}(\\
&\quad \texttt{abi.encodePacked}(\\
&\quad\quad \texttt{address}(\texttt{IncentivePoolFactory}),\\
&\quad\quad \texttt{unsaltedPoolId}\\
&\quad)\\
&) << 188,
\end{aligned}
\tag{82}
$$

[102] Section Section 3.2.1.2.

Draft - NoFeeSwap

where:

$$\begin{aligned}\texttt{unsaltedPoolId} = \texttt{unpepperedPoolId} \; + (\\ \texttt{keccak256}(\\ \texttt{abi.encodePacked}(\\ \texttt{msg.sender},\\ \texttt{unpepperedPoolId}\\ )\\ ) << 188,\end{aligned} \tag{83}$$

where '+' denotes addition modulo 2^{256} *and the '<<' is equivalent to the EVM opcode* `shl`*. This way,* `poolId` *can be determined/mined a priori.*

After initialization, `poolId` *is used as the sole identifying parameter of every pool for all purposes.*

Remark 79. *To initialize pools with incentive hooks, the guidelines in Table 4 must be followed. Some relevant comments:*

- *The required values for the hook flags (top* 17 *in Table 4) are usually specified by the hook. For example, this hook only records some information from pool activities and needs* 5 *hook flags.*
- *As in Section 6.1.5.1, the pool creators do not take any pool growth portion from such pools. And, this parameter must be mutable to allow governance/Timelock to determine this parameter for these pools.*

Draft - NoFeeSwap

- `modifyKernel`. Accepts a new pending kernel for pools that are initialized by this contract. Can be called by the NFT owner or their operator only. See Section 6.1.5.1. At initialization, the `modifyKernel` flag should be turned on for this to be applicable.
  - `poolId`
  - `kernelCompactArray`
- `setAdmin`. It sets a new admin for the contract. It should be run by the admin, currently `Timelock`, i.e., it is modifiable by governance.
  - `NewAdmin`: Address of the new admin.

Table 4: Permitted flag values for initialization of pools with an `Incentive` hook.

Flag	**Value**
isPreInitialize	1
isPostInitialize	0
isPreMint	0
isMidMint	1
isPostMint	0
isPreBurn	0
isMidBurn	1
isPostBurn	0
isPreSwap	0
isMidSwap	1
isPostSwap	0
isPreDonate	0
isMidDonate	1 (iff donate allowed)
isPostDonate	0
isPreModifyKernel	0
isMidModifyKernel	0
isPostModifyKernel	0
isMutableKernel	0 or 1
isMutablePoolGrowthPortion	1
isDonateAllowed	0 or 1

Draft NoFeeSwap

- `modifyConversionPools`. Modifies or adds conversion pools. The pool should be NOFEE versus another token. It should be run by the admin, i.e., it is modifiable by governance.
  - `poolIds`: The selected pools.

Remark 80. *The conversion pools use* `IncentivePoolFactory` *as hook. See Section 6.2.*

Remark 81. *The current NOFEE reward conversion pools are owned by* `Timelock`*, and their kernels are modifiable by governance. See the specifications of the current conversion pools in Section 4.4.2.*

Remark 82. *When this function is called, if a conversion pool for that tag-pair is already set, then that older pool is replaced with a new one.*

- `updatePoolGrowthPortion`. Syncs the pool growth portion for a previously initialized pool with the current pool growth portion of this contract. It is needed as the change of global values happens in batches by governance, per tag-pair, and each pool should be synced with it separately. It can be run by any address.
  - `poolId`
- `disburse`. Can be called by any address to convert `IncentivePoolFactory` contract's collected poolGrowthPortion of any particular tag to NOFEE tokens and transfer them to `XNofee` contract, i.e., disburse to xNOFEE holders.[103]
  - `tag`: The tag value to be converted to NOFEE and disbursed.
  - [output] `amount`: The current balance of `this` with respect to `tag`.
  - [output] `nofeeAmount`: The resulting amount of NOFEE.

Remark 83. *For each incentive pool, as in no-hook pools,* `collectPool` *should be called to collect poolGrowthPortion. For incentive hooks, the amount collected from each tag goes to the* `IncentivePoolFactory` *contract. Such amounts collected in the* `IncentivePoolFactory` *contract can be given to xNOFEE holders, by anyone through* `disburse`.

`IncentivePoolFactory` has the following public parameters.

- `nofeeswap`. NoFeeSwap's contract address.
- `nofeeToken`. ERC-20 address of the reward token to be distributed.
- `XNofee`. The address of `xNOFEE` stacking contract.
- `admin`. The admin of the contract, who can change the admin, modify the pool growth portion, or modify or switch conversion pools.

[103] There is a minimum gap of `disburseGap` (number of blocks) between any two consecutive disbursements of any tag.

Draft - NoFeeSwap

- `poolGrowthPortion`. The portions for every tag-pair that is initialized via this contract.
  - `tag0/1`: The tags to obtain their corresponding pool growth portion.
- `conversionPools`. The mapping of pools that are used to convert tokens to `noFeeToken`.
  - `tag`: The tag clarifies which pool is intended.

Remark 84. *The other parameters of* `IncentivePoolFactory` *contract,* `disburseGap` *and* `delay`*, together with their corresponding setter functions, are discussed in Section 6.2.1.*

6.1.7.2 External functions of each `Incentive` contract. There are a few functions that are specific to the pools which are in the `Incentive` program.

- `collect`. It collects NOFEE rewards for an LP. It should be run by the NFT holder of an LP position of this incentive hook.
  - `tokenId`: The corresponding `tokenId` of an LP position.
  - [output] `amount`: The amount of rewards collected.
- `preInitialize`. The pool requirements are verified, `IncentivePoolFactory` is specified as the pool owner and the pool's NOFEE reward parameters are set.
- `midMint`. For a mint, a NoFeeSwap position is created here based on the given inputs and a corresponding NFT is minted for the owner. The user will owe the corresponding tokens. The caller needs to include additional actions to settle. See Operator.
- `midBurn`. For a burn, a NoFeeSwap position is burned here based on the given inputs and a corresponding NFT is burned for the owner. The user will be owed the corresponding tokens. The caller needs to include additional actions to settle. See Operator.

Remark 85. *For* `modifyPosition`, `Incentive` *contract exchanges its expected position and performs its NFT exchange in* `midMint`/`midBurn`. *All of its corresponding debit or credit comes to the singleton accounting*

Draft - NoFeeSwap

for the caller to settle on its behalf. Then, the caller can handle and settle its mint/burn actions together with all the other actions using tools like the operator discussed in Section 5.

Note that the same `modifyPosition` *function of* `NofeeswapDelegatee` *is used for pools with an incentive hook.*

- `midSwap`. Evanescent points accounting gets updated.
- `midDonate`. Evanescent points accounting gets updated.

Remark 86. *This contract inherits Uniswap v4's* `Multicall_v4` *and* `ERC721Permit_v4`.

6.2 `IncentivePoolFactory` Hook Contract

As discussed in Section 6.1, `IncentivePoolFactory` takes ownership of every `Incentive` pool. Additionally, `IncentivePoolFactory` acts as a hook for every conversion pool that is used to transform the accrued rewards of `Incentive` pools to NOFEE. The following requirements are enforced on the initial conversion pools:

- disallow the trades in the opposite direction of the disbursements by a number block given by the `delay` parameter after each disbursement.[104] The number of blocks, `delay`, is modifiable by the admin.
- disallow any disburse earlier than `disburseGap`$= 50,000$ blocks (~ 1 week) following a prior disburse.

Remark 87. *The* `preInitialize` *and* `midSwap` *flags are mandatory for the* `IncentivePoolFactory` *hook.*

6.2.1 Functions Applicable to the Hook Application

- `setDisburseGap`: Sets a new value for `disburseGap`. Can be called by admin only.
  - `_disburseGap`: The new `disburseGap` value.
- `setDelay`: Sets a new value for `delay`. Can be called by admin only.
  - `_delay`: The new `delay` value.

[104] `delay` is initially set to 0, i.e., only in the remainder of the block after the disbursement such opposite direction trades are disallowed.

Draft - NoFeeSwap

6.3 `Oracle` Hook Contract

There is a hook developed for the price oracle. The oracle hook gives access to the time weighted geometric mean of price (weighted arithmetic mean of log-price) based on the following formula:

$$\texttt{logPriceCumulative} = \sum_{k=\texttt{initialBlock}}^{\texttt{lastObservationBlock}} (time(k) - time(k-1)) * q(k), \tag{84}$$

where:

- $time(k)$ denotes the timestamp corresponding to block k;
- $q(k)$ denotes the log-price value at the end of block k.
- `lastObservationBlock`: Block of the latest recorded data.
- `initialBlock`: Block of the very first recorded data.

Observations are stored in an observation array. Each observation consists of the following information:

- An index
- The length of the observation array at the time of observation.
- Timestamp at the time of observation.
- `logPriceCumulative` based on the above formula.

The very first slot of the observation array stores the last observation. This includes the length of the array as well. Hence, one can access the last observation by reading a single slot. The next slot stores observation index zero and so on. Each observation is written both on the first slot and the slot corresponding to its *index* which is equal to *index % length*.

Any address can pay the gas to expand the observation array for any pool. The corresponding slots are then populated, and the length of the observation array increases once for each observation until it reaches the end of the expanded array.

- `lastObservation`: Reads the most recent oracle observation from storage.

Draft - NoFeeSwap

  - `poolId`: The corresponding poolId.
  - [output] `index`: The index of the last observation.
  - [output] `length`: The total length of the observation array.
  - [output] `blockTimeStamp`: Timestamp associated with the last observation.
  - [output] `logPriceCumulative`: The cumulative logPrice value.

- `observation`: Reads the oracle observation corresponding to the given index.
  - `poolId`: The corresponding poolId.
  - `index`: The corresponding index.
  - [output] `blockTimeStamp`: Timestamp associated with this observation.
  - [output] `logPriceCumulative`: The cumulative logPrice value.

- `grow`: Expands the observation array. Any address can run and expand the length.
  - `poolId`: The corresponding poolId.
  - `newLength`: The new length for the observation array.

The hook functions are as below. See Table 5.

- `postInitialize`: The very first observation is written.
- `midSwap`: A new observation is written if it is the first swap of a block.

Draft - NoFeeSwap

References

[1] "NoFeeSwap Litepaper," 2025, [Access].

[2] R. Hanson, "Logarithmic markets scoring rules for modular combinatorial information aggregation," *The Journal of Prediction Markets*, vol. 1, no. 1, pp. 3–15, 2007.

[3] "Uniswap v3 Whitepaper," 2021, [Access].

Table 5: Permitted flag values for initialization of pools with an `Oracle` hook.

Flag	**Value**
isPreInitialize	0
isPostInitialize	1
isPreMint	0
isMidMint	0
isPostMint	0
isPreBurn	0
isMidBurn	0
isPostBurn	0
isPreSwap	0
isMidSwap	1
isPostSwap	0
isPreDonate	0
isMidDonate	0
isPostDonate	0
isPreModifyKernel	0
isMidModifyKernel	0
isPostModifyKernel	0
isMutableKernel	0 or 1
isMutablePoolGrowthPortion	0 or 1
isDonateAllowed	0 or 1

Draft - NoFeeSwap

[4] "Curve v1 Whitepaper," 2021, [Access].

[5] "Balancer v1 Whitepaper," 2019, [Access].

[6] "Uniswap v2 Whitepaper," 2020, [Access].

[7] "NoFeeSwap GitHub," 2025, [Access].

[8] "Compound Governor Bravo," 2022, [Access].

[9] "Sushiswap Whitepaper," 2020, [Access].

[10] "OpenZeppelin ERC-4626," 2024, [GitHub].

[11] "Solmate ERC-6909," 2023, [GitHub].